MASTER OVERVIEW — 25 AI FEATURES PRODUCTIVITY PLATFORM

READ THIS FIRST — Then Build Sprint by Sprint

HOW TO USE THESE FILES

This project is split into **sprint files** to save credits and reduce errors.

DO NOT try to build everything at once.

BUILD ORDER:

1. Read this overview file completely
2. Then paste Sprint 1 → build it → verify it works
3. Then paste Sprint 2 → build it → verify it works
4. Continue through Sprint 10
5. Finish with Sprint 11 (polish)

RULE: Never start a new sprint until the current one compiles and runs without errors.

EXISTING PLATFORM (ALREADY BUILT)

The following modules ALREADY exist. Do NOT rebuild them. The AI features will integrate with them.

```
EXISTING MODULES:
├── Authentication (NextAuth.js — Google OAuth, email/password, magic link)
├── Workspaces (multi-tenant, roles: owner/admin/editor/viewer/guest)
├── Task Management (databases with table/board/calendar/gallery/list/timeline views)
├── Documents (Tiptap block editor, nested pages, real-time collab via Yjs)
├── Messaging (channels: public/private/DM, threads, reactions, files, Socket.io)
├── Meetings (scheduling, audio recording, Whisper transcription)
```

```
├── Calendar (Google Calendar two-way sync)
└── Focus Tools (Pomodoro timer, focus sessions)
```

TECH STACK (MANDATORY — Do Not Change)

```
Frontend:        Next.js 14+ (App Router), TypeScript strict mode, Tailwind CSS, Shadcn/UI
State:           Zustand + React Query (TanStack Query v5)
Backend:         Next.js API Routes + tRPC v11
Database:        PostgreSQL + Prisma ORM + pgvector extension
Real-Time:       Socket.io with Redis adapter
AI:              OpenAI API (GPT-4o / GPT-4o-mini), provider-agnostic service layer
Vector Search:   pgvector (1536-dimension embeddings)
File Storage:    Cloudflare R2 (S3-compatible)
Cache:           Redis (Upstash)
Background Jobs: BullMQ with Redis
Deployment:      Replit Deployments
```

CODING STANDARDS (APPLY TO EVERY FILE)

1. TypeScript strict mode. Zero "any" types. Use proper generics.
2. Every tRPC procedure: Zod input validation + error handling + auth check
3. Every Prisma model: id (UUID), createdAt, updatedAt, deletedAt (soft delete)
4. Every UI component: loading state, error state, empty state, responsive
5. All AI calls go through src/lib/ai/ai-service.ts (NEVER call OpenAI directly)
6. Optimistic UI updates with React Query mutation + rollback on error
7. Background jobs (BullMQ) for anything > 2 seconds
8. No pricing logic. No plan checks. No credit limits. All features available.
9. Use barrel exports (index.ts) for clean imports
10. Comments on every function explaining business logic

FILE STRUCTURE

```
src/
├── app/                    # Next.js App Router pages
│   ├── (auth)/             # Auth pages (login, register)
│   ├── (dashboard)/        # Main app layout
│   │   ├── [workspaceId]/
│   │   │   ├── ai-agents/      # Feature 4: Agent list, builder, chat
│   │   │   ├── calendar/       # Feature 16: Calendar optimizer
│   │   │   ├── command-center/ # Feature 23: Personal dashboard
│   │   │   ├── daily-plan/     # Feature 3: Daily planner
```

```
│   │   │   ├── documents/       # Existing + Feature 1 AI writing
│   │   │   ├── email/           # Feature 13: Email assistant
│   │   │   ├── goals/           # Feature 15: Goals & OKRs
│   │   │   ├── habits/          # Feature 14: Habit tracker
│   │   │   ├── integrations/    # Feature 24: Integration hub
│   │   │   ├── knowledge/       # Feature 2: Knowledge base browser
│   │   │   ├── meetings/        # Existing + Feature 7 intelligence
│   │   │   ├── messaging/       # Existing + Feature 11 intelligence
│   │   │   ├── my-day/          # Feature 3: My Day view
│   │   │   ├── projects/        # Existing + Feature 5 AI PM
│   │   │   ├── settings/
│   │   │   │   ├── ai/          # AI settings (models, brand voice)
│   │   │   │   ├── integrations/ # Connected apps settings
│   │   │   │   └── workspace/   # Workspace settings
│   │   │   ├── tasks/           # Existing + Feature 6 intelligence
│   │   │   ├── team-insights/   # Feature 22: Team analytics
│   │   │   ├── templates/       # Feature 17: Template gallery
│   │   │   ├── time-tracking/   # Feature 21: Time analytics
│   │   │   └── workflows/       # Feature 19: Workflow recorder
│   │   └── layout.tsx           # Dashboard layout with sidebar
│   ├── api/
│   │   └── trpc/[trpc]/route.ts  # tRPC API handler
│   └── layout.tsx               # Root layout
│
├── server/                      # Backend logic
│   ├── routers/                 # tRPC routers (one per feature)
│   │   ├── ai-writing.ts        # Feature 1
│   │   ├── knowledge.ts         # Feature 2
│   │   ├── daily-plan.ts        # Feature 3
│   │   ├── agents.ts            # Feature 4
│   │   ├── project-manager.ts   # Feature 5
│   │   ├── task-intelligence.ts # Feature 6
│   │   ├── meetings.ts          # Feature 7
│   │   ├── database-ai.ts       # Feature 8
│   │   ├── focus.ts             # Feature 9
│   │   ├── doc-generator.ts     # Feature 10
│   │   ├── messaging-ai.ts      # Feature 11
│   │   ├── ai-engine.ts         # Feature 12
│   │   ├── email.ts             # Feature 13
│   │   ├── habits.ts            # Feature 14
│   │   ├── goals.ts             # Feature 15
│   │   ├── calendar-optimizer.ts# Feature 16
│   │   ├── templates.ts         # Feature 17
│   │   ├── voice.ts             # Feature 18
```

```
│   │   ├── workflows.ts        # Feature 19
│   │   ├── notifications.ts    # Feature 20
│   │   ├── time-tracking.ts    # Feature 21
│   │   ├── team-insights.ts    # Feature 22
│   │   ├── command-center.ts   # Feature 23
│   │   ├── integrations.ts     # Feature 24
│   │   ├── onboarding.ts       # Feature 25
│   │   └── _app.ts             # Root router merging all routers
│   └── services/               # Business logic services
│       ├── ai-service.ts       # Feature 12: Multi-model AI engine
│       ├── knowledge-indexer.ts  # Feature 2: Indexing pipeline
│       ├── plan-generator.ts   # Feature 3: Daily plan algorithm
│       ├── agent-executor.ts   # Feature 4: Agent execution engine
│       ├── risk-detector.ts    # Feature 5: Risk detection
│       └── calendar-engine.ts  # Feature 16: Calendar optimization
│
├── lib/                        # Shared utilities
│   ├── ai/
│   │   ├── ai-service.ts       # Central AI service (provider-agnostic)
│   │   ├── providers/
│   │   │   ├── openai.ts       # OpenAI provider
│   │   │   ├── anthropic.ts    # Anthropic provider
│   │   │   └── gemini.ts       # Google Gemini provider
│   │   ├── prompts/            # System prompts per feature
│   │   │   ├── writing.ts
│   │   │   ├── task-breakdown.ts
│   │   │   ├── meeting-summary.ts
│   │   │   └── ...
│   │   └── embeddings.ts       # Embedding generation + caching
│   ├── prisma.ts               # Prisma client singleton
│   ├── redis.ts                # Redis client singleton
│   ├── trpc.ts                 # tRPC config
│   └── utils.ts                # Shared helpers
│
├── components/                 # Reusable UI components
│   ├── ai/                     # AI-specific components
│   │   ├── ai-prompt-bar.tsx   # Cmd+J prompt bar (used everywhere)
│   │   ├── ai-streaming-display.tsx  # Streaming text display
│   │   ├── ai-diff-view.tsx    # Before/after text comparison
│   │   ├── ai-suggestion-card.tsx   # Suggestion popover
│   │   ├── ai-model-selector.tsx    # Model dropdown
│   │   └── ai-loading.tsx      # AI processing spinner
│   ├── daily-plan/
│   │   ├── plan-timeline.tsx   # Feature 3: Timeline UI
```

```
│   │   ├── plan-block.tsx       # Feature 3: Single time block
│   │   └── morning-ritual.tsx   # Feature 3: Planning flow
│   ├── agents/
│   │   ├── agent-chat.tsx       # Feature 4: Chat interface
│   │   ├── agent-builder.tsx    # Feature 4: Create/edit agent
│   │   └── agent-card.tsx       # Feature 4: Agent list card
│   └── ...                      # More per feature
│
├── hooks/                       # Custom React hooks
│   ├── use-ai-stream.ts         # Hook for streaming AI responses
│   ├── use-keyboard-shortcut.ts # Keyboard shortcut handler
│   └── use-focus-session.ts     # Focus session state
│
├── stores/                      # Zustand stores
│   ├── ai-store.ts              # AI state (current model, loading)
│   ├── daily-plan-store.ts      # Daily plan state
│   ├── focus-store.ts           # Focus session state
│   └── notification-store.ts    # Notification state
│
├── jobs/                        # BullMQ background jobs
│   ├── index-knowledge.ts       # Feature 2: Index pages/tasks/messages
│   ├── generate-daily-plan.ts   # Feature 3: Generate daily plan
│   ├── execute-agent.ts         # Feature 4: Run agent
│   ├── detect-risks.ts          # Feature 5: Risk detection scan
│   ├── process-meeting.ts       # Feature 7: Transcribe + summarize
│   ├── sync-calendar.ts         # Feature 16: Calendar sync
│   ├── send-notifications.ts    # Feature 20: Batch notifications
│   └── queue-config.ts          # Queue definitions + retry config
│
└── prisma/
    └── schema.prisma            # ALL database models (built in Sprint 1)
```

ALL 25 FEATURES — SUMMARY TABLE

#	Feature	Sprint	Description
1	AI Writing Assistant	2	Context-aware writing in editor + messaging
2	AI Knowledge Manager & Search	2	Universal search + natural language Q&A with citations
3	AI Auto-Scheduler & Daily Planner	3	Auto-schedule tasks into optimized daily plan

4	AI Autonomous Agents	4	24/7 AI assistants with triggers and tool-calling
5	AI Project Manager	5	Standups, risk detection, status reports, automations
6	AI Task Intelligence	5	Task breakdown, prioritization, NLP input, duration estimation
7	AI Meeting Intelligence	6	Recording, transcription, summaries, action items, prep
8	AI Database Intelligence	5	Auto-fill columns, NL queries, AI database creation
9	AI Focus & Productivity Coach	3	Focus protection, sessions, analytics, coaching
10	AI Document Generator	6	Generate reports from workspace data
11	AI Messaging Intelligence	7	Summarize, catch-up, smart replies, translate, decisions
12	Multi-Model AI Engine	1	GPT-4o + Claude + Gemini, auto-routing, centralized service
13	AI Email Assistant	7	Compose, triage, follow-ups, smart labels
14	AI Habit Tracker & Streak Engine	3	Recurring habits, AI scheduling, streaks
15	AI Goal & OKR System	8	Hierarchical goals, auto-progress tracking, risk alerts
16	AI Calendar Optimizer	8	Meeting defrag, focus protection, team scheduling
17	AI Template Engine	9	AI template generation, gallery, recommendations
18	AI Voice Commands & Notes	9	Voice input, transcription, commands
19	AI Workflow Recorder	4	Record workflows, AI cleans up, replay forever
20	AI Smart Notifications	9	AI triage, priority inbox, batching
21	AI Time Tracking	8	Auto-tracking, estimation, analytics

22	AI Team Insights	10	Workload, focus, burnout detection
23	AI Personal Command Center	10	Unified dashboard aggregating all features
24	AI Integration Hub	10	Connect Google Drive, GitHub, Jira, Slack
25	AI Onboarding Assistant	10	Interactive walkthroughs, training chatbot

---

SPRINT DEPENDENCIES (NEVER SKIP)

Sprint 1 (Foundation) → Required by ALL other sprints
Sprint 2 (Writing + Knowledge) → Required by Sprint 4 (Agents), Sprint 5 (PM), Sprint 6 (Meetings)
Sprint 3 (Planner + Habits + Focus) → Required by Sprint 8 (Calendar), Sprint 10 (Dashboard)
Sprint 4 (Agents + Workflows) → Required by Sprint 5 (PM uses agents)
Sprint 5 (PM + Tasks + Database) → Required by Sprint 6 (Meetings create tasks)
Sprint 6 (Meetings + Documents) → Can proceed independently after Sprint 5
Sprint 7 (Messaging + Email) → Can proceed independently after Sprint 2
Sprint 8 (Goals + Calendar + Time) → Requires Sprint 3 + Sprint 5
Sprint 9 (Notifications + Templates + Voice) → Requires Sprint 2 + Sprint 3
Sprint 10 (Dashboard + Team + Integrations + Onboarding) → Requires most prior sprints
Sprint 11 (Polish) → Final pass after all features built

CROSS-FEATURE DATA FLOW

Feature 12 (AI Engine) ──powers──→ ALL other AI features
Feature 2 (Knowledge) ──indexes──→ Pages, Tasks, Messages, Meetings, Files
Feature 3 (Daily Plan) ──uses──→ Tasks (Feature 6), Habits (Feature 14), Calendar (Feature 16)
Feature 4 (Agents) ──uses──→ Knowledge (Feature 2), Writing (Feature 1), Tasks (Feature 6)
Feature 5 (Project Manager) ──uses──→ Tasks (Feature 6), Knowledge (Feature 2), Team Insights (Feature 22)
Feature 7 (Meetings) ──feeds──→ Knowledge (Feature 2), Tasks (Feature 6), Messaging (Feature 11)
Feature 9 (Focus) ──integrates──→ Daily Plan (Feature 3), Calendar (Feature 16), Notifications (Feature 20)
Feature 15 (Goals) ──tracks──→ Tasks (Feature 6), Habits (Feature 14)
Feature 23 (Command Center) ──aggregates──→ Daily Plan, Habits, Goals, Notifications, Meetings, Tasks

READY TO START

Now paste the Sprint 1 file into Replit AI Agent. Sprint 1 builds: ALL database schemas + AI service layer + shared UI components. Without Sprint 1, nothing else works.

SPRINT 1 — FOUNDATION

Build ALL database schemas + AI service + shared components

Estimated build time: 2-3 Replit sessions

PASTE THIS ENTIRE FILE INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Complete Prisma schema with ALL models for all 25 features
2. Multi-model AI service layer (Feature 12)
3. BullMQ job queue configuration
4. Shared AI UI components
5. Knowledge indexing pipeline backend (Feature 2 — indexing only, not search UI)

After this sprint, verify:

- `npx prisma migrate dev` runs without errors
- `npx prisma generate` creates client successfully
- AI service can call OpenAI and stream responses
- BullMQ worker starts and processes test jobs
- All shared components render in Storybook or test page

STEP 1: PRISMA SCHEMA

Add ALL of the following models to `prisma/schema.prisma`. These cover all 25 features.

```
// ================================================================
// prisma/schema.prisma — COMPLETE SCHEMA FOR ALL 25 AI FEATURES
// ================================================================

generator client {
  provider        = "prisma-client-js"
  previewFeatures = ["postgresqlExtensions"]
}

datasource db {
  provider   = "postgresql"
  url        = env("DATABASE_URL")
  extensions = [pgvector(map: "vector", schema: "public"), uuidOssp(map: "uuid-ossp")]
}

// ================================================================
// EXISTING MODELS (already built — do NOT recreate, just reference)
// Add the new fields marked with "// NEW" to existing models
// ================================================================

// If these models already exist, just add the relation fields below.
// If they don't exist exactly like this, adapt the new models to match your existing schema.

// model User { ... existing fields ... }
// model Workspace { ... existing fields ... }
// model Page { ... existing fields ... }
// model Task { ... existing fields ... }
// model Channel { ... existing fields ... }
// model Message { ... existing fields ... }
// model Meeting { ... existing fields ... }

// ================================================================
// FEATURE 1: AI WRITING ASSISTANT
// ================================================================

model AiWritingHistory {
  id            String    @id @default(uuid()) @db.Uuid
  userId        String    @db.Uuid
```

```
  workspaceId    String    @db.Uuid
  pageId         String?   @db.Uuid
  channelId      String?   @db.Uuid
  action         String    //
'write','improve','shorten','lengthen','tone','grammar','translate','summarize','brainstorm','outline','c
ontinue','explain','action_items','custom'
  inputText      String?   @db.Text
  prompt         String    @db.Text
  outputText     String    @db.Text
  model          String    // 'gpt-4o','gpt-4o-mini','claude-sonnet','gemini-pro'
  tone           String?   // 'professional','casual','friendly','academic','persuasive'
  language       String?   // target language for translation
  writerRole     String?   // 'marketer','engineer','executive','designer','sales','support','hr','general'
  tokensUsed     Int       @default(0)
  accepted       Boolean   @default(false)
  feedbackRating Int?      // 1-5
  createdAt      DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
  @@index([workspaceId])
}

model BrandVoice {
  id             String   @id @default(uuid()) @db.Uuid
  workspaceId    String   @unique @db.Uuid
  defaultTone    String   @default("professional")
  styleGuideText  String?  @db.Text  // parsed content from uploaded style guide
  vocabularyRules Json?    // [{"use": "customers", "notUse": "users"}, ...]
  formattingRules String?  @db.Text
  customRoles    Json?    // [{"name": "marketer", "systemPrompt": "..."}, ...]
  createdAt      DateTime @default(now())
  updatedAt      DateTime @updatedAt
}

// ============================================================
// FEATURE 2: AI KNOWLEDGE MANAGER & SEARCH
// ============================================================

model KnowledgeIndex {
  id             String                  @id @default(uuid()) @db.Uuid
  workspaceId    String                  @db.Uuid
  sourceType     String                  //
'page','task','message','meeting_transcript','meeting_summary','comment','file'
  sourceId       String                  @unique @db.Uuid
```

```
  parentSourceId  String?                  @db.Uuid
  title           String
  content         String                  @db.Text
  contentChunks   Json?                   // [{chunkIndex: number, text: string}]
  embedding       Unsupported("vector(1536)")?
  metadata        Json                    //
{authorId,authorName,channelName,databaseName,pagePath,tags,createdDate,lastModifiedD
ate}
  accessLevel     String                  @default("workspace") //
'workspace','channel_members','page_viewers','private'
  lastIndexedAt   DateTime                @default(now())
  isStale         Boolean                 @default(false)
  createdAt       DateTime                @default(now())
  updatedAt       DateTime                @updatedAt

  @@index([workspaceId, sourceType])
  @@index([workspaceId, lastIndexedAt])
  @@index([workspaceId, accessLevel])
}

model KnowledgeQuery {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  query           String   @db.Text
  queryEmbedding  Unsupported("vector(1536)")?
  results         Json?    // [{sourceType,sourceId,title,snippet,relevanceScore,url}]
  aiAnswer        String?  @db.Text
  citations       Json?    // [{sourceId,sourceType,title,snippet}]
  responseTimeMs  Int?
  feedbackRating  Int?     // 1-5
  createdAt       DateTime @default(now())

  @@index([userId, createdAt(sort: Desc)])
  @@index([workspaceId])
}

model KnowledgeSuggestion {
  id              String   @id @default(uuid()) @db.Uuid
  workspaceId     String   @db.Uuid
  channelId       String?  @db.Uuid
  messageId       String?  @db.Uuid
  questionDetected String   @db.Text
  suggestedAnswer  String   @db.Text
```

```
  sources          Json     // [{sourceId,sourceType,title}]
  confidence       Float
  status           String   @default("suggested") // 'suggested','accepted','dismissed','expired'
  shownToUserId    String?  @db.Uuid
  createdAt        DateTime @default(now())

  @@index([workspaceId, status])
  @@index([channelId])
}

model KnowledgeItem {
  id           String   @id @default(uuid()) @db.Uuid
  workspaceId  String   @db.Uuid
  category     String   // 'how_to','decision','policy','faq','reference'
  title        String
  content      String   @db.Text
  sourceType   String   // where this was extracted from
  sourceId     String   @db.Uuid
  confidence   Float    @default(0)
  status       String   @default("draft") // 'draft','published','archived'
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([workspaceId, category])
  @@index([workspaceId, status])
}

// ==============================================================
// FEATURE 3: AI AUTO-SCHEDULER & DAILY PLANNER
// ==============================================================

model DailyPlan {
  id               String   @id @default(uuid()) @db.Uuid
  userId           String   @db.Uuid
  workspaceId      String   @db.Uuid
  date             DateTime @db.Date
  status           String   @default("generated") // 'generated','active','completed'
  timeBlocks       Json     // Array — see TimeBlock type below
  aiSummary        Json?    //
{totalPlannedMinutes,focusMinutes,meetingMinutes,mustDoCount,shouldDoCount,niceToDoCo
unt,topPriority,aiReasoning,completionLikelihood}
  completionRate     Float?
  userModifications  Int      @default(0)
  reviewNote         String?  @db.Text
```

```
  createdAt          DateTime @default(now())
  updatedAt          DateTime @updatedAt

  @@unique([userId, date])
  @@index([userId, date(sort: Desc)])
}

// TimeBlock JSON structure (stored in DailyPlan.timeBlocks array):
// {
//   id: string (uuid),
//   type: 'task' | 'meeting' | 'focus' | 'break' | 'habit' | 'communication' | 'planning' | 'buffer' |
'custom',
//   title: string,
//   description?: string,
//   startTime: string ("09:00" — 24hr format),
//   endTime: string ("10:30"),
//   durationMinutes: number,
//   taskId?: string (uuid — linked task),
//   meetingId?: string (uuid — linked calendar event),
//   habitId?: string (uuid — linked habit),
//   priority: 'must_do' | 'should_do' | 'nice_to_do',
//   energyLevel: 'high' | 'medium' | 'low',
//   isFixed: boolean (meetings = true, tasks = false),
//   isCompleted: boolean,
//   completedAt?: string (ISO datetime),
//   isSkipped: boolean,
//   actualDurationMinutes?: number,
//   scheduledByAI: boolean,
//   aiReason?: string ("Scheduled in your peak focus window 10-12 AM")
// }

model SchedulingPreference {
  id                    String   @id @default(uuid()) @db.Uuid
  userId                String   @unique @db.Uuid
  workspaceId           String   @db.Uuid
  workingHours          Json     @default("{\\"start\\":\\"09:00\\",\\"end\\":\\"18:00\\"}")
  timezone              String   @default("UTC")
  workDays              Int[]    @default([1, 2, 3, 4, 5]) // 0=Sun, 1=Mon...6=Sat
  lunchTime             Json     @default("{\\"start\\":\\"12:30\\",\\"end\\":\\"13:30\\"}")
  peakFocusWindow       Json?    // {"start":"10:00","end":"12:00"} — auto-learned
  preferMeetingsIn      String   @default("any") // 'morning','afternoon','any'
  minFocusBlockMinutes  Int      @default(60)
  bufferBetweenMeetings Int      @default(15) // minutes
  autoScheduleEnabled   Boolean  @default(true)
```

```
  morningPlanningEnabled   Boolean  @default(true)
  morningPlanningTime      String   @default("08:30")
  eveningReviewEnabled     Boolean  @default(true)
  eveningReviewTime        String   @default("17:30")
  planMode                 String   @default("suggest") // 'automatic','suggest','manual'
  createdAt                DateTime @default(now())
  updatedAt                DateTime @updatedAt
}

// ==============================================================
// FEATURE 4: AI AUTONOMOUS AGENTS
// ==============================================================

model AiAgent {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
  name            String
  description     String?   @db.Text
  icon            String?
  type            String    @default("custom") // 'built_in','custom','template'
  createdBy       String    @db.Uuid
  isShared        Boolean   @default(false)
  visibility      String    @default("workspace") // 'private','workspace','specific_members'
  visibleToUserIds  String[]  @db.Uuid
  systemPrompt    String    @db.Text
  model           String    @default("gpt-4o")
  temperature     Float     @default(0.7)
  capabilities    String[]  //
['read_pages','write_pages','read_tasks','write_tasks','read_messages','send_messages','search
_workspace','create_database_rows','update_database_rows','send_notifications','read_calenda
r','web_search','generate_content']
  dataAccess      Json      //
{allPages:bool,specificPageIds:string[],allDatabases:bool,specificDatabaseIds:string[],allChanne
ls:bool,specificChannelIds:string[],allMeetings:bool}
  triggerType     String    @default("manual") //
'manual','scheduled','event','message_in_channel','form_submission','webhook'
  triggerConfig   Json?     // trigger-specific config
  isActive        Boolean   @default(true)
  maxRunsPerDay   Int       @default(100)
  maxActionsPerRun  Int     @default(25)
  timeoutMinutes  Int       @default(5)
  totalRuns       Int       @default(0)
  totalTokensUsed  Int      @default(0)
  lastRunAt       DateTime?
```

```
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
  deletedAt         DateTime?

  runs              AgentRun[]
  conversations     AgentConversation[]

  @@index([workspaceId, isActive])
  @@index([workspaceId, type])
  @@index([createdBy])
}

model AgentRun {
  id              String    @id @default(uuid()) @db.Uuid
  agentId         String    @db.Uuid
  workspaceId     String    @db.Uuid
  triggeredBy     String    // 'manual','schedule','event','message','form','webhook'
  triggerData     Json?
  invokedByUserId  String?   @db.Uuid
  status          String    @default("queued") //
'queued','running','completed','failed','cancelled','timeout'
  steps           Json[]    //
[{stepNumber,action,input,output,durationMs,status:'success'|'failed'|'skipped',error?}]
  finalOutput     String?   @db.Text
  tokensUsed      Int       @default(0)
  durationMs      Int?
  error           String?   @db.Text
  startedAt       DateTime?
  completedAt     DateTime?
  createdAt       DateTime  @default(now())

  agent           AiAgent   @relation(fields: [agentId], references: [id])

  @@index([agentId, createdAt(sort: Desc)])
  @@index([workspaceId])
  @@index([status])
}

model AgentConversation {
  id        String   @id @default(uuid()) @db.Uuid
  agentId   String   @db.Uuid
  userId    String   @db.Uuid
  messages  Json[]   // [{role:'user'|'agent',content:string,timestamp:string,runId?:string}]
  status    String   @default("active") // 'active','closed'
```

```
  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt

  agent     AiAgent  @relation(fields: [agentId], references: [id])

  @@index([agentId, userId])
  @@index([userId, updatedAt(sort: Desc)])
}

// ==============================================================
// FEATURE 5: AI PROJECT MANAGER
// ==============================================================

model ProjectStatusReport {
  id              String   @id @default(uuid()) @db.Uuid
  workspaceId       String   @db.Uuid
  databaseId        String   @db.Uuid // which task database/project
  generatedBy       String   // 'ai_auto','ai_on_demand','manual'
  reportType        String   // 'daily_standup','weekly_status','sprint_review','risk_report'
  periodStart       DateTime @db.Date
  periodEnd         DateTime @db.Date
  content           Json     //
{summary,completedTasks[],inProgressTasks[],blockedTasks[],overdueTasks[],upcomingDeadlin
es[],risks[],velocity,prediction,highlights[],concerns[]}
  postedToChannelId String?  @db.Uuid
  postedMessageId   String?  @db.Uuid
  createdAt         DateTime @default(now())

  @@index([workspaceId, databaseId])
  @@index([workspaceId, reportType])
}

model AiRiskAlert {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId      String    @db.Uuid
  databaseId       String    @db.Uuid
  riskType         String    //
'deadline_at_risk','team_member_overloaded','blocked_task_chain','scope_creep','velocity_decli
ning','unassigned_urgent','dependency_conflict'
  severity         String    // 'low','medium','high','critical'
  title            String
  description      String    @db.Text
  affectedTaskIds  String[]  @db.Uuid
  affectedUserIds  String[]  @db.Uuid
```

```
  suggestedAction  String    @db.Text
  status           String    @default("active") // 'active','acknowledged','resolved','dismissed'
  acknowledgedBy   String?   @db.Uuid
  resolvedAt       DateTime?
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt

  @@index([workspaceId, status])
  @@index([workspaceId, severity])
}

model AiWorkflowAutomation {
  id               String    @id @default(uuid()) @db.Uuid
  workspaceId      String    @db.Uuid
  name             String
  description      String?   @db.Text
  createdFromPrompt String?  @db.Text // natural language input
  trigger          Json      //
{type:'task_created'|'task_status_changed'|'task_overdue'|'scheduled'|...,config:{}}
  conditions       Json[]    // [{field,operator,value}]
  actions          Json[]    //
[{type:'update_task'|'send_message'|'create_task'|'assign'|'notify',config:{}}]
  isActive         Boolean   @default(true)
  runCount         Int       @default(0)
  lastRunAt        DateTime?
  createdBy        String    @db.Uuid
  createdAt        DateTime  @default(now())
  updatedAt        DateTime  @updatedAt
  deletedAt        DateTime?

  @@index([workspaceId, isActive])
}

// ============================================================
// FEATURE 7: AI MEETING INTELLIGENCE
// ============================================================

model MeetingIntelligence {
  id               String   @id @default(uuid()) @db.Uuid
  meetingId        String   @unique @db.Uuid // FK to existing Meeting model
  workspaceId      String   @db.Uuid
  audioUrl         String?  // S3/R2 URL of recording
  audioDurationSecs Int?
  transcriptRaw    String?  @db.Text // full transcription
```

```
  transcriptSegments Json?   // [{speakerLabel,startTime,endTime,text}]
  speakerLabels     Json?    // [{label:"Speaker 1",userId?:string,name?:string}]
  aiSummary         String?  @db.Text
  keyPoints         Json?    // string[]
  decisions         Json?    // [{decision:string,context:string,decidedBy?:string}]
  actionItems       Json?    //
[{title,description,suggestedOwner?,suggestedDueDate?,taskCreated:bool,taskId?}]
  openQuestions     Json?    // string[]
  prepBrief         String?  @db.Text
  processingStatus  String   @default("pending") //
'pending','transcribing','summarizing','completed','failed'
  processingError   String?
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@index([workspaceId])
  @@index([processingStatus])
}

// ============================================================
// FEATURE 8: AI DATABASE INTELLIGENCE
// ============================================================

model AiAutoFillColumn {
  id          String   @id @default(uuid()) @db.Uuid
  workspaceId  String   @db.Uuid
  databaseId   String   @db.Uuid // which task database this column belongs to
  columnName   String
  aiPrompt     String   @db.Text // "Based on {Title} and {Description}, assign category: Bug,
Feature, Enhancement"
  inputFields  String[] // ['title', 'description'] — which fields the AI reads
  outputType   String   @default("text") // 'text','select','number'
  selectOptions String[]? // if outputType='select', the allowed values
  isActive     Boolean  @default(true)
  createdBy    String   @db.Uuid
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  @@index([workspaceId, databaseId])
}

// ============================================================
// FEATURE 9: AI FOCUS & PRODUCTIVITY COACH
// ============================================================
```

```
model FocusSession {
  id              String    @id @default(uuid()) @db.Uuid
  userId          String    @db.Uuid
  workspaceId     String    @db.Uuid
  taskId          String?   @db.Uuid // linked task
  title           String    // what the user is focusing on
  plannedMinutes  Int
  actualMinutes   Int?
  startedAt       DateTime
  endedAt         DateTime?
  pausedMinutes   Int       @default(0) // total time paused
  status          String    @default("active") // 'active','paused','completed','abandoned'
  notificationsPaused Boolean @default(true)
  queuedNotificationCount Int @default(0) // notifications held during session
  createdAt       DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
  @@index([userId, status])
}

model FocusTimeBlock {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  date            DateTime @db.Date
  startTime       String   // "10:00"
  endTime         String   // "12:00"
  durationMinutes Int
  calendarEventId String?  // Google Calendar event ID (synced)
  isProtected     Boolean  @default(true) // shows as "busy" on calendar
  source          String   @default("ai") // 'ai','manual'
  createdAt       DateTime @default(now())

  @@index([userId, date])
}

model ProductivityScore {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  date            DateTime @db.Date
  focusMinutes    Int      @default(0)
  meetingMinutes  Int      @default(0)
```

```
  tasksCompleted      Int      @default(0)
  tasksPlanned        Int      @default(0)
  habitsCompleted     Int      @default(0)
  habitsPlanned       Int      @default(0)
  interruptionCount   Int      @default(0)
  productivityScore   Float?   // 0-100 AI-calculated
  aiInsight           String?  @db.Text // "You focused 2h more than last week. Peak: 10-11 AM."
  createdAt           DateTime @default(now())

  @@unique([userId, date])
  @@index([userId, date(sort: Desc)])
}

// ==============================================================
// FEATURE 13: AI EMAIL ASSISTANT
// ==============================================================

model EmailAccount {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  provider        String   // 'gmail','outlook'
  email           String
  accessToken     String   @db.Text // encrypted
  refreshToken    String?  @db.Text // encrypted
  syncEnabled     Boolean  @default(true)
  lastSyncAt      DateTime?
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@unique([userId, email])
}

model EmailThread {
  id              String   @id @default(uuid()) @db.Uuid
  emailAccountId  String   @db.Uuid
  externalId      String   // provider thread ID
  subject         String
  participants    Json     // [{name,email,role:'from'|'to'|'cc'}]
  latestSnippet   String?  @db.Text
  messageCount    Int      @default(1)
  isRead          Boolean  @default(false)
  aiPriority      String?  // 'urgent','important','fyi','low'
  aiLabels        String[] // ['action_required','waiting','informational','scheduled']
```

```
  aiSummary       String?  @db.Text
  lastMessageAt   DateTime
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  @@index([emailAccountId, lastMessageAt(sort: Desc)])
  @@index([emailAccountId, aiPriority])
}

// ==============================================================
// FEATURE 14: AI HABIT TRACKER & STREAK ENGINE
// ==============================================================

model Habit {
  id                 String    @id @default(uuid()) @db.Uuid
  userId             String    @db.Uuid
  workspaceId        String    @db.Uuid
  name               String
  icon               String?   // emoji
  color              String?   // hex
  durationMinutes    Int
  frequency          String    @default("daily") // 'daily','weekdays','custom'
  customDays         Int[]     // [1,2,3,4,5] for weekdays
  preferredTimeRange  Json?     // {"earliest":"09:00","latest":"17:00"}
  priority           String    @default("medium") // 'high','medium','low'
  isFlexible         Boolean   @default(true)
  isProtected        Boolean   @default(false) // block on shared calendar
  streakCurrent      Int       @default(0)
  streakLongest      Int       @default(0)
  isActive           Boolean   @default(true)
  goalId             String?   @db.Uuid // linked to Feature 15 goal
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt
  deletedAt          DateTime?

  completions        HabitCompletion[]

  @@index([userId, isActive])
}

model HabitCompletion {
  id          String   @id @default(uuid()) @db.Uuid
  habitId     String   @db.Uuid
  userId      String   @db.Uuid
```

```
  date        DateTime @db.Date
  completedAt DateTime @default(now())

  habit       Habit    @relation(fields: [habitId], references: [id])

  @@unique([habitId, date])
  @@index([userId, date(sort: Desc)])
}

// ==============================================================
// FEATURE 15: AI GOAL & OKR SYSTEM
// ==============================================================

model Goal {
  id              String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
  title           String
  description     String?   @db.Text
  ownerId         String    @db.Uuid // user who owns this goal
  level           String    // 'company','team','individual'
  parentGoalId    String?   @db.Uuid // hierarchical: company > team > individual
  status          String    @default("on_track") // 'on_track','at_risk','behind','completed','cancelled'
  progressPercent Float     @default(0) // 0-100, auto-calculated from key results
  startDate       DateTime  @db.Date
  targetDate      DateTime  @db.Date
  linkedDatabaseIds String[] @db.Uuid // task databases that contribute to this goal
  aiRiskAssessment String?  @db.Text
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  keyResults      KeyResult[]

  @@index([workspaceId, level])
  @@index([workspaceId, status])
  @@index([ownerId])
  @@index([parentGoalId])
}

model KeyResult {
  id              String   @id @default(uuid()) @db.Uuid
  goalId          String   @db.Uuid
  title           String
  metricType      String   // 'number','percentage','currency','boolean'
```

```
  currentValue    Float    @default(0)
  targetValue     Float
  startValue      Float    @default(0)
  unit            String?  // '%', '$', 'users', 'tickets', etc.
  progressPercent Float    @default(0) // auto-calculated: (current-start)/(target-start)*100
  autoTrackSource String?  // 'task_count','task_completion','habit_streak','manual'
  autoTrackConfig Json?    // {databaseId?,statusFilter?,habitId?}
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  goal            Goal     @relation(fields: [goalId], references: [id], onDelete: Cascade)

  @@index([goalId])
}

// ============================================================
// FEATURE 16: AI CALENDAR OPTIMIZER
// ============================================================

model CalendarOptimizationRule {
  id                  String   @id @default(uuid()) @db.Uuid
  workspaceId         String   @db.Uuid
  ruleType            String   //
'no_meeting_window','buffer_time','meeting_defrag','focus_protection'
  config              Json     // depends on ruleType
  // no_meeting_window: {days:[2,4],startTime:"10:00",endTime:"12:00"}
  // buffer_time: {minutes:15,applyBetween:'all'|'external_only'}
  // meeting_defrag: {enabled:true,preferMorning:true}
  // focus_protection: {minBlockMinutes:60,dailyTargetMinutes:120}
  scope               String   @default("workspace") // 'workspace','team','individual'
  scopeId             String?  @db.Uuid // teamId or userId
  isActive            Boolean  @default(true)
  createdBy           String   @db.Uuid
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt

  @@index([workspaceId, ruleType])
}

// ============================================================
// FEATURE 17: AI TEMPLATE ENGINE
// ============================================================

model Template {
```

```
  id            String   @id @default(uuid()) @db.Uuid
  workspaceId   String?  @db.Uuid // null = global/public template
  name          String
  description   String?  @db.Text
  category      String   //
'project_management','personal','marketing','engineering','hr','sales','design','custom'
  type          String   // 'page','database','workspace','workflow','agent'
  content       Json     // full template content (page blocks, database schema, etc.)
  thumbnail     String?  // URL to preview image
  isPublic      Boolean  @default(false) // visible in global gallery
  usageCount    Int      @default(0)
  rating        Float    @default(0)
  tags          String[]
  createdBy     String   @db.Uuid
  createdAt     DateTime @default(now())
  updatedAt     DateTime @updatedAt
  deletedAt     DateTime?

  @@index([workspaceId])
  @@index([category, isPublic])
  @@index([usageCount(sort: Desc)])
}

// ============================================================
// FEATURE 18: AI VOICE COMMANDS & NOTES
// ============================================================

model VoiceNote {
  id            String    @id @default(uuid()) @db.Uuid
  userId        String    @db.Uuid
  workspaceId   String    @db.Uuid
  audioUrl      String    // S3/R2 URL
  durationSecs  Int
  transcript    String?   @db.Text
  aiSummary     String?   @db.Text
  convertedTo   String?   // 'task','page','meeting_note',null
  convertedId   String?   @db.Uuid // ID of created task/page
  status        String    @default("recording") // 'recording','transcribing','completed','failed'
  createdAt     DateTime  @default(now())

  @@index([userId, createdAt(sort: Desc)])
}

// ============================================================
```

```
// FEATURE 19: AI WORKFLOW RECORDER & PLAYBACK
// ============================================================

model Workflow {
  id            String    @id @default(uuid()) @db.Uuid
  workspaceId     String    @db.Uuid
  name            String
  description     String?   @db.Text
  recordedSteps   Json[]    //
[{stepNumber,actionType:'create_task'|'send_message'|'create_page'|...,rawData:{},cleanedData
:{},variables:[]}]
  variables       Json[]    // [{name:'project_name',type:'text'|'user'|'date'|'database',defaultValue?}]
  isPublic        Boolean   @default(false) // shared with workspace
  runCount        Int       @default(0)
  triggerType     String    @default("manual") // 'manual','scheduled','event'
  triggerConfig   Json?
  createdBy       String    @db.Uuid
  createdAt       DateTime  @default(now())
  updatedAt       DateTime  @updatedAt
  deletedAt       DateTime?

  @@index([workspaceId])
  @@index([createdBy])
}

model WorkflowRun {
  id           String    @id @default(uuid()) @db.Uuid
  workflowId   String    @db.Uuid
  userId       String    @db.Uuid
  variableValues Json    // filled-in variable values
  steps        Json[]    // [{stepNumber,status:'success'|'failed'|'skipped',output?,error?}]
  status       String    @default("running") // 'running','completed','failed'
  startedAt    DateTime  @default(now())
  completedAt  DateTime?

  @@index([workflowId, startedAt(sort: Desc)])
}

// ============================================================
// FEATURE 20: AI SMART NOTIFICATIONS
// ============================================================

model NotificationPreference {
  id                  String   @id @default(uuid()) @db.Uuid
```

```
  userId              String   @unique @db.Uuid
  workspaceId         String   @db.Uuid
  alwaysNotify        String[] //
['direct_mention','direct_message','task_assigned','deadline_today']
  batchNotify         String[] // ['channel_message','task_updated','comment_added']
  muteNotify          String[] // ['channel_joined','reaction_added']
  batchFrequency      String   @default("3_per_day") // '2_per_day','3_per_day','hourly'
  batchTimes          String[] @default(["09:00", "13:00", "17:00"])
  focusModeAction     String   @default("queue") // 'queue','urgent_only','all'
  aiTriageEnabled     Boolean  @default(true)
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
}

model NotificationItem {
  id              String   @id @default(uuid()) @db.Uuid
  userId          String   @db.Uuid
  workspaceId     String   @db.Uuid
  type            String   //
'mention','dm','task_assigned','task_due','comment','reaction','channel_activity','agent_result','ris
k_alert','goal_update','system'
  title           String
  body            String   @db.Text
  sourceType      String?  // 'message','task','meeting','agent','goal','system'
  sourceId        String?  @db.Uuid
  aiPriority      String   @default("normal") // 'urgent','important','normal','low'
  isRead          Boolean  @default(false)
  isBatched       Boolean  @default(false) // part of a batch digest
  batchId         String?  @db.Uuid
  deliveredAt     DateTime?
  readAt          DateTime?
  createdAt       DateTime @default(now())

  @@index([userId, isRead, createdAt(sort: Desc)])
  @@index([userId, aiPriority])
}

// ==============================================================
// FEATURE 21: AI TIME TRACKING
// ==============================================================

model TimeEntry {
  id              String    @id @default(uuid()) @db.Uuid
  userId          String    @db.Uuid
```

```
  workspaceId     String    @db.Uuid
  taskId          String?   @db.Uuid
  focusSessionId  String?   @db.Uuid // linked to FocusSession
  description     String?
  durationMinutes Int
  startedAt       DateTime
  endedAt         DateTime?
  source          String    @default("manual") // 'manual','focus_session','auto_detected'
  createdAt       DateTime  @default(now())

  @@index([userId, startedAt(sort: Desc)])
  @@index([taskId])
}

model TaskDurationEstimate {
  id                String   @id @default(uuid()) @db.Uuid
  taskId            String   @db.Uuid
  workspaceId       String   @db.Uuid
  estimatedMinutes  Int
  actualMinutes     Int?
  confidence        Float    @default(0.5) // 0-1
  estimatedByAi     Boolean  @default(true)
  adjustedByUser    Boolean  @default(false)
  createdAt         DateTime @default(now())
  updatedAt         DateTime @updatedAt

  @@unique([taskId])
}

// ============================================================
// FEATURE 22: AI TEAM COLLABORATION INSIGHTS
// ============================================================

model TeamInsightSnapshot {
  id                String   @id @default(uuid()) @db.Uuid
  workspaceId       String   @db.Uuid
  date              DateTime @db.Date
  memberStats       Json     //
[{userId,focusMinutes,meetingMinutes,tasksCompleted,activeTaskCount,overdueTasks,workloa
dScore:'green'|'yellow'|'red'}]
  teamMetrics       Json     //
{avgFocusMinutes,avgMeetingMinutes,avgTasksCompleted,meetingFreeCompliance,collaborati
onPatterns}
  aiInsights        Json     // [{type:'burnout_risk'|'overload'|'recommendation',userId?,message}]
```

```
  createdAt          DateTime @default(now())

  @@unique([workspaceId, date])
  @@index([workspaceId, date(sort: Desc)])
}

// ============================================================
// FEATURE 24: AI INTEGRATION HUB
// ============================================================

model ExternalIntegration {
  id                 String    @id @default(uuid()) @db.Uuid
  workspaceId        String    @db.Uuid
  type               String    // 'google_drive','github','gitlab','jira','linear','slack','figma','confluence'
  displayName        String    // "Acme Corp Google Drive"
  accessTokenEncrypted String   @db.Text
  refreshTokenEncrypted String? @db.Text
  scopes             String[]
  syncEnabled        Boolean   @default(true)
  syncStatus         String    @default("active") // 'active','syncing','error','disconnected'
  lastSyncAt         DateTime?
  documentCount      Int       @default(0)
  errorMessage       String?
  createdBy          String    @db.Uuid
  createdAt          DateTime  @default(now())
  updatedAt          DateTime  @updatedAt

  @@index([workspaceId, type])
}

// ============================================================
// FEATURE 25: AI ONBOARDING ASSISTANT
// ============================================================

model OnboardingProgress {
  id                 String   @id @default(uuid()) @db.Uuid
  userId             String   @unique @db.Uuid
  workspaceId        String   @db.Uuid
  completedSteps     String[] //
['welcome','profile','first_task','first_page','join_channel','first_habit','ai_search','daily_plan']
  currentStep        String?
  isComplete         Boolean  @default(false)
  dismissedAt        DateTime?
```

```
  featureSuggestions  Json?    // [{feature,reason,suggested:bool,dismissed:bool}] — AI
suggests unused features
  createdAt           DateTime @default(now())
  updatedAt           DateTime @updatedAt
}

// ============================================================
// AI USAGE TRACKING (shared across all features)
// ============================================================

model AiUsageLog {
  id            String   @id @default(uuid()) @db.Uuid
  userId        String   @db.Uuid
  workspaceId   String   @db.Uuid
  feature       String   //
'writing','search','planner','agent','project_manager','task','meeting','database','focus','doc_gen','
messaging','email','voice','template'
  action        String   // specific action within feature
  model         String   // which AI model was used
  inputTokens   Int      @default(0)
  outputTokens  Int      @default(0)
  durationMs    Int?
  success       Boolean  @default(true)
  error         String?
  createdAt     DateTime @default(now())

  @@index([workspaceId, feature, createdAt])
  @@index([userId, createdAt(sort: Desc)])
}
```

Run `npx prisma migrate dev --name add_all_ai_features` after adding all models.

STEP 2: MULTI-MODEL AI SERVICE (Feature 12)

Create the centralized AI service that ALL other features use. This is the most critical file.

Create file: `src/lib/ai/ai-service.ts`

```
// src/lib/ai/ai-service.ts
// CENTRAL AI SERVICE — Every AI feature calls this, never OpenAI directly
// Provider-agnostic: swap providers by changing config only
```

```
import OpenAI from "openai";

// ==============================================================
// TYPES
// ==============================================================

export type AiModel = "gpt-4o" | "gpt-4o-mini" | "claude-sonnet" | "gemini-pro";
export type AiProvider = "openai" | "anthropic" | "google";

export interface AiGenerateOptions {
  model?: AiModel;
  maxTokens?: number;
  temperature?: number;
  systemPrompt?: string;
  stream?: boolean;
}

export interface AiStreamChunk {
  text: string;
  done: boolean;
}

export interface AiEmbeddingResult {
  embedding: number[];
  tokensUsed: number;
}

interface ModelConfig {
  provider: AiProvider;
  modelId: string;       // actual model ID sent to provider
  maxTokens: number;
  costPer1kInput: number;
  costPer1kOutput: number;
}

// ==============================================================
// MODEL REGISTRY
// ==============================================================

const MODEL_REGISTRY: Record<AiModel, ModelConfig> = {
  "gpt-4o": {
    provider: "openai",
    modelId: "gpt-4o",
```

```
    maxTokens: 128000,
    costPer1kInput: 0.005,
    costPer1kOutput: 0.015,
  },
  "gpt-4o-mini": {
    provider: "openai",
    modelId: "gpt-4o-mini",
    maxTokens: 128000,
    costPer1kInput: 0.00015,
    costPer1kOutput: 0.0006,
  },
  "claude-sonnet": {
    provider: "anthropic",
    modelId: "claude-sonnet-4-20250514",
    maxTokens: 200000,
    costPer1kInput: 0.003,
    costPer1kOutput: 0.015,
  },
  "gemini-pro": {
    provider: "google",
    modelId: "gemini-1.5-pro",
    maxTokens: 1000000,
    costPer1kInput: 0.00125,
    costPer1kOutput: 0.005,
  },
};

// Auto-routing: best model per task type
const AUTO_MODEL_MAP: Record<string, AiModel> = {
  writing: "gpt-4o",        // Good all-round writing
  code: "gpt-4o",           // Strong at code
  analysis: "gpt-4o",       // Data analysis
  summary: "gpt-4o-mini",   // Summaries are simpler
  quick: "gpt-4o-mini",     // Fast, cheap tasks
  creative: "gpt-4o",       // Creative writing
  translation: "gpt-4o",    // Multi-language
};

// ==============================================================
// AI SERVICE CLASS
// ==============================================================

class AiService {
  private openai: OpenAI;
```

```
constructor() {
  this.openai = new OpenAI({
    apiKey: process.env.OPENAI_API_KEY,
  });
}

/**
 * Select the best model for a task type, or use the specified model
 */
resolveModel(taskType?: string, requestedModel?: AiModel): AiModel {
  if (requestedModel) return requestedModel;
  if (taskType && AUTO_MODEL_MAP[taskType]) return AUTO_MODEL_MAP[taskType];
  return "gpt-4o"; // default
}

/**
 * Generate text (non-streaming). Use for short responses.
 */
async generateText(
  userPrompt: string,
  options: AiGenerateOptions = {}
): Promise<{ text: string; tokensUsed: { input: number; output: number } }> {
  const model = options.model || "gpt-4o";
  const config = MODEL_REGISTRY[model];

  // Currently only OpenAI implemented. Add Anthropic/Google providers here later.
  const messages: OpenAI.Chat.ChatCompletionMessageParam[] = [];
  if (options.systemPrompt) {
    messages.push({ role: "system", content: options.systemPrompt });
  }
  messages.push({ role: "user", content: userPrompt });

  const response = await this.openai.chat.completions.create({
    model: config.modelId,
    messages,
    max_tokens: options.maxTokens || 2000,
    temperature: options.temperature ?? 0.7,
  });

  const text = response.choices[0]?.message?.content || "";
  const usage = response.usage;

  return {
```

```
    text,
    tokensUsed: {
      input: usage?.prompt_tokens || 0,
      output: usage?.completion_tokens || 0,
    },
  };
}

/**
 * Generate text with streaming. Use for all user-facing AI responses.
 * Returns an async generator that yields text chunks.
 */
async *generateStream(
  userPrompt: string,
  options: AiGenerateOptions = {}
): AsyncGenerator<AiStreamChunk> {
  const model = options.model || "gpt-4o";
  const config = MODEL_REGISTRY[model];

  const messages: OpenAI.Chat.ChatCompletionMessageParam[] = [];
  if (options.systemPrompt) {
    messages.push({ role: "system", content: options.systemPrompt });
  }
  messages.push({ role: "user", content: userPrompt });

  const stream = await this.openai.chat.completions.create({
    model: config.modelId,
    messages,
    max_tokens: options.maxTokens || 2000,
    temperature: options.temperature ?? 0.7,
    stream: true,
  });

  for await (const chunk of stream) {
    const text = chunk.choices[0]?.delta?.content || "";
    const done = chunk.choices[0]?.finish_reason === "stop";
    if (text || done) {
      yield { text, done };
    }
  }
}

/**
 * Generate text with function/tool calling. Used by AI Agents (Feature 4).
```

```
 */
async generateWithTools(
  messages: OpenAI.Chat.ChatCompletionMessageParam[],
  tools: OpenAI.Chat.ChatCompletionTool[],
  options: AiGenerateOptions = {}
): Promise<OpenAI.Chat.ChatCompletionMessage> {
  const model = options.model || "gpt-4o";
  const config = MODEL_REGISTRY[model];

  const response = await this.openai.chat.completions.create({
    model: config.modelId,
    messages,
    tools,
    max_tokens: options.maxTokens || 4000,
    temperature: options.temperature ?? 0.5,
  });

  return response.choices[0].message;
}

/**
 * Generate embedding vector for semantic search. Used by Feature 2 (Knowledge).
 */
async generateEmbedding(text: string): Promise<AiEmbeddingResult> {
  const response = await this.openai.embeddings.create({
    model: "text-embedding-3-small",
    input: text,
    dimensions: 1536,
  });

  return {
    embedding: response.data[0].embedding,
    tokensUsed: response.usage.total_tokens,
  };
}

/**
 * Generate embeddings in batch (up to 20 texts). Used by Knowledge Indexer.
 */
async generateEmbeddingBatch(texts: string[]): Promise<AiEmbeddingResult[]> {
  const response = await this.openai.embeddings.create({
    model: "text-embedding-3-small",
    input: texts,
    dimensions: 1536,
```

```
    });

    return response.data.map((item, i) => ({
      embedding: item.embedding,
      tokensUsed: Math.ceil(response.usage.total_tokens / texts.length),
    }));
  }

  /**
   * Generate structured JSON output. Used by Task Breakdown, Database AI, etc.
   */
  async generateJSON<T>(
    userPrompt: string,
    options: AiGenerateOptions & { jsonSchema?: string } = {}
  ): Promise<T> {
    const systemPrompt = `${options.systemPrompt || ""}\\n\\nIMPORTANT: Respond with valid
JSON only. No markdown, no backticks, no explanation. Just the JSON object.`;

    const { text } = await this.generateText(userPrompt, {
      ...options,
      systemPrompt,
      temperature: options.temperature ?? 0.3, // lower temp for structured output
    });

    // Clean potential markdown backticks
    const cleaned = text.replace(/```json\\n?/g, "").replace(/```\\n?/g, "").trim();
    return JSON.parse(cleaned) as T;
  }
}

// Singleton export
export const aiService = new AiService();
```

Create file: `src/lib/ai/embeddings.ts`

```
// src/lib/ai/embeddings.ts
// Embedding generation with Redis caching

import { aiService } from "./ai-service";
import { redis } from "../redis";
import crypto from "crypto";

const CACHE_TTL = 86400; // 24 hours
```

```
const CACHE_PREFIX = "emb:";

function hashText(text: string): string {
  return crypto.createHash("sha256").update(text).digest("hex").slice(0, 16);
}

/**
 * Get embedding for text, with Redis cache
 */
export async function getEmbedding(text: string): Promise<number[]> {
  const cacheKey = `${CACHE_PREFIX}${hashText(text)}`;

  // Check cache
  const cached = await redis.get(cacheKey);
  if (cached) {
    return JSON.parse(cached as string);
  }

  // Generate new embedding
  const result = await aiService.generateEmbedding(text);

  // Cache it
  await redis.setex(cacheKey, CACHE_TTL, JSON.stringify(result.embedding));

  return result.embedding;
}

/**
 * Get embeddings for multiple texts in batch, with caching
 */
export async function getEmbeddingBatch(texts: string[]): Promise<number[][]> {
  const results: (number[] | null)[] = new Array(texts.length).fill(null);
  const uncachedIndices: number[] = [];
  const uncachedTexts: string[] = [];

  // Check cache for each text
  for (let i = 0; i < texts.length; i++) {
    const cacheKey = `${CACHE_PREFIX}${hashText(texts[i])}`;
    const cached = await redis.get(cacheKey);
    if (cached) {
      results[i] = JSON.parse(cached as string);
    } else {
      uncachedIndices.push(i);
      uncachedTexts.push(texts[i]);
```

```
    }
  }

  // Generate embeddings for uncached texts (batch of 20 max)
  if (uncachedTexts.length > 0) {
    for (let batch = 0; batch < uncachedTexts.length; batch += 20) {
      const batchTexts = uncachedTexts.slice(batch, batch + 20);
      const batchResults = await aiService.generateEmbeddingBatch(batchTexts);

      for (let j = 0; j < batchResults.length; j++) {
        const originalIndex = uncachedIndices[batch + j];
        results[originalIndex] = batchResults[j].embedding;

        // Cache
        const cacheKey = `${CACHE_PREFIX}${hashText(batchTexts[j])}`;
        await redis.setex(cacheKey, CACHE_TTL, JSON.stringify(batchResults[j].embedding));
      }
    }
  }

  return results as number[][];
}
```

STEP 3: BULLMQ JOB QUEUES

Create file: `src/jobs/queue-config.ts`

```
// src/jobs/queue-config.ts
// BullMQ queue definitions for all background jobs

import { Queue, Worker, QueueEvents } from "bullmq";
import { redis } from "../lib/redis";

const connection = { connection: redis };

// ============================================================
// QUEUE DEFINITIONS — One queue per feature area
// ============================================================

export const knowledgeQueue = new Queue("knowledge", connection);     // Feature 2: Index
pages/tasks/messages/meetings
```

```
export const dailyPlanQueue = new Queue("daily-plan", connection);    // Feature 3: Generate
daily plans
export const agentQueue = new Queue("agent", connection);             // Feature 4: Execute
agent runs
export const riskDetectionQueue = new Queue("risk-detection", connection); // Feature 5: Scan
for project risks
export const meetingQueue = new Queue("meeting", connection);         // Feature 7: Transcribe
+ summarize meetings
export const calendarQueue = new Queue("calendar", connection);       // Feature 16: Calendar
sync + optimization
export const notificationQueue = new Queue("notification", connection); // Feature 20: Batch +
deliver notifications
export const workflowQueue = new Queue("workflow", connection);       // Feature 19: Execute
recorded workflows

// ============================================================
// DEFAULT JOB OPTIONS — applied to all jobs
// ============================================================

export const DEFAULT_JOB_OPTIONS = {
  attempts: 3,                    // retry 3 times on failure
  backoff: {
    type: "exponential" as const,
    delay: 5000,                  // 5s, 10s, 20s
  },
  removeOnComplete: {
    age: 86400,                   // keep completed jobs for 24h
    count: 1000,                  // keep last 1000
  },
  removeOnFail: {
    age: 604800,                  // keep failed jobs for 7 days
  },
};

// ============================================================
// RECURRING JOBS (cron schedules)
// ============================================================

export async function setupRecurringJobs() {
  // Feature 2: Full knowledge reindex — every Sunday 2 AM
  await knowledgeQueue.add(
    "full-reindex",
    {},
    {
```

```
    repeat: { pattern: "0 2 * * 0" }, // Sunday 2 AM
    ...DEFAULT_JOB_OPTIONS,
  }
);

// Feature 5: Risk detection scan — every 6 hours
await riskDetectionQueue.add(
  "scan-all-projects",
  {},
  {
    repeat: { pattern: "0 */6 * * *" }, // every 6 hours
    ...DEFAULT_JOB_OPTIONS,
  }
);

// Feature 16: Calendar sync — every 15 minutes
await calendarQueue.add(
  "sync-all-calendars",
  {},
  {
    repeat: { pattern: "*/15 * * * *" }, // every 15 min
    ...DEFAULT_JOB_OPTIONS,
  }
);

// Feature 20: Process notification batches — 3 times daily (9am, 1pm, 5pm UTC)
await notificationQueue.add(
  "batch-digest",
  {},
  {
    repeat: { pattern: "0 9,13,17 * * *" },
    ...DEFAULT_JOB_OPTIONS,
  }
);
}
```

STEP 4: SHARED AI UI COMPONENTS

Create file: `src/components/ai/ai-prompt-bar.tsx`

```
// src/components/ai/ai-prompt-bar.tsx
// Universal AI prompt bar — Cmd+J / Ctrl+J from anywhere
```

```
"use client";

import { useState, useRef, useEffect, useCallback } from "react";
import { Sparkles, Send, X, ChevronDown } from "lucide-react";
import { AiStreamingDisplay } from "./ai-streaming-display";
import { AiModelSelector } from "./ai-model-selector";

interface AiPromptBarProps {
  context?: string;          // page content, thread content, etc.
  contextLabel?: string;     // "This page", "This thread", etc.
  placeholder?: string;
  onResult?: (text: string) => void;
  onClose?: () => void;
  feature: string;           // 'writing','search','planner', etc. — for usage tracking
}

export function AiPromptBar({
  context,
  contextLabel,
  placeholder = "Ask AI anything...",
  onResult,
  onClose,
  feature,
}: AiPromptBarProps) {
  const [input, setInput] = useState("");
  const [isLoading, setIsLoading] = useState(false);
  const [streamedText, setStreamedText] = useState("");
  const [isComplete, setIsComplete] = useState(false);
  const [selectedModel, setSelectedModel] = useState<string>("auto");
  const inputRef = useRef<HTMLTextAreaElement>(null);

  useEffect(() => {
    inputRef.current?.focus();
  }, []);

  // Global keyboard shortcut: Cmd+J / Ctrl+J
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if ((e.metaKey || e.ctrlKey) && e.key === "j") {
        e.preventDefault();
        inputRef.current?.focus();
      }
      if (e.key === "Escape" && onClose) {
```

```
      onClose();
    }
  };
  document.addEventListener("keydown", handler);
  return () => document.removeEventListener("keydown", handler);
}, [onClose]);

const handleSubmit = useCallback(async () => {
  if (!input.trim() || isLoading) return;
  setIsLoading(true);
  setStreamedText("");
  setIsComplete(false);

  try {
    const response = await fetch("/api/trpc/ai-engine.stream", {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        prompt: input,
        context: context || undefined,
        model: selectedModel === "auto" ? undefined : selectedModel,
        feature,
      }),
    });

    if (!response.body) throw new Error("No response body");

    const reader = response.body.getReader();
    const decoder = new TextDecoder();
    let fullText = "";

    while (true) {
      const { done, value } = await reader.read();
      if (done) break;
      const chunk = decoder.decode(value, { stream: true });
      fullText += chunk;
      setStreamedText(fullText);
    }

    setIsComplete(true);
  } catch (error) {
    setStreamedText("Sorry, something went wrong. Please try again.");
    setIsComplete(true);
  } finally {
```

```
      setIsLoading(false);
    }
  }, [input, context, selectedModel, feature, isLoading]);

  return (
    <div className="border rounded-lg bg-white shadow-lg p-4 max-w-2xl w-full">
      {/* Context indicator */}
      {contextLabel && (
        <div className="text-xs text-gray-500 mb-2 flex items-center gap-1">
          <Sparkles className="w-3 h-3" />
          Context: {contextLabel}
        </div>
      )}

      {/* Input area */}
      <div className="flex items-end gap-2">
        <textarea
          ref={inputRef}
          value={input}
          onChange={(e) => setInput(e.target.value)}
          onKeyDown={(e) => {
            if (e.key === "Enter" && !e.shiftKey) {
              e.preventDefault();
              handleSubmit();
            }
          }}
          placeholder={placeholder}
          className="flex-1 resize-none border rounded-md p-2 text-sm min-h-[40px]
max-h-[120px] focus:outline-none focus:ring-2 focus:ring-blue-500"
          rows={1}
        />
        <AiModelSelector value={selectedModel} onChange={setSelectedModel} />
        <button
          onClick={handleSubmit}
          disabled={isLoading || !input.trim()}
          className="p-2 bg-blue-600 text-white rounded-md hover:bg-blue-700
disabled:opacity-50 disabled:cursor-not-allowed"
        >
          <Send className="w-4 h-4" />
        </button>
        {onClose && (
          <button onClick={onClose} className="p-2 text-gray-400 hover:text-gray-600">
            <X className="w-4 h-4" />
          </button>
```

```
      )}
    </div>

    {/* Streaming response */}
    {streamedText && (
      <div className="mt-3 border-t pt-3">
        <AiStreamingDisplay text={streamedText} isComplete={isComplete} />
        {isComplete && onResult && (
          <div className="mt-2 flex gap-2">
            <button
              onClick={() => onResult(streamedText)}
              className="text-sm px-3 py-1 bg-green-600 text-white rounded hover:bg-green-700"
            >
              ✅ Use this
            </button>
            <button
              onClick={() => {
                setStreamedText("");
                setIsComplete(false);
              }}
              className="text-sm px-3 py-1 bg-gray-200 text-gray-700 rounded
hover:bg-gray-300"
            >
              🔄 Try again
            </button>
            <button
              onClick={() => {
                setStreamedText("");
                setIsComplete(false);
                setInput("");
              }}
              className="text-sm px-3 py-1 text-gray-500 hover:text-gray-700"
            >
              ❌ Discard
            </button>
          </div>
        )}
      </div>
    )}
  </div>
);
}
```

Create file: `src/components/ai/ai-streaming-display.tsx`

```
// src/components/ai/ai-streaming-display.tsx
// Displays AI-generated text with typing animation

"use client";

import { useEffect, useRef } from "react";
import { Sparkles, Loader2 } from "lucide-react";

interface AiStreamingDisplayProps {
  text: string;
  isComplete: boolean;
  className?: string;
}

export function AiStreamingDisplay({ text, isComplete, className = "" }:
AiStreamingDisplayProps) {
  const containerRef = useRef<HTMLDivElement>(null);

  // Auto-scroll as text streams in
  useEffect(() => {
    if (containerRef.current) {
      containerRef.current.scrollTop = containerRef.current.scrollHeight;
    }
  }, [text]);

  return (
    <div className={`relative ${className}`}>
      <div className="flex items-center gap-1 text-xs text-purple-600 mb-1">
        <Sparkles className="w-3 h-3" />
        {isComplete ? "AI Response" : "Generating..."}
        {!isComplete && <Loader2 className="w-3 h-3 animate-spin" />}
      </div>
      <div
        ref={containerRef}
        className="text-sm text-gray-800 prose prose-sm max-h-[300px] overflow-y-auto
whitespace-pre-wrap"
      >
        {text}
        {!isComplete && <span className="animate-pulse">▌</span>}
      </div>
    </div>
  );
```

```
}
```

Create file: `src/components/ai/ai-model-selector.tsx`

```
// src/components/ai/ai-model-selector.tsx
// Dropdown for selecting AI model

"use client";

interface AiModelSelectorProps {
  value: string;
  onChange: (model: string) => void;
}

const MODELS = [
  { id: "auto", label: "Auto", description: "Best model for the task" },
  { id: "gpt-4o", label: "GPT-4o", description: "Balanced, versatile" },
  { id: "gpt-4o-mini", label: "GPT-4o Mini", description: "Faster, cheaper" },
  { id: "claude-sonnet", label: "Claude", description: "Strong writing" },
  { id: "gemini-pro", label: "Gemini", description: "Data analysis" },
];

export function AiModelSelector({ value, onChange }: AiModelSelectorProps) {
  return (
    <select
      value={value}
      onChange={(e) => onChange(e.target.value)}
      className="text-xs border rounded px-2 py-1 bg-gray-50 text-gray-600 cursor-pointer
hover:bg-gray-100"
    >
      {MODELS.map((m) => (
        <option key={m.id} value={m.id}>
          {m.label}
        </option>
      ))}
    </select>
  );
}
```

Create file: `src/components/ai/ai-diff-view.tsx`

```
// src/components/ai/ai-diff-view.tsx
// Before/After text comparison for AI edits (Feature 1: Writing Assistant)
```

```
"use client";

interface AiDiffViewProps {
  original: string;
  modified: string;
  onAccept: () => void;
  onReject: () => void;
}

export function AiDiffView({ original, modified, onAccept, onReject }: AiDiffViewProps) {
  return (
    <div className="border rounded-lg overflow-hidden">
      <div className="grid grid-cols-2 divide-x">
        <div className="p-3">
          <div className="text-xs font-medium text-red-600 mb-1">Original</div>
          <div className="text-sm text-gray-600 line-through
whitespace-pre-wrap">{original}</div>
        </div>
        <div className="p-3">
          <div className="text-xs font-medium text-green-600 mb-1">AI Suggestion</div>
          <div className="text-sm text-gray-800 bg-green-50 rounded p-2
whitespace-pre-wrap">{modified}</div>
        </div>
      </div>
      <div className="flex gap-2 p-2 bg-gray-50 border-t">
        <button
          onClick={onAccept}
          className="text-sm px-3 py-1 bg-green-600 text-white rounded hover:bg-green-700"
        >
          ✅ Accept
        </button>
        <button
          onClick={onReject}
          className="text-sm px-3 py-1 bg-gray-200 text-gray-700 rounded hover:bg-gray-300"
        >
          ❌ Reject
        </button>
      </div>
    </div>
  );
}
```

Create file: `src/components/ai/ai-suggestion-card.tsx`

```
// src/components/ai/ai-suggestion-card.tsx
// Subtle suggestion card (used in Knowledge Surfacing, Smart Notifications, etc.)

"use client";

import { Lightbulb, X } from "lucide-react";

interface AiSuggestionCardProps {
  message: string;
  sources?: { title: string; url: string }[];
  onAccept?: () => void;
  onDismiss?: () => void;
}

export function AiSuggestionCard({ message, sources, onAccept, onDismiss }:
AiSuggestionCardProps) {
  return (
    <div className="bg-blue-50 border border-blue-200 rounded-lg p-3 text-sm max-w-md">
      <div className="flex items-start gap-2">
        <Lightbulb className="w-4 h-4 text-blue-600 mt-0.5 flex-shrink-0" />
        <div className="flex-1">
          <p className="text-gray-800">{message}</p>
          {sources && sources.length > 0 && (
            <div className="mt-1 space-y-0.5">
              {sources.map((s, i) => (
                <a key={i} href={s.url} className="text-xs text-blue-600 hover:underline block">
                  {s.title}
                </a>
              ))}
            </div>
          )}
          <div className="mt-2 flex gap-2">
            {onAccept && (
              <button
                onClick={onAccept}
                className="text-xs px-2 py-0.5 bg-blue-600 text-white rounded hover:bg-blue-700"
              >
                View Source
              </button>
            )}
            {onDismiss && (
              <button
```

```
            onClick={onDismiss}
            className="text-xs text-gray-500 hover:text-gray-700"
          >
            Dismiss
          </button>
        )}
      </div>
    </div>
    {onDismiss && (
      <button onClick={onDismiss} className="text-gray-400 hover:text-gray-600">
        <X className="w-3 h-3" />
      </button>
    )}
  </div>
</div>
);
}
```

Create file: `src/components/ai/ai-loading.tsx`

```
// src/components/ai/ai-loading.tsx
// AI processing states

"use client";

import { Sparkles, Loader2 } from "lucide-react";

interface AiLoadingProps {
  message?: string;
  size?: "sm" | "md" | "lg";
}

export function AiLoading({ message = "AI is thinking...", size = "md" }: AiLoadingProps) {
  const sizeClasses = {
    sm: "text-xs gap-1",
    md: "text-sm gap-2",
    lg: "text-base gap-2",
  };

  return (
    <div className={`flex items-center ${sizeClasses[size]} text-purple-600`}>
      <Sparkles className={size === "sm" ? "w-3 h-3" : "w-4 h-4"} />
      <span>{message}</span>
```

```
      <Loader2 className={`animate-spin ${size === "sm" ? "w-3 h-3" : "w-4 h-4"}`} />
    </div>
  );
}
```

STEP 5: AI USAGE LOGGING MIDDLEWARE

Create file: `src/server/services/ai-usage-logger.ts`

```
// src/server/services/ai-usage-logger.ts
// Logs all AI usage for analytics. Called after every AI operation.

import { prisma } from "@/lib/prisma";

interface LogAiUsageParams {
  userId: string;
  workspaceId: string;
  feature: string;
  action: string;
  model: string;
  inputTokens: number;
  outputTokens: number;
  durationMs?: number;
  success?: boolean;
  error?: string;
}

export async function logAiUsage(params: LogAiUsageParams): Promise<void> {
  try {
    await prisma.aiUsageLog.create({
      data: {
        userId: params.userId,
        workspaceId: params.workspaceId,
        feature: params.feature,
        action: params.action,
        model: params.model,
        inputTokens: params.inputTokens,
        outputTokens: params.outputTokens,
        durationMs: params.durationMs,
        success: params.success ?? true,
        error: params.error,
      },
```

```
    });
  } catch (err) {
    // Never throw from logging — don't break the feature because logging failed
    console.error("Failed to log AI usage:", err);
  }
}
```

STEP 6: KNOWLEDGE INDEXING PIPELINE (Feature 2 — Backend Only)

Create file: `src/server/services/knowledge-indexer.ts`

```
// src/server/services/knowledge-indexer.ts
// Indexes pages, tasks, messages, and meetings for AI search (Feature 2)

import { prisma } from "@/lib/prisma";
import { getEmbedding, getEmbeddingBatch } from "@/lib/ai/embeddings";

// ==============================================================
// INDEX A SINGLE PAGE
// ==============================================================

export async function indexPage(pageId: string, workspaceId: string): Promise<void> {
  const page = await prisma.page.findUnique({
    where: { id: pageId },
    include: { blocks: true }, // adjust to your actual page/block schema
  });
  if (!page) return;

  // Extract plain text from all blocks
  const content = extractTextFromBlocks(page.blocks);
  if (!content.trim()) return;

  // Generate embedding
  const embedding = await getEmbedding(content.slice(0, 8000)); // limit to ~8k chars for
embedding

  // Chunk long content for better search precision
  const chunks = content.length > 2000 ? chunkText(content, 500) : null;

  // Upsert into knowledge index
```

```
  await prisma.$executeRaw`
    INSERT INTO "KnowledgeIndex" (
      id, "workspaceId", "sourceType", "sourceId", title, content,
      "contentChunks", embedding, metadata, "accessLevel",
      "lastIndexedAt", "isStale", "createdAt", "updatedAt"
    ) VALUES (
      gen_random_uuid(), ${workspaceId}, 'page', ${pageId},
      ${page.title || "Untitled"}, ${content},
      ${chunks ? JSON.stringify(chunks) : null}::jsonb,
      ${embedding}::vector,
      ${JSON.stringify({
        authorId: page.createdBy,
        createdDate: page.createdAt,
        lastModifiedDate: page.updatedAt,
      })}::jsonb,
      'workspace',
      NOW(), false, NOW(), NOW()
    )
    ON CONFLICT ("sourceId") DO UPDATE SET
      title = EXCLUDED.title,
      content = EXCLUDED.content,
      "contentChunks" = EXCLUDED."contentChunks",
      embedding = EXCLUDED.embedding,
      metadata = EXCLUDED.metadata,
      "lastIndexedAt" = NOW(),
      "isStale" = false,
      "updatedAt" = NOW()
  `;
}

// ==============================================================
// INDEX MESSAGES IN BATCH (every 5 minutes)
// ==============================================================

export async function indexMessageBatch(
  messages: { id: string; content: string; channelId: string; authorId: string; authorName: string;
createdAt: Date }[],
  workspaceId: string
): Promise<void> {
  if (messages.length === 0) return;

  const texts = messages.map((m) => m.content);
  const embeddings = await getEmbeddingBatch(texts);
```

```
  for (let i = 0; i < messages.length; i++) {
    const msg = messages[i];
    await prisma.$executeRaw`
      INSERT INTO "KnowledgeIndex" (
        id, "workspaceId", "sourceType", "sourceId", "parentSourceId",
        title, content, embedding, metadata, "accessLevel",
        "lastIndexedAt", "isStale", "createdAt", "updatedAt"
      ) VALUES (
        gen_random_uuid(), ${workspaceId}, 'message', ${msg.id}, ${msg.channelId},
        ${msg.content.slice(0, 100)}, ${msg.content},
        ${embeddings[i]}::vector,
        ${JSON.stringify({
          authorId: msg.authorId,
          authorName: msg.authorName,
          channelId: msg.channelId,
          createdDate: msg.createdAt,
        })}::jsonb,
        'channel_members',
        NOW(), false, NOW(), NOW()
      )
      ON CONFLICT ("sourceId") DO NOTHING
    `;
  }
}

// ============================================================
// INDEX A TASK
// ============================================================

export async function indexTask(taskId: string, workspaceId: string): Promise<void> {
  const task = await prisma.task.findUnique({ where: { id: taskId } });
  if (!task) return;

  const content = `${task.title}\\n${task.description || ""}`;
  const embedding = await getEmbedding(content);

  await prisma.$executeRaw`
    INSERT INTO "KnowledgeIndex" (
      id, "workspaceId", "sourceType", "sourceId",
      title, content, embedding, metadata, "accessLevel",
      "lastIndexedAt", "isStale", "createdAt", "updatedAt"
    ) VALUES (
      gen_random_uuid(), ${workspaceId}, 'task', ${taskId},
      ${task.title}, ${content},
```

```
      ${embedding}::vector,
      ${JSON.stringify({
        authorId: task.createdBy,
        createdDate: task.createdAt,
        lastModifiedDate: task.updatedAt,
      })}::jsonb,
      'workspace',
      NOW(), false, NOW(), NOW()
    )
    ON CONFLICT ("sourceId") DO UPDATE SET
      title = EXCLUDED.title,
      content = EXCLUDED.content,
      embedding = EXCLUDED.embedding,
      metadata = EXCLUDED.metadata,
      "lastIndexedAt" = NOW(),
      "isStale" = false,
      "updatedAt" = NOW()
  `;
}

// ==============================================================
// HELPER FUNCTIONS
// ==============================================================

function extractTextFromBlocks(blocks: any[]): string {
  // Adapt this to your actual block structure
  return blocks
    .map((block) => {
      if (typeof block.content === "string") return block.content;
      if (block.content?.text) return block.content.text;
      if (block.type === "heading") return block.content?.text || "";
      if (block.type === "paragraph") return block.content?.text || "";
      if (block.type === "list") {
        return (block.content?.items || []).map((item: any) => item.text || "").join("\\n");
      }
      return "";
    })
    .filter(Boolean)
    .join("\\n");
}

function chunkText(text: string, chunkSize: number): { chunkIndex: number; text: string }[] {
  const chunks: { chunkIndex: number; text: string }[] = [];
  const sentences = text.split(/(?<=[.!?])\\s+/);
```

```
  let current = "";
  let index = 0;

  for (const sentence of sentences) {
    if (current.length + sentence.length > chunkSize && current.length > 0) {
      chunks.push({ chunkIndex: index++, text: current.trim() });
      current = "";
    }
    current += sentence + " ";
  }
  if (current.trim()) {
    chunks.push({ chunkIndex: index, text: current.trim() });
  }

  return chunks;
}
```

SPRINT 1 VERIFICATION CHECKLIST

After building everything above, verify:

□ npx prisma migrate dev --name add_all_ai_features → runs without errors
□ npx prisma generate → creates client successfully
□ Import aiService and call generateText("Hello") → returns response
□ Import aiService and call generateEmbedding("test") → returns 1536-dimension vector
□ BullMQ queues initialize without errors (check Redis connection)
□ All shared components render (create a test page that imports each one)
□ TypeScript compilation: npx tsc --noEmit → zero errors
□ Redis connection works: import redis, call redis.ping() → "PONG"

Once all 8 checks pass, proceed to Sprint 2.

SPRINT 2 — AI WRITING ASSISTANT + AI KNOWLEDGE SEARCH

Features 1 and 2 — Full Implementation

PREREQUISITE: Sprint 1 must be complete and verified

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Feature 1: AI Writing Assistant (all 4 triggers + role-based writing + brand voice)
2. Feature 2: AI Knowledge Search (universal search UI + Ask AI mode + proactive suggestions)

After this sprint, verify:

- Press Space on empty editor line → AI prompt bar appears → type request → AI streams response into editor
- Select text → click AI button → choose "Improve" → see diff view → Accept/Reject
- Press Cmd+J → AI prompt bar opens → ask about workspace data → get answer with citations
- Press Cmd+K → universal search → type keyword → see results from pages/tasks/messages
- Switch to "Ask AI" tab → type natural language question → get AI answer with sources

---

FEATURE 1: AI WRITING ASSISTANT

1.1 tRPC Router

Create file: `src/server/routers/ai-writing.ts`

```
// src/server/routers/ai-writing.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
```

```
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logger";
import { prisma } from "@/lib/prisma";
import { observable } from "@trpc/server/observable";

// ==============================================================
// INPUT SCHEMAS
// ==============================================================

const writeActionSchema = z.enum([
  "write", "improve", "shorten", "lengthen", "tone", "grammar",
  "translate", "summarize", "brainstorm", "outline", "continue",
  "explain", "action_items", "custom",
]);

const toneSchema = z.enum([
  "professional", "casual", "friendly", "academic", "persuasive",
]);

const writerRoleSchema = z.enum([
  "general", "marketer", "engineer", "executive", "designer", "sales", "support", "hr",
]);

// ==============================================================
// SYSTEM PROMPTS (one per action)
// ==============================================================

const ACTION_PROMPTS: Record<string, string> = {
  write: "You are an expert writer. Generate clear, well-structured content based on the user's
request. Write in a natural, engaging style.",
  improve: "You are an expert editor. Improve the following text to be clearer, more professional,
and better structured. Keep the same meaning but make it significantly better.",
  shorten: "You are an expert editor. Condense the following text to be shorter while keeping all
key information. Target 50-60% of the original length.",
  lengthen: "You are an expert writer. Expand the following text with more detail, examples, and
explanation. Target 150-200% of the original length.",
  tone: "You are an expert editor. Rewrite the following text in the specified tone while keeping
the same meaning and information.",
  grammar: "You are a grammar expert. Fix all spelling, grammar, and punctuation errors. Make
minimal changes — only fix errors, do not change style or meaning.",
  translate: "You are an expert translator. Translate the following text accurately into the
specified language. Preserve formatting and tone.",
  summarize: "You are an expert summarizer. Create a clear, concise summary of the following
text in 2-4 sentences. Capture the key points.",
```

```
  brainstorm: "You are a creative brainstorming expert. Generate 5-10 unique, practical ideas
based on the topic. Each idea should be one sentence with a brief explanation.",
  outline: "You are a document structuring expert. Create a clear, hierarchical outline with H2
headings and bullet points for the given topic.",
  continue: "You are an expert writer. Continue writing from where the text ends. Match the
existing style, tone, and topic seamlessly.",
  explain: "You are an expert explainer. Provide a clear, easy-to-understand explanation of the
following text. Use simple language and examples if helpful.",
  action_items: "You are a task extraction expert. Extract all action items, tasks, and to-dos from
the following text. Format each as a clear, actionable checkbox item.",
  custom: "You are an expert AI assistant helping with writing and editing.",
};

const ROLE_PROMPTS: Record<string, string> = {
  general: "",
  marketer: "Write in a persuasive, benefit-focused style. Use strong calls-to-action. Focus on
value propositions and customer impact.",
  engineer: "Write technically precise content. Use proper technical terminology. Include
examples and edge cases where relevant.",
  executive: "Write concise, strategic content. Focus on business impact, decisions, and
outcomes. Use data when available.",
  designer: "Write with attention to user experience. Focus on usability, aesthetics, and user
journey. Use visual language.",
  sales: "Write in a convincing, relationship-focused style. Highlight benefits, social proof, and
urgency. Be warm but professional.",
  support: "Write in a helpful, empathetic style. Be clear and solution-oriented. Anticipate
follow-up questions.",
  hr: "Write in an inclusive, professional style. Be clear about policies and expectations. Use
respectful, people-first language.",
};

function buildSystemPrompt(
  action: string,
  role?: string,
  tone?: string,
  brandVoice?: { defaultTone?: string; vocabularyRules?: any; formattingRules?: string } | null,
): string {
  let prompt = ACTION_PROMPTS[action] || ACTION_PROMPTS.custom;

  if (role && role !== "general" && ROLE_PROMPTS[role]) {
    prompt += `\\n\\nWriting perspective: ${ROLE_PROMPTS[role]}`;
  }

  if (tone) {
```

```
    prompt += `\\n\\nTone: Write in a ${tone} tone.`;
  }

  if (brandVoice) {
    if (brandVoice.defaultTone && !tone) {
      prompt += `\\n\\nDefault tone: ${brandVoice.defaultTone}`;
    }
    if (brandVoice.vocabularyRules) {
      const rules = brandVoice.vocabularyRules as Array<{ use: string; notUse: string }>;
      if (rules.length > 0) {
        prompt += `\\n\\nVocabulary rules: ${rules.map(r => `Say "${r.use}" instead of
"${r.notUse}"`).join(". ")}`;
      }
    }
    if (brandVoice.formattingRules) {
      prompt += `\\n\\nFormatting rules: ${brandVoice.formattingRules}`;
    }
  }

  return prompt;
}

function buildUserPrompt(action: string, inputText?: string, context?: string, customPrompt?:
string, language?: string): string {
  switch (action) {
    case "write":
      return customPrompt || "Write about this topic.";
    case "improve":
    case "shorten":
    case "lengthen":
    case "grammar":
      return `Text to ${action}:\\n\\n${inputText}`;
    case "tone":
      return `Rewrite in the specified tone:\\n\\n${inputText}`;
    case "translate":
      return `Translate to ${language || "Spanish"}:\\n\\n${inputText}`;
    case "summarize":
      return `Summarize this:\\n\\n${inputText || context}`;
    case "brainstorm":
      return `Brainstorm ideas about: ${customPrompt || inputText}`;
    case "outline":
      return `Create an outline for: ${customPrompt || inputText}`;
    case "continue":
      return `Continue writing from here:\\n\\n${context?.slice(-2000) || inputText}`;
```

```
    case "explain":
      return `Explain this:\\n\\n${inputText}`;
    case "action_items":
      return `Extract action items from:\\n\\n${inputText || context}`;
    case "custom":
      return `${customPrompt}\\n\\n${inputText ? `Text:\\n${inputText}` : ""}${context ?
`\\nContext:\\n${context.slice(-3000)}` : ""}`;
    default:
      return customPrompt || inputText || "";
  }
}

// ================================================================
// tRPC ROUTER
// ================================================================

export const aiWritingRouter = router({
  /**
   * Generate AI writing (streaming via SSE endpoint — see API route below)
   * This non-streaming version is for programmatic use (agents, etc.)
   */
  generate: protectedProcedure
    .input(z.object({
      action: writeActionSchema,
      inputText: z.string().optional(),
      context: z.string().optional(),
      customPrompt: z.string().optional(),
      tone: toneSchema.optional(),
      writerRole: writerRoleSchema.optional(),
      language: z.string().optional(),
      model: z.string().optional(),
      workspaceId: z.string().uuid(),
      pageId: z.string().uuid().optional(),
      channelId: z.string().uuid().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const startTime = Date.now();

      // Load brand voice for workspace
      const brandVoice = await prisma.brandVoice.findUnique({
        where: { workspaceId: input.workspaceId },
      });

      const systemPrompt = buildSystemPrompt(
```

```
  input.action,
  input.writerRole,
  input.tone,
  brandVoice,
);

const userPrompt = buildUserPrompt(
  input.action,
  input.inputText,
  input.context,
  input.customPrompt,
  input.language,
);

const maxTokensMap: Record<string, number> = {
  summarize: 500, brainstorm: 1000, grammar: 1500,
  shorten: 1000, action_items: 800, write: 2000,
  outline: 1500, continue: 1500, improve: 2000,
  lengthen: 3000, translate: 2000, explain: 1000,
  tone: 2000, custom: 2000,
};

const temperatureMap: Record<string, number> = {
  grammar: 0.1, translate: 0.2, summarize: 0.3,
  action_items: 0.2, improve: 0.4, shorten: 0.3,
  lengthen: 0.5, tone: 0.4, write: 0.7,
  brainstorm: 0.9, outline: 0.5, continue: 0.7,
  explain: 0.4, custom: 0.7,
};

const result = await aiService.generateText(userPrompt, {
  systemPrompt,
  model: (input.model as any) || undefined,
  maxTokens: maxTokensMap[input.action] || 2000,
  temperature: temperatureMap[input.action] || 0.7,
});

// Save to history
const history = await prisma.aiWritingHistory.create({
  data: {
    userId: ctx.user.id,
    workspaceId: input.workspaceId,
    pageId: input.pageId,
    channelId: input.channelId,
```

```
        action: input.action,
        inputText: input.inputText,
        prompt: userPrompt,
        outputText: result.text,
        model: input.model || "gpt-4o",
        tone: input.tone,
        language: input.language,
        writerRole: input.writerRole,
        tokensUsed: result.tokensUsed.input + result.tokensUsed.output,
      },
    });

    // Log usage
    await logAiUsage({
      userId: ctx.user.id,
      workspaceId: input.workspaceId,
      feature: "writing",
      action: input.action,
      model: input.model || "gpt-4o",
      inputTokens: result.tokensUsed.input,
      outputTokens: result.tokensUsed.output,
      durationMs: Date.now() - startTime,
    });

    return {
      id: history.id,
      text: result.text,
      tokensUsed: result.tokensUsed,
    };
  }),

/**
 * Mark a writing result as accepted (user clicked "Keep")
 */
acceptResult: protectedProcedure
  .input(z.object({ id: z.string().uuid() }))
  .mutation(async ({ input }) => {
    await prisma.aiWritingHistory.update({
      where: { id: input.id },
      data: { accepted: true },
    });
  }),

/**
```

```
 * Rate a writing result (1-5 stars)
 */
rateResult: protectedProcedure
  .input(z.object({ id: z.string().uuid(), rating: z.number().min(1).max(5) }))
  .mutation(async ({ input }) => {
    await prisma.aiWritingHistory.update({
      where: { id: input.id },
      data: { feedbackRating: input.rating },
    });
  }),

/**
 * Get writing history for a user
 */
getHistory: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    limit: z.number().min(1).max(50).default(20),
    cursor: z.string().uuid().optional(),
  }))
  .query(async ({ input, ctx }) => {
    const items = await prisma.aiWritingHistory.findMany({
      where: { userId: ctx.user.id, workspaceId: input.workspaceId },
      orderBy: { createdAt: "desc" },
      take: input.limit + 1,
      ...(input.cursor ? { cursor: { id: input.cursor }, skip: 1 } : {}),
    });

    const hasMore = items.length > input.limit;
    return {
      items: items.slice(0, input.limit),
      nextCursor: hasMore ? items[input.limit - 1].id : undefined,
    };
  }),

/**
 * Save/update brand voice settings
 */
saveBrandVoice: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    defaultTone: z.string().optional(),
    styleGuideText: z.string().optional(),
    vocabularyRules: z.array(z.object({ use: z.string(), notUse: z.string() })).optional(),
```

```
    formattingRules: z.string().optional(),
    customRoles: z.array(z.object({ name: z.string(), systemPrompt: z.string() })).optional(),
  }))
  .mutation(async ({ input }) => {
    return prisma.brandVoice.upsert({
      where: { workspaceId: input.workspaceId },
      create: {
        workspaceId: input.workspaceId,
        defaultTone: input.defaultTone || "professional",
        styleGuideText: input.styleGuideText,
        vocabularyRules: input.vocabularyRules,
        formattingRules: input.formattingRules,
        customRoles: input.customRoles,
      },
      update: {
        defaultTone: input.defaultTone,
        styleGuideText: input.styleGuideText,
        vocabularyRules: input.vocabularyRules,
        formattingRules: input.formattingRules,
        customRoles: input.customRoles,
      },
    });
  }),

 /**
  * Get brand voice settings
  */
 getBrandVoice: protectedProcedure
  .input(z.object({ workspaceId: z.string().uuid() }))
  .query(async ({ input }) => {
    return prisma.brandVoice.findUnique({
      where: { workspaceId: input.workspaceId },
    });
  }),
});
```

1.2 Streaming API Route (for real-time text generation)

Create file: `src/app/api/ai/writing/stream/route.ts`

```
// src/app/api/ai/writing/stream/route.ts
// Server-Sent Events endpoint for streaming AI writing responses
```

```
import { NextRequest, NextResponse } from "next/server";
import { getServerSession } from "next-auth";
import { authOptions } from "@/lib/auth";
import { aiService } from "@/lib/ai/ai-service";
import { prisma } from "@/lib/prisma";

export async function POST(req: NextRequest) {
  const session = await getServerSession(authOptions);
  if (!session?.user?.id) {
    return NextResponse.json({ error: "Unauthorized" }, { status: 401 });
  }

  const body = await req.json();
  const { action, inputText, context, customPrompt, tone, writerRole, language, model,
workspaceId } = body;

  // Load brand voice
  const brandVoice = await prisma.brandVoice.findUnique({
    where: { workspaceId },
  });

  // Build prompts (reuse same logic from router)
  const ACTION_PROMPTS: Record<string, string> = {
    write: "You are an expert writer. Generate clear, well-structured content based on the user's
request.",
    improve: "You are an expert editor. Improve the following text to be clearer and better
structured.",
    shorten: "Condense the following text to be shorter while keeping all key information.",
    lengthen: "Expand the following text with more detail and examples.",
    grammar: "Fix all spelling, grammar, and punctuation errors. Make minimal changes.",
    translate: "Translate accurately into the specified language. Preserve formatting and tone.",
    summarize: "Create a concise summary in 2-4 sentences.",
    brainstorm: "Generate 5-10 unique, practical ideas.",
    outline: "Create a clear, hierarchical outline with headings and bullet points.",
    continue: "Continue writing from where the text ends. Match the existing style.",
    explain: "Provide a clear, easy-to-understand explanation.",
    action_items: "Extract all action items as clear, actionable checkbox items.",
    custom: "You are an expert AI assistant.",
  };

  let systemPrompt = ACTION_PROMPTS[action] || ACTION_PROMPTS.custom;
  if (tone) systemPrompt += ` Write in a ${tone} tone.`;
  if (writerRole && writerRole !== "general") systemPrompt += ` Writing as a ${writerRole}.`;
```

```
  let userPrompt = customPrompt || inputText || "";
  if (action === "improve" && inputText) userPrompt = `Improve this text:\\n\\n${inputText}`;
  if (action === "translate") userPrompt = `Translate to ${language}:\\n\\n${inputText}`;
  if (action === "continue" && context) userPrompt = `Continue
from:\\n\\n${context.slice(-2000)}`;

  // Stream response using ReadableStream
  const encoder = new TextEncoder();
  const stream = new ReadableStream({
    async start(controller) {
      try {
        const generator = aiService.generateStream(userPrompt, {
          systemPrompt,
          model: model || undefined,
          maxTokens: 2000,
          temperature: 0.7,
        });

        for await (const chunk of generator) {
          if (chunk.text) {
            controller.enqueue(encoder.encode(chunk.text));
          }
          if (chunk.done) {
            controller.close();
            return;
          }
        }
        controller.close();
      } catch (error) {
        controller.enqueue(encoder.encode("\\n\\n[Error: AI generation failed. Please try again.]"));
        controller.close();
      }
    },
  });

  return new NextResponse(stream, {
    headers: {
      "Content-Type": "text/plain; charset=utf-8",
      "Cache-Control": "no-cache",
      "Transfer-Encoding": "chunked",
    },
  });
}
```

1.3 Writing Assistant UI — Editor Integration

Create file: `src/components/ai/ai-writing-toolbar.tsx`

```
// src/components/ai/ai-writing-toolbar.tsx
// Floating toolbar that appears when text is selected in the editor

"use client";

import { useState } from "react";
import { Sparkles, ChevronDown } from "lucide-react";
import {
  DropdownMenu, DropdownMenuTrigger, DropdownMenuContent,
  DropdownMenuItem, DropdownMenuSub, DropdownMenuSubTrigger,
  DropdownMenuSubContent, DropdownMenuSeparator,
} from "@/components/ui/dropdown-menu";

interface AiWritingToolbarProps {
  selectedText: string;
  onAction: (action: string, options?: { tone?: string; language?: string; customPrompt?: string })
=> void;
}

const LANGUAGES = [
  "Spanish", "Hindi", "French", "German", "Japanese", "Chinese",
  "Portuguese", "Arabic", "Korean", "Russian", "Italian", "Dutch",
  "Turkish", "Vietnamese", "Thai", "Polish", "Swedish", "Indonesian",
  "Bengali", "Tamil", "Telugu", "Marathi",
];

const TONES = [
  { value: "professional", label: "Professional" },
  { value: "casual", label: "Casual" },
  { value: "friendly", label: "Friendly" },
  { value: "academic", label: "Academic" },
  { value: "persuasive", label: "Persuasive" },
];

export function AiWritingToolbar({ selectedText, onAction }: AiWritingToolbarProps) {
  const [customPrompt, setCustomPrompt] = useState("");
  const [showCustomInput, setShowCustomInput] = useState(false);

  if (!selectedText) return null;
```

```
  return (
    <DropdownMenu>
      <DropdownMenuTrigger asChild>
        <button className="flex items-center gap-1 px-2 py-1 text-sm bg-purple-600 text-white
rounded hover:bg-purple-700">
          <Sparkles className="w-3 h-3" />
          AI
          <ChevronDown className="w-3 h-3" />
        </button>
      </DropdownMenuTrigger>
      <DropdownMenuContent align="start" className="w-56">
        <DropdownMenuItem onClick={() => onAction("improve")}>
          ✨ Improve writing
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("grammar")}>
          📝 Fix spelling & grammar
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("shorten")}>
          📐 Make shorter
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("lengthen")}>
          📏 Make longer
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuSub>
          <DropdownMenuSubTrigger>🎭 Change tone</DropdownMenuSubTrigger>
          <DropdownMenuSubContent>
            {TONES.map((t) => (
              <DropdownMenuItem key={t.value} onClick={() => onAction("tone", { tone: t.value })}>
                {t.label}
              </DropdownMenuItem>
            ))}
          </DropdownMenuSubContent>
        </DropdownMenuSub>
        <DropdownMenuItem onClick={() => onAction("explain")}>
          💡 Simplify language
        </DropdownMenuItem>
        <DropdownMenuSeparator />
        <DropdownMenuItem onClick={() => onAction("summarize")}>
          📋 Summarize
        </DropdownMenuItem>
        <DropdownMenuItem onClick={() => onAction("explain")}>
          🔍 Explain this
        </DropdownMenuItem>
```

```
      <DropdownMenuSub>
        <DropdownMenuSubTrigger>🌍 Translate</DropdownMenuSubTrigger>
        <DropdownMenuSubContent className="max-h-[300px] overflow-y-auto">
          {LANGUAGES.map((lang) => (
            <DropdownMenuItem key={lang} onClick={() => onAction("translate", { language: lang
})}>
              {lang}
            </DropdownMenuItem>
          ))}
        </DropdownMenuSubContent>
      </DropdownMenuSub>
      <DropdownMenuItem onClick={() => onAction("action_items")}>
        ☑️ Extract action items
      </DropdownMenuItem>
      <DropdownMenuSeparator />
      <DropdownMenuItem onClick={() => setShowCustomInput(true)}>
        ✏️ Custom instruction...
      </DropdownMenuItem>
    </DropdownMenuContent>

    {/* Custom prompt modal */}
    {showCustomInput && (
      <div className="fixed inset-0 bg-black/50 flex items-center justify-center z-50">
        <div className="bg-white rounded-lg p-4 w-96 shadow-xl">
          <h3 className="text-sm font-medium mb-2">Custom AI instruction</h3>
          <textarea
            value={customPrompt}
            onChange={(e) => setCustomPrompt(e.target.value)}
            placeholder="Tell AI what to do with the selected text..."
            className="w-full border rounded p-2 text-sm h-24 resize-none focus:outline-none
focus:ring-2 focus:ring-purple-500"
            autoFocus
          />
          <div className="flex justify-end gap-2 mt-2">
            <button
              onClick={() => setShowCustomInput(false)}
              className="text-sm px-3 py-1 text-gray-600 hover:text-gray-800"
            >
              Cancel
            </button>
            <button
              onClick={() => {
                onAction("custom", { customPrompt });
                setShowCustomInput(false);
```

```
                setCustomPrompt("");
              }}
              className="text-sm px-3 py-1 bg-purple-600 text-white rounded
hover:bg-purple-700"
            >
              Run AI
            </button>
          </div>
        </div>
      </div>
    )}
  </DropdownMenu>
);
}
```

1.4 Slash Command AI Menu

Create file: `src/components/ai/ai-slash-menu.tsx`

```
// src/components/ai/ai-slash-menu.tsx
// Shows AI options when user types "/" in editor

"use client";

import { useState } from "react";
import { Sparkles, FileText, Lightbulb, List, ArrowRight, Languages, CheckSquare, Table, Code }
from "lucide-react";

interface SlashMenuItem {
  id: string;
  label: string;
  description: string;
  icon: React.ReactNode;
  action: string;
}

const AI_SLASH_ITEMS: SlashMenuItem[] = [
  { id: "write", label: "AI Write", description: "Generate content from a prompt", icon: <Sparkles
className="w-4 h-4" />, action: "write" },
  { id: "brainstorm", label: "AI Brainstorm", description: "Generate 5-10 ideas", icon: <Lightbulb
className="w-4 h-4" />, action: "brainstorm" },
  { id: "outline", label: "AI Outline", description: "Create structured outline", icon: <List
className="w-4 h-4" />, action: "outline" },
```

```
  { id: "continue", label: "AI Continue", description: "Continue writing from here", icon:
<ArrowRight className="w-4 h-4" />, action: "continue" },
  { id: "summarize", label: "AI Summarize", description: "Summarize this page", icon: <FileText
className="w-4 h-4" />, action: "summarize" },
  { id: "translate", label: "AI Translate", description: "Translate page content", icon: <Languages
className="w-4 h-4" />, action: "translate" },
  { id: "action_items", label: "AI Action Items", description: "Extract tasks from page", icon:
<CheckSquare className="w-4 h-4" />, action: "action_items" },
  { id: "table", label: "AI Table", description: "Generate a data table", icon: <Table
className="w-4 h-4" />, action: "write" },
  { id: "code", label: "AI Code", description: "Generate code block", icon: <Code className="w-4
h-4" />, action: "write" },
];

interface AiSlashMenuProps {
  filter: string; // what the user typed after "/"
  onSelect: (action: string, label: string) => void;
  onClose: () => void;
}

export function AiSlashMenu({ filter, onSelect, onClose }: AiSlashMenuProps) {
  const filtered = AI_SLASH_ITEMS.filter(
    (item) =>
      item.label.toLowerCase().includes(filter.toLowerCase()) ||
      item.description.toLowerCase().includes(filter.toLowerCase())
  );

  const [selectedIndex, setSelectedIndex] = useState(0);

  if (filtered.length === 0) return null;

  return (
    <div className="absolute z-50 bg-white border rounded-lg shadow-lg w-72 max-h-80
overflow-y-auto">
      <div className="p-1.5 text-xs text-gray-500 font-medium border-b">AI Actions</div>
      {filtered.map((item, index) => (
        <button
          key={item.id}
          onClick={() => onSelect(item.action, item.label)}
          className={`w-full flex items-center gap-3 px-3 py-2 text-left hover:bg-gray-100 ${
            index === selectedIndex ? "bg-gray-100" : ""
          }`}
        >
          <span className="text-purple-600">{item.icon}</span>
```

```
        <div>
          <div className="text-sm font-medium">{item.label}</div>
          <div className="text-xs text-gray-500">{item.description}</div>
        </div>
      </button>
    ))}
  </div>
);
}
```

FEATURE 2: AI KNOWLEDGE SEARCH

2.1 tRPC Router

Create file: `src/server/routers/knowledge.ts`

```
// src/server/routers/knowledge.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { getEmbedding } from "@/lib/ai/embeddings";
import { logAiUsage } from "@/server/services/ai-usage-logger";

export const knowledgeRouter = router({
  /**
   * Keyword search — instant results as user types (for Search tab)
   */
  search: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      query: z.string().min(1).max(500),
      sourceType: z.string().optional(), // filter by type
      limit: z.number().min(1).max(50).default(20),
    }))
    .query(async ({ input, ctx }) => {
      // Full-text search using PostgreSQL ts_vector
      const results = await prisma.$queryRaw<Array<{
        id: string;
        sourceType: string;
```

```
      sourceId: string;
      title: string;
      content: string;
      metadata: any;
      accessLevel: string;
      rank: number;
    }>>`
      SELECT
        id, "sourceType", "sourceId", title,
        ts_headline('english', content, plainto_tsquery('english', ${input.query}),
          'StartSel=<mark>, StopSel=</mark>, MaxFragments=2, MaxWords=40') as content,
        metadata, "accessLevel",
        ts_rank(to_tsvector('english', content), plainto_tsquery('english', ${input.query})) as rank
      FROM "KnowledgeIndex"
      WHERE "workspaceId" = ${input.workspaceId}::uuid
        AND to_tsvector('english', content) @@ plainto_tsquery('english', ${input.query})
        ${input.sourceType ? prisma.$queryRaw`AND "sourceType" = ${input.sourceType}` :
prisma.$queryRaw``}
      ORDER BY rank DESC
      LIMIT ${input.limit}
    `;

    // TODO: Add permission filtering based on accessLevel and user's channel memberships

    // Group results by sourceType
    const grouped: Record<string, typeof results> = {};
    for (const r of results) {
      if (!grouped[r.sourceType]) grouped[r.sourceType] = [];
      grouped[r.sourceType].push(r);
    }

    return { results, grouped, totalCount: results.length };
  }),

  /**
   * AI Q&A — natural language question → AI-generated answer with citations
   */
  askAi: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      query: z.string().min(1).max(1000),
      model: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
```

```
    const startTime = Date.now();

    // Step 1: Generate embedding for the question
    const queryEmbedding = await getEmbedding(input.query);

    // Step 2: Semantic search — find most relevant content
    const semanticResults = await prisma.$queryRaw<Array<{
      id: string;
      sourceType: string;
      sourceId: string;
      title: string;
      content: string;
      metadata: any;
      similarity: number;
    }>>`
      SELECT
        id, "sourceType", "sourceId", title,
        LEFT(content, 1000) as content,
        metadata,
        1 - (embedding <=> ${queryEmbedding}::vector) as similarity
      FROM "KnowledgeIndex"
      WHERE "workspaceId" = ${input.workspaceId}::uuid
        AND embedding IS NOT NULL
      ORDER BY embedding <=> ${queryEmbedding}::vector
      LIMIT 10
    `;

    // Step 3: Also do keyword search for hybrid results
    const keywordResults = await prisma.$queryRaw<Array<{
      id: string;
      sourceType: string;
      sourceId: string;
      title: string;
      content: string;
      metadata: any;
    }>>`
      SELECT id, "sourceType", "sourceId", title, LEFT(content, 1000) as content, metadata
      FROM "KnowledgeIndex"
      WHERE "workspaceId" = ${input.workspaceId}::uuid
        AND to_tsvector('english', content) @@ plainto_tsquery('english', ${input.query})
      ORDER BY ts_rank(to_tsvector('english', content), plainto_tsquery('english',
${input.query})) DESC
      LIMIT 5
    `;
```

```
    // Step 4: Merge and deduplicate results
    const allResults = [...semanticResults];
    for (const kr of keywordResults) {
      if (!allResults.find((r) => r.sourceId === kr.sourceId)) {
        allResults.push({ ...kr, similarity: 0.5 });
      }
    }

    // Step 5: Build context for AI from top results
    const contextForAI = allResults
      .slice(0, 8)
      .map((r, i) => `[Source ${i + 1}: ${r.sourceType} "${r.title}"]\\n${r.content}`)
      .join("\\n\\n---\\n\\n");

    // Step 6: Generate AI answer with citations
    const result = await aiService.generateText(
      `Based on the following workspace data, answer the user's question accurately.
Include citations in your answer like [Source 1], [Source 2] when you reference specific
information.
If the information is not found in the sources, say "I couldn't find information about that in your
workspace."

USER QUESTION: ${input.query}

WORKSPACE DATA:
${contextForAI}`,
      {
        systemPrompt: "You are an AI assistant that answers questions based on workspace
data. Always cite your sources. Be concise and accurate. If you're not sure, say so.",
        model: (input.model as any) || "gpt-4o",
        maxTokens: 1000,
        temperature: 0.3,
      }
    );

    // Step 7: Build citations array
    const citations = allResults.slice(0, 8).map((r) => ({
      sourceId: r.sourceId,
      sourceType: r.sourceType,
      title: r.title,
      snippet: r.content.slice(0, 200),
      metadata: r.metadata,
    }));
```

```
    // Step 8: Save query for analytics
    await prisma.knowledgeQuery.create({
      data: {
        userId: ctx.user.id,
        workspaceId: input.workspaceId,
        query: input.query,
        results: allResults.slice(0, 8).map((r) => ({
          sourceType: r.sourceType,
          sourceId: r.sourceId,
          title: r.title,
          relevanceScore: r.similarity,
        })),
        aiAnswer: result.text,
        citations,
        responseTimeMs: Date.now() - startTime,
      },
    });

    // Log usage
    await logAiUsage({
      userId: ctx.user.id,
      workspaceId: input.workspaceId,
      feature: "search",
      action: "ask_ai",
      model: input.model || "gpt-4o",
      inputTokens: result.tokensUsed.input,
      outputTokens: result.tokensUsed.output,
      durationMs: Date.now() - startTime,
    });

    return {
      answer: result.text,
      citations,
      responseTimeMs: Date.now() - startTime,
    };
  }),

/**
 * Rate a search/AI answer
 */
rateFeedback: protectedProcedure
  .input(z.object({ queryId: z.string().uuid(), rating: z.number().min(1).max(5) }))
  .mutation(async ({ input }) => {
```

```
      await prisma.knowledgeQuery.update({
        where: { id: input.queryId },
        data: { feedbackRating: input.rating },
      });
    }),
});
```

2.2 Universal Search UI

Create file: `src/components/search/universal-search.tsx`

```
// src/components/search/universal-search.tsx
// Universal search modal — Ctrl+K / Cmd+K

"use client";

import { useState, useEffect, useRef, useCallback } from "react";
import { Search, Sparkles, X, FileText, CheckSquare, MessageCircle, Paperclip, Loader2,
ThumbsUp, ThumbsDown } from "lucide-react";
import { AiStreamingDisplay } from "@/components/ai/ai-streaming-display";
import { trpc } from "@/lib/trpc-client";
import { useDebounce } from "@/hooks/use-debounce";

interface UniversalSearchProps {
  workspaceId: string;
  isOpen: boolean;
  onClose: () => void;
}

type Tab = "search" | "ask_ai";

const SOURCE_ICONS: Record<string, React.ReactNode> = {
  page: <FileText className="w-4 h-4 text-blue-500" />,
  task: <CheckSquare className="w-4 h-4 text-green-500" />,
  message: <MessageCircle className="w-4 h-4 text-purple-500" />,
  meeting_transcript: <FileText className="w-4 h-4 text-orange-500" />,
  file: <Paperclip className="w-4 h-4 text-gray-500" />,
};

export function UniversalSearch({ workspaceId, isOpen, onClose }: UniversalSearchProps) {
  const [tab, setTab] = useState<Tab>("search");
  const [query, setQuery] = useState("");
  const debouncedQuery = useDebounce(query, 200);
```

```
  const inputRef = useRef<HTMLInputElement>(null);

  // Keyword search
  const searchQuery = trpc.knowledge.search.useQuery(
    { workspaceId, query: debouncedQuery },
    { enabled: tab === "search" && debouncedQuery.length > 0 }
  );

  // AI Q&A
  const askAiMutation = trpc.knowledge.askAi.useMutation();

  // Focus input on open
  useEffect(() => {
    if (isOpen) {
      setTimeout(() => inputRef.current?.focus(), 100);
    }
  }, [isOpen]);

  // Keyboard shortcut: Cmd+K / Ctrl+K
  useEffect(() => {
    const handler = (e: KeyboardEvent) => {
      if ((e.metaKey || e.ctrlKey) && e.key === "k") {
        e.preventDefault();
        if (isOpen) onClose();
        // Parent component should handle opening
      }
      if (e.key === "Escape" && isOpen) onClose();
    };
    document.addEventListener("keydown", handler);
    return () => document.removeEventListener("keydown", handler);
  }, [isOpen, onClose]);

  const handleAskAi = useCallback(() => {
    if (!query.trim()) return;
    askAiMutation.mutate({ workspaceId, query });
  }, [query, workspaceId, askAiMutation]);

  if (!isOpen) return null;

  return (
    <div className="fixed inset-0 z-50 flex items-start justify-center pt-[15vh] bg-black/50"
onClick={onClose}>
      <div
        className="bg-white rounded-xl shadow-2xl w-full max-w-2xl max-h-[65vh] flex flex-col"
```

```
      onClick={(e) => e.stopPropagation()}
    >
      {/* Tabs */}
      <div className="flex border-b">
        <button
          onClick={() => setTab("search")}
          className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2 ${
            tab === "search" ? "border-blue-600 text-blue-600" : "border-transparent text-gray-500"
          }`}
        >
          <Search className="w-4 h-4" /> Search
        </button>
        <button
          onClick={() => setTab("ask_ai")}
          className={`flex items-center gap-2 px-4 py-3 text-sm font-medium border-b-2 ${
            tab === "ask_ai" ? "border-purple-600 text-purple-600" : "border-transparent
text-gray-500"
          }`}
        >
          <Sparkles className="w-4 h-4" /> Ask AI
        </button>
        <div className="flex-1" />
        <button onClick={onClose} className="px-3 text-gray-400 hover:text-gray-600">
          <X className="w-4 h-4" />
        </button>
      </div>

      {/* Search input */}
      <div className="px-4 py-3 border-b">
        <div className="flex items-center gap-2">
          {tab === "search" ? <Search className="w-4 h-4 text-gray-400" /> : <Sparkles
className="w-4 h-4 text-purple-400" />}
          <input
            ref={inputRef}
            value={query}
            onChange={(e) => setQuery(e.target.value)}
            onKeyDown={(e) => {
              if (e.key === "Enter" && tab === "ask_ai") {
                e.preventDefault();
                handleAskAi();
              }
            }}
            placeholder={tab === "search" ? "Search pages, tasks, messages..." : "Ask a question
about your workspace..."}
```

```
            className="flex-1 text-sm outline-none"
          />
          {tab === "ask_ai" && (
            <button
              onClick={handleAskAi}
              disabled={askAiMutation.isPending || !query.trim()}
              className="text-sm px-3 py-1 bg-purple-600 text-white rounded hover:bg-purple-700
disabled:opacity-50"
            >
              Ask
            </button>
          )}
        </div>
      </div>

      {/* Results */}
      <div className="flex-1 overflow-y-auto p-4">
        {tab === "search" && (
          <>
            {searchQuery.isLoading && <div className="text-sm text-gray-500 text-center
py-8">Searching...</div>}
            {searchQuery.data && searchQuery.data.totalCount === 0 && (
              <div className="text-sm text-gray-500 text-center py-8">No results found for
"{query}"</div>
            )}
            {searchQuery.data && Object.entries(searchQuery.data.grouped).map(([type, items])
=> (
              <div key={type} className="mb-4">
                <h3 className="text-xs font-medium text-gray-500 uppercase mb-2 flex
items-center gap-1">
                  {SOURCE_ICONS[type]} {type}s ({(items as any[]).length})
                </h3>
                {(items as any[]).map((item: any) => (
                  <button
                    key={item.id}
                    className="w-full text-left px-3 py-2 rounded hover:bg-gray-100 block"
                    onClick={() => {
                      // Navigate to source
                      onClose();
                    }}
                  >
                    <div className="text-sm font-medium text-gray-800">{item.title}</div>
                    <div
                      className="text-xs text-gray-500 mt-0.5 line-clamp-2"
```

```
                dangerouslySetInnerHTML={{ __html: item.content }}
              />
            </button>
          ))}
        </div>
      ))}
    </>
  )}

  {tab === "ask_ai" && (
    <>
      {askAiMutation.isPending && (
        <div className="flex items-center gap-2 text-sm text-purple-600 py-8 justify-center">
          <Sparkles className="w-4 h-4" />
          <span>Searching workspace and generating answer...</span>
          <Loader2 className="w-4 h-4 animate-spin" />
        </div>
      )}
      {askAiMutation.data && (
        <div>
          <div className="flex items-center gap-1 text-xs text-purple-600 mb-2">
            <Sparkles className="w-3 h-3" /> AI Answer
          </div>
          <div className="text-sm text-gray-800 prose prose-sm mb-4
whitespace-pre-wrap">
            {askAiMutation.data.answer}
          </div>
          {askAiMutation.data.citations.length > 0 && (
            <div>
              <div className="text-xs font-medium text-gray-500 mb-1">Sources:</div>
              {askAiMutation.data.citations.map((c: any, i: number) => (
                <button
                  key={i}
                  className="flex items-center gap-2 text-xs text-blue-600 hover:underline
py-0.5"
                >
                  {SOURCE_ICONS[c.sourceType]}
                  {c.title}
                </button>
              ))}
            </div>
          )}
          <div className="flex items-center gap-2 mt-3 pt-3 border-t">
            <span className="text-xs text-gray-400">Helpful?</span>
```

```
                <button className="text-gray-400 hover:text-green-600"><ThumbsUp
className="w-3 h-3" /></button>
                <button className="text-gray-400 hover:text-red-600"><ThumbsDown
className="w-3 h-3" /></button>
                <span className="text-xs text-gray-400 ml-2">
                  {askAiMutation.data.responseTimeMs}ms
                </span>
              </div>
            </div>
          )}
          {!askAiMutation.data && !askAiMutation.isPending && (
            <div className="text-sm text-gray-500 text-center py-8">
              Ask any question about your workspace data.
              <br />
              <span className="text-xs text-gray-400">
                "What was decided about pricing?" · "Who is working on the auth feature?" ·
"Summarize last week's meetings"
              </span>
            </div>
          )}
        </>
      )}
    </div>
  </div>
</div>
);
}
```

EDGE CASES FOR SPRINT 2

FEATURE 1 (Writing):
1. Empty selection + "Improve" action → Show toast: "Select some text first"
2. Selected text > 10,000 chars → Truncate to last 10k chars, show warning
3. AI returns empty response → Show "AI couldn't generate content. Try rephrasing."
4. User clicks "Try Again" → Re-run with temperature + 0.1 (max 3 retries)
5. Network failure during streaming → Save partial text, show [Keep Partial] [Retry]
6. Concurrent AI requests → Queue them, show "Processing previous request..."
7. Brand voice not set → Silently skip brand voice in prompt (no error)
8. Unknown action type → Fallback to "custom" action

FEATURE 2 (Knowledge Search):
1. No results found → Show "No results" with suggestion to try different keywords

2. Empty knowledge index (new workspace) → Show "Your workspace is empty. Add pages and tasks to make them searchable."
3. AI Q&A with no relevant results → AI says "I couldn't find information about that in your workspace"
4. Very long AI answer → Cap at 1000 tokens, show full answer in expandable section
5. Search query < 2 chars → Don't trigger search, show recent searches instead
6. Indexing job fails → Mark entry as isStale, retry on next indexing cycle
7. Embedding generation fails → Fall back to keyword-only search for that document
8. Permission filtering: Message from private channel → Only return if user is channel member

SPRINT 2 VERIFICATION CHECKLIST

□ Open editor → press Space on empty line → AI input appears
□ Type "Write an introduction for our product" → AI streams text into editor
□ Click [Keep] → text inserted into editor
□ Select text → click AI button → choose "Improve" → see diff → Accept
□ Type /ai brainstorm → generates ideas
□ Press Cmd+J → prompt bar opens → type question → get response
□ AI writing with "Write as marketer" role → noticeably different tone
□ Brand voice settings saved → AI follows vocabulary rules
□ Press Cmd+K → universal search opens → type keyword → results appear grouped by type
□ Switch to "Ask AI" tab → ask question → get answer with citations
□ Click citation source → navigates to correct page/message/task
□ Create a new page → within 5 minutes it appears in search results
□ AI Q&A with no relevant data → shows "couldn't find" message gracefully

Once all checks pass, proceed to Sprint 3.

SPRINT 3 — DAILY PLANNER + HABITS + FOCUS COACH

Features 3, 14, and 9 — Full Implementation

PREREQUISITE: Sprint 1 + Sprint 2 must be complete

PASTE THIS INTO REPLIT AI AGENT

WHAT TO BUILD IN THIS SPRINT

1. Feature 3: AI Auto-Scheduler & Daily Planner (plan generation, timeline UI, morning/evening rituals, smart rescheduling)
2. Feature 14: AI Habit Tracker & Streak Engine (CRUD, AI scheduling, streaks, analytics)
3. Feature 9: AI Focus & Productivity Coach (focus sessions, focus protection, analytics, coaching)

After this sprint, verify:

- Navigate to "My Day" → see auto-generated daily plan with time blocks
- Drag blocks to reorder → saves changes
- Click "Start Focus" on a task → focus timer starts, notifications pause
- Create a habit "Deep Work 2h daily" → appears in daily plan
- Complete a habit → streak increments
- End of day → evening review shows completion stats

FEATURE 3: AI AUTO-SCHEDULER & DAILY PLANNER

3.1 tRPC Router

Create file: `src/server/routers/daily-plan.ts`

```
// src/server/routers/daily-plan.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logger";
```

```
const timeBlockSchema = z.object({
  id: z.string(),
  type: z.enum(["task", "meeting", "focus", "break", "habit", "communication", "planning", "buffer",
"custom"]),
  title: z.string(),
  description: z.string().optional(),
  startTime: z.string(), // "09:00"
  endTime: z.string(),
  durationMinutes: z.number(),
  taskId: z.string().uuid().optional().nullable(),
  meetingId: z.string().uuid().optional().nullable(),
  habitId: z.string().uuid().optional().nullable(),
  priority: z.enum(["must_do", "should_do", "nice_to_do"]),
  energyLevel: z.enum(["high", "medium", "low"]),
  isFixed: z.boolean(),
  isCompleted: z.boolean().default(false),
  completedAt: z.string().optional().nullable(),
  isSkipped: z.boolean().default(false),
  actualDurationMinutes: z.number().optional().nullable(),
  scheduledByAI: z.boolean().default(false),
  aiReason: z.string().optional().nullable(),
});

export const dailyPlanRouter = router({
  /**
   * Get daily plan for a specific date
   */
  getByDate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      date: z.string(), // "2026-03-04" ISO date string
    }))
    .query(async ({ input, ctx }) => {
      const plan = await prisma.dailyPlan.findUnique({
        where: {
          userId_date: {
            userId: ctx.user.id,
            date: new Date(input.date),
          },
        },
      });

      if (!plan) return null;
      return plan;
```

```
  }),

/**
 * Generate AI daily plan for a specific date
 */
generate: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    date: z.string(),
  }))
  .mutation(async ({ input, ctx }) => {
    const startTime = Date.now();
    const targetDate = new Date(input.date);

    // STEP 1: Gather all inputs for planning

    // Get user's scheduling preferences
    const prefs = await prisma.schedulingPreference.findUnique({
      where: { userId: ctx.user.id },
    });
    const workingHours = (prefs?.workingHours as any) || { start: "09:00", end: "18:00" };
    const lunchTime = (prefs?.lunchTime as any) || { start: "12:30", end: "13:30" };
    const peakFocus = (prefs?.peakFocusWindow as any) || { start: "10:00", end: "12:00" };

    // Get tasks due within 7 days assigned to this user
    const sevenDaysFromNow = new Date(targetDate);
    sevenDaysFromNow.setDate(sevenDaysFromNow.getDate() + 7);

    const tasks = await prisma.task.findMany({
      where: {
        assigneeId: ctx.user.id,
        workspaceId: input.workspaceId,
        completedAt: null,
        deletedAt: null,
        dueDate: { lte: sevenDaysFromNow },
      },
      orderBy: [{ dueDate: "asc" }, { priority: "desc" }],
      take: 20,
    });

    // Get today's calendar events (meetings)
    // Adjust this query to your actual Meeting/CalendarEvent model
    const meetings = await prisma.meeting.findMany({
      where: {
```

```
      participants: { some: { userId: ctx.user.id } },
      startTime: {
        gte: new Date(`${input.date}T00:00:00`),
        lt: new Date(`${input.date}T23:59:59`),
      },
    },
    orderBy: { startTime: "asc" },
  });

  // Get active habits
  const habits = await prisma.habit.findMany({
    where: {
      userId: ctx.user.id,
      isActive: true,
      deletedAt: null,
    },
  });

  // Get yesterday's incomplete tasks (carry-over)
  const yesterday = new Date(targetDate);
  yesterday.setDate(yesterday.getDate() - 1);
  const yesterdayPlan = await prisma.dailyPlan.findUnique({
    where: { userId_date: { userId: ctx.user.id, date: yesterday } },
  });
  const carryOverBlocks = yesterdayPlan
    ? (yesterdayPlan.timeBlocks as any[]).filter((b: any) => !b.isCompleted && !b.isSkipped &&
b.taskId)
    : [];

  // Get duration estimates for tasks
  const taskIds = tasks.map(t => t.id);
  const estimates = await prisma.taskDurationEstimate.findMany({
    where: { taskId: { in: taskIds } },
  });
  const estimateMap = new Map(estimates.map(e => [e.taskId, e.estimatedMinutes]));

  // STEP 2: Build the schedule with AI

  const tasksForAI = tasks.map(t => ({
    id: t.id,
    title: t.title,
    dueDate: t.dueDate?.toISOString().split("T")[0],
    priority: t.priority || "medium",
    estimatedMinutes: estimateMap.get(t.id) || 60,
```

```
    isOverdue: t.dueDate ? t.dueDate < targetDate : false,
    isDueToday: t.dueDate?.toISOString().split("T")[0] === input.date,
  }));

  const meetingsForAI = meetings.map(m => ({
    id: m.id,
    title: m.title,
    startTime: m.startTime.toTimeString().slice(0, 5),
    endTime: m.endTime.toTimeString().slice(0, 5),
    durationMinutes: Math.round((m.endTime.getTime() - m.startTime.getTime()) / 60000),
  }));

  const habitsForAI = habits.map(h => ({
    id: h.id,
    name: h.name,
    durationMinutes: h.durationMinutes,
    preferredTimeRange: h.preferredTimeRange,
    priority: h.priority,
  }));

  const aiResponse = await aiService.generateJSON<{
    timeBlocks: Array<{
      type: string; title: string; startTime: string; endTime: string;
      durationMinutes: number; taskId?: string; meetingId?: string; habitId?: string;
      priority: string; energyLevel: string; isFixed: boolean;
      aiReason: string;
    }>;
    summary: {
      totalPlannedMinutes: number; focusMinutes: number; meetingMinutes: number;
      mustDoCount: number; shouldDoCount: number; niceToDoCount: number;
      topPriority: string; aiReasoning: string; completionLikelihood: number;
    };
  }>(
    `Create an optimized daily schedule for ${input.date}.

WORKING HOURS: ${workingHours.start} to ${workingHours.end}
LUNCH: ${lunchTime.start} to ${lunchTime.end}
PEAK FOCUS WINDOW: ${peakFocus.start} to ${peakFocus.end}
BUFFER BETWEEN MEETINGS: ${prefs?.bufferBetweenMeetings || 15} minutes

FIXED MEETINGS (cannot move):
${JSON.stringify(meetingsForAI, null, 2)}

TASKS TO SCHEDULE:
```

```
${JSON.stringify(tasksForAI, null, 2)}

HABITS TO INCLUDE:
${JSON.stringify(habitsForAI, null, 2)}

CARRY-OVER FROM YESTERDAY: ${carryOverBlocks.length} incomplete tasks

RULES:
1. Place meetings first (they are fixed and cannot move)
2. Place MUST DO tasks (overdue + due today + high priority) in peak focus window
3. Place habits in their preferred time ranges
4. Add 15-min breaks every 90 minutes
5. Add morning planning block (15 min at start of day)
6. Add evening review block (15 min at end of day)
7. Add communication blocks (30 min morning + 30 min after lunch)
8. Add buffer time between meetings
9. Leave at least 30 min unscheduled as buffer
10. Place SHOULD DO tasks in remaining good time slots
11. Place NICE TO DO tasks in lower-energy afternoon slots
12. Never overlap blocks
13. MUST DO = overdue + due today; SHOULD DO = due within 3 days; NICE TO DO = rest

Return JSON with timeBlocks array sorted by startTime and a summary object.
Each block needs: type, title, startTime (HH:MM), endTime (HH:MM), durationMinutes,
taskId/meetingId/habitId if applicable, priority (must_do/should_do/nice_to_do), energyLevel
(high/medium/low), isFixed (true for meetings), aiReason.`,
      {
        systemPrompt: "You are an expert productivity scheduler. Create optimal daily schedules
that maximize deep work time and respect human energy patterns. Return ONLY valid JSON.",
        temperature: 0.3,
      }
    );

    // STEP 3: Save the plan

    const timeBlocks = aiResponse.timeBlocks.map((block, index) => ({
      ...block,
      id: crypto.randomUUID(),
      isCompleted: false,
      isSkipped: false,
      scheduledByAI: true,
    }));

    const plan = await prisma.dailyPlan.upsert({
```

```
      where: { userId_date: { userId: ctx.user.id, date: targetDate } },
      create: {
        userId: ctx.user.id,
        workspaceId: input.workspaceId,
        date: targetDate,
        status: prefs?.planMode === "automatic" ? "active" : "generated",
        timeBlocks: timeBlocks as any,
        aiSummary: aiResponse.summary as any,
      },
      update: {
        timeBlocks: timeBlocks as any,
        aiSummary: aiResponse.summary as any,
        status: "generated",
      },
    });

    await logAiUsage({
      userId: ctx.user.id,
      workspaceId: input.workspaceId,
      feature: "planner",
      action: "generate_plan",
      model: "gpt-4o",
      inputTokens: 0, outputTokens: 0,
      durationMs: Date.now() - startTime,
    });

    return plan;
  }),

/**
 * Update time blocks (user dragged, completed, skipped, etc.)
 */
updateBlocks: protectedProcedure
  .input(z.object({
    planId: z.string().uuid(),
    timeBlocks: z.array(timeBlockSchema),
  }))
  .mutation(async ({ input }) => {
    return prisma.dailyPlan.update({
      where: { id: input.planId },
      data: {
        timeBlocks: input.timeBlocks as any,
        userModifications: { increment: 1 },
      },
```

```
    });
  }),

/**
 * Complete a time block
 */
completeBlock: protectedProcedure
  .input(z.object({
    planId: z.string().uuid(),
    blockId: z.string(),
    actualDurationMinutes: z.number().optional(),
  }))
  .mutation(async ({ input }) => {
    const plan = await prisma.dailyPlan.findUnique({ where: { id: input.planId } });
    if (!plan) throw new Error("Plan not found");

    const blocks = plan.timeBlocks as any[];
    const blockIndex = blocks.findIndex((b: any) => b.id === input.blockId);
    if (blockIndex === -1) throw new Error("Block not found");

    blocks[blockIndex].isCompleted = true;
    blocks[blockIndex].completedAt = new Date().toISOString();
    blocks[blockIndex].actualDurationMinutes = input.actualDurationMinutes;

    // Calculate completion rate
    const totalBlocks = blocks.filter((b: any) => b.type !== "break" && b.type !== "buffer");
    const completedBlocks = totalBlocks.filter((b: any) => b.isCompleted);
    const completionRate = completedBlocks.length / totalBlocks.length;

    return prisma.dailyPlan.update({
      where: { id: input.planId },
      data: { timeBlocks: blocks, completionRate },
    });
  }),

/**
 * Skip a time block
 */
skipBlock: protectedProcedure
  .input(z.object({ planId: z.string().uuid(), blockId: z.string() }))
  .mutation(async ({ input }) => {
    const plan = await prisma.dailyPlan.findUnique({ where: { id: input.planId } });
    if (!plan) throw new Error("Plan not found");
```

```
    const blocks = plan.timeBlocks as any[];
    const blockIndex = blocks.findIndex((b: any) => b.id === input.blockId);
    if (blockIndex === -1) throw new Error("Block not found");

    blocks[blockIndex].isSkipped = true;

    return prisma.dailyPlan.update({
      where: { id: input.planId },
      data: { timeBlocks: blocks },
    });
  }),

/**
 * Save evening review note
 */
saveReview: protectedProcedure
  .input(z.object({
    planId: z.string().uuid(),
    reviewNote: z.string().optional(),
  }))
  .mutation(async ({ input }) => {
    return prisma.dailyPlan.update({
      where: { id: input.planId },
      data: { reviewNote: input.reviewNote, status: "completed" },
    });
  }),

/**
 * Get or create scheduling preferences
 */
getPreferences: protectedProcedure
  .input(z.object({ workspaceId: z.string().uuid() }))
  .query(async ({ input, ctx }) => {
    return prisma.schedulingPreference.upsert({
      where: { userId: ctx.user.id },
      create: { userId: ctx.user.id, workspaceId: input.workspaceId },
      update: {},
    });
  }),

/**
 * Update scheduling preferences
 */
updatePreferences: protectedProcedure
```

```
    .input(z.object({
      workingHours: z.any().optional(),
      timezone: z.string().optional(),
      workDays: z.array(z.number()).optional(),
      lunchTime: z.any().optional(),
      peakFocusWindow: z.any().optional(),
      preferMeetingsIn: z.string().optional(),
      minFocusBlockMinutes: z.number().optional(),
      bufferBetweenMeetings: z.number().optional(),
      autoScheduleEnabled: z.boolean().optional(),
      morningPlanningEnabled: z.boolean().optional(),
      morningPlanningTime: z.string().optional(),
      eveningReviewEnabled: z.boolean().optional(),
      eveningReviewTime: z.string().optional(),
      planMode: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.schedulingPreference.update({
        where: { userId: ctx.user.id },
        data: input,
      });
    }),
});
```

3.2 Daily Plan Timeline UI

Create file: `src/components/daily-plan/plan-timeline.tsx`

```
// src/components/daily-plan/plan-timeline.tsx
// Main daily planner view with draggable time blocks

"use client";

import { useState, useCallback } from "react";
import { ChevronLeft, ChevronRight, RefreshCw, Settings, Play, Check, SkipForward, Clock }
from "lucide-react";
import { PlanBlock } from "./plan-block";
import { trpc } from "@/lib/trpc-client";
import { format, addDays, subDays, isToday } from "date-fns";

interface PlanTimelineProps {
  workspaceId: string;
}
```

```
export function PlanTimeline({ workspaceId }: PlanTimelineProps) {
  const [selectedDate, setSelectedDate] = useState(new Date());
  const dateStr = format(selectedDate, "yyyy-MM-dd");

  const planQuery = trpc.dailyPlan.getByDate.useQuery({ workspaceId, date: dateStr });
  const generateMutation = trpc.dailyPlan.generate.useMutation({
    onSuccess: () => planQuery.refetch(),
  });
  const completeBlockMutation = trpc.dailyPlan.completeBlock.useMutation({
    onSuccess: () => planQuery.refetch(),
  });
  const skipBlockMutation = trpc.dailyPlan.skipBlock.useMutation({
    onSuccess: () => planQuery.refetch(),
  });

  const plan = planQuery.data;
  const timeBlocks = (plan?.timeBlocks as any[]) || [];
  const aiSummary = plan?.aiSummary as any;

  const handleGenerate = () => {
    generateMutation.mutate({ workspaceId, date: dateStr });
  };

  const handleComplete = (blockId: string) => {
    if (!plan) return;
    completeBlockMutation.mutate({ planId: plan.id, blockId });
  };

  const handleSkip = (blockId: string) => {
    if (!plan) return;
    skipBlockMutation.mutate({ planId: plan.id, blockId });
  };

  return (
    <div className="max-w-2xl mx-auto p-4">
      {/* Header */}
      <div className="flex items-center justify-between mb-4">
        <div className="flex items-center gap-2">
          <h1 className="text-xl font-semibold">
            📅 My Day — {format(selectedDate, "EEEE, MMMM d")}
          </h1>
        </div>
        <div className="flex items-center gap-2">
```

```
        <button onClick={() => setSelectedDate(subDays(selectedDate, 1))} className="p-1
hover:bg-gray-100 rounded">
          <ChevronLeft className="w-4 h-4" />
        </button>
        <button
          onClick={() => setSelectedDate(new Date())}
          className={`px-2 py-1 text-xs rounded ${isToday(selectedDate) ? "bg-blue-100
text-blue-700" : "bg-gray-100"}`}
        >
          Today
        </button>
        <button onClick={() => setSelectedDate(addDays(selectedDate, 1))} className="p-1
hover:bg-gray-100 rounded">
          <ChevronRight className="w-4 h-4" />
        </button>
        <button
          onClick={handleGenerate}
          disabled={generateMutation.isPending}
          className="flex items-center gap-1 px-3 py-1 text-sm bg-purple-600 text-white rounded
hover:bg-purple-700 disabled:opacity-50"
        >
          <RefreshCw className={`w-3 h-3 ${generateMutation.isPending ? "animate-spin" : ""}`}
/>
          {plan ? "Regenerate" : "Generate Plan"}
        </button>
      </div>
    </div>

    {/* AI Briefing */}
    {aiSummary && (
      <div className="bg-purple-50 border border-purple-200 rounded-lg p-3 mb-4">
        <div className="flex items-center gap-1 text-xs text-purple-600 font-medium mb-1">
          🤖 AI Briefing
        </div>
        <p className="text-sm text-gray-800">{aiSummary.aiReasoning}</p>
        <p className="text-xs text-gray-500 mt-1">
          Estimated completion: {aiSummary.completionLikelihood}% likely
        </p>
      </div>
    )}

    {/* Time Blocks */}
    {planQuery.isLoading && (
      <div className="text-center py-12 text-gray-500">Loading your plan...</div>
```

```
    )}

    {!plan && !planQuery.isLoading && (
      <div className="text-center py-12">
        <p className="text-gray-500 mb-4">No plan for this day yet.</p>
        <button
          onClick={handleGenerate}
          disabled={generateMutation.isPending}
          className="px-4 py-2 bg-purple-600 text-white rounded-lg hover:bg-purple-700"
        >
          🤖 Generate AI Plan
        </button>
      </div>
    )}

    {timeBlocks.length > 0 && (
      <div className="space-y-2">
        {timeBlocks.map((block: any) => (
          <PlanBlock
            key={block.id}
            block={block}
            onComplete={() => handleComplete(block.id)}
            onSkip={() => handleSkip(block.id)}
            onStartFocus={() => {
              // Navigate to focus session (Feature 9)
              // Will be connected when Feature 9 is built
            }}
          />
        ))}
      </div>
    )}

    {/* Summary Bar */}
    {aiSummary && (
      <div className="mt-4 bg-gray-50 border rounded-lg p-3 flex items-center justify-between
text-sm">
        <div className="flex items-center gap-4">
          <span>
            Must: {timeBlocks.filter((b: any) => b.priority === "must_do" && b.isCompleted).length}/
            {timeBlocks.filter((b: any) => b.priority === "must_do").length} ✅
          </span>
          <span>
            Should: {timeBlocks.filter((b: any) => b.priority === "should_do" &&
b.isCompleted).length}/
```

```
            {timeBlocks.filter((b: any) => b.priority === "should_do").length}
          </span>
          <span>
            Nice: {timeBlocks.filter((b: any) => b.priority === "nice_to_do" &&
b.isCompleted).length}/
            {timeBlocks.filter((b: any) => b.priority === "nice_to_do").length}
          </span>
        </div>
        <span className="text-gray-500">
          Focus: {aiSummary.focusMinutes}min | Meetings: {aiSummary.meetingMinutes}min
        </span>
      </div>
    )}
  </div>
 );
}
```

Create file: `src/components/daily-plan/plan-block.tsx`

```
// src/components/daily-plan/plan-block.tsx

"use client";

import { Check, SkipForward, Play, Clock, GripVertical } from "lucide-react";

interface PlanBlockProps {
  block: {
    id: string; type: string; title: string; startTime: string; endTime: string;
    durationMinutes: number; priority: string; energyLevel: string;
    isFixed: boolean; isCompleted: boolean; isSkipped: boolean; aiReason?: string;
  };
  onComplete: () => void;
  onSkip: () => void;
  onStartFocus: () => void;
}

const TYPE_STYLES: Record<string, { bg: string; icon: string }> = {
  task: { bg: "border-l-4 border-l-blue-500", icon: "📋" },
  meeting: { bg: "border-l-4 border-l-purple-500 bg-purple-50", icon: "🟪" },
  focus: { bg: "border-l-4 border-l-red-500", icon: "🎯" },
  break: { bg: "border-l-4 border-l-green-500 bg-green-50", icon: "☕" },
  habit: { bg: "border-l-4 border-l-orange-500", icon: "🔥" },
  communication: { bg: "border-l-4 border-l-yellow-500", icon: "📧" },
```

```
  planning: { bg: "border-l-4 border-l-indigo-500 bg-indigo-50", icon: "🌅" },
  buffer: { bg: "border-l-4 border-l-gray-300 bg-gray-50", icon: "⏳" },
  custom: { bg: "border-l-4 border-l-teal-500", icon: "✏️" },
};

const PRIORITY_BADGES: Record<string, string> = {
  must_do: "bg-red-100 text-red-700",
  should_do: "bg-yellow-100 text-yellow-700",
  nice_to_do: "bg-blue-100 text-blue-700",
};

export function PlanBlock({ block, onComplete, onSkip, onStartFocus }: PlanBlockProps) {
  const style = TYPE_STYLES[block.type] || TYPE_STYLES.custom;

  return (
    <div
      className={`rounded-lg p-3 ${style.bg} ${
        block.isCompleted ? "opacity-50 line-through" : ""
      } ${block.isSkipped ? "opacity-30" : ""} bg-white shadow-sm hover:shadow-md
transition-shadow`}
    >
      <div className="flex items-start gap-2">
        <GripVertical className="w-4 h-4 text-gray-300 cursor-grab mt-0.5 flex-shrink-0" />
        <div className="flex-1 min-w-0">
          <div className="flex items-center gap-2">
            <span className="text-xs text-gray-500">{block.startTime} – {block.endTime}</span>
            <span className="text-xs">{style.icon}</span>
            <span className="font-medium text-sm truncate">{block.title}</span>
            {block.isFixed && <span className="text-xs text-gray-400">📍 Fixed</span>}
            {block.priority !== "nice_to_do" && (
              <span className={`text-xs px-1.5 py-0.5 rounded
${PRIORITY_BADGES[block.priority] || ""}`}>
                {block.priority === "must_do" ? "⚡MUST DO" : "SHOULD DO"}
              </span>
            )}
          </div>
          {block.aiReason && (
            <p className="text-xs text-gray-400 mt-0.5">{block.aiReason}</p>
          )}
          <div className="text-xs text-gray-500 mt-0.5">
            {block.durationMinutes} min • Energy: {block.energyLevel === "high" ? "🔴" :
block.energyLevel === "medium" ? "🟡" : "🟢"} {block.energyLevel}
          </div>
        </div>
```

```
        {!block.isCompleted && !block.isSkipped && !block.isFixed && (
          <div className="flex items-center gap-1 flex-shrink-0">
            <button onClick={onComplete} className="p-1 text-green-600 hover:bg-green-50
rounded" title="Complete">
              <Check className="w-4 h-4" />
            </button>
            {block.type === "task" && (
              <button onClick={onStartFocus} className="p-1 text-blue-600 hover:bg-blue-50
rounded" title="Start Focus">
                <Play className="w-4 h-4" />
              </button>
            )}
            <button onClick={onSkip} className="p-1 text-gray-400 hover:bg-gray-100 rounded"
title="Skip">
              <SkipForward className="w-4 h-4" />
            </button>
          </div>
        )}
      </div>
    </div>
  );
}
```

FEATURE 14: AI HABIT TRACKER

14.1 tRPC Router

Create file: `src/server/routers/habits.ts`

```
// src/server/routers/habits.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const habitsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.habit.findMany({
        where: { userId: ctx.user.id, isActive: true, deletedAt: null },
```

```
      include: { completions: { where: { date: { gte: new Date(new Date().setDate(new
Date().getDate() - 30)) } }, orderBy: { date: "desc" } } },
      orderBy: { createdAt: "asc" },
    });
  }),

  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1).max(100),
      icon: z.string().optional(),
      color: z.string().optional(),
      durationMinutes: z.number().min(5).max(480),
      frequency: z.enum(["daily", "weekdays", "custom"]).default("daily"),
      customDays: z.array(z.number().min(0).max(6)).optional(),
      preferredTimeRange: z.object({ earliest: z.string(), latest: z.string() }).optional(),
      priority: z.enum(["high", "medium", "low"]).default("medium"),
      isFlexible: z.boolean().default(true),
      isProtected: z.boolean().default(false),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.habit.create({
        data: { ...input, userId: ctx.user.id },
      });
    }),

  update: protectedProcedure
    .input(z.object({
      id: z.string().uuid(),
      name: z.string().optional(),
      icon: z.string().optional(),
      color: z.string().optional(),
      durationMinutes: z.number().optional(),
      frequency: z.enum(["daily", "weekdays", "custom"]).optional(),
      customDays: z.array(z.number()).optional(),
      preferredTimeRange: z.any().optional(),
      priority: z.enum(["high", "medium", "low"]).optional(),
      isFlexible: z.boolean().optional(),
      isProtected: z.boolean().optional(),
      isActive: z.boolean().optional(),
    }))
    .mutation(async ({ input }) => {
      const { id, ...data } = input;
      return prisma.habit.update({ where: { id }, data });
```

```
  }),

complete: protectedProcedure
  .input(z.object({ habitId: z.string().uuid(), date: z.string() }))
  .mutation(async ({ input, ctx }) => {
    const dateObj = new Date(input.date);

    // Create completion
    await prisma.habitCompletion.upsert({
      where: { habitId_date: { habitId: input.habitId, date: dateObj } },
      create: { habitId: input.habitId, userId: ctx.user.id, date: dateObj },
      update: {},
    });

    // Update streak
    const habit = await prisma.habit.findUnique({
      where: { id: input.habitId },
      include: { completions: { orderBy: { date: "desc" }, take: 60 } },
    });

    if (habit) {
      // Calculate current streak
      let streak = 0;
      const today = new Date();
      for (let i = 0; i < 365; i++) {
        const checkDate = new Date(today);
        checkDate.setDate(checkDate.getDate() - i);
        const dateStr = checkDate.toISOString().split("T")[0];
        const completed = habit.completions.some(
          (c) => c.date.toISOString().split("T")[0] === dateStr
        );
        if (completed) {
          streak++;
        } else if (i > 0) {
          break; // streak broken
        }
      }

      await prisma.habit.update({
        where: { id: input.habitId },
        data: {
          streakCurrent: streak,
          streakLongest: Math.max(streak, habit.streakLongest),
        },
```

```
      });
    }

    return { success: true };
  }),

  uncomplete: protectedProcedure
    .input(z.object({ habitId: z.string().uuid(), date: z.string() }))
    .mutation(async ({ input }) => {
      await prisma.habitCompletion.delete({
        where: { habitId_date: { habitId: input.habitId, date: new Date(input.date) } },
      }).catch(() => {}); // ignore if not found
      return { success: true };
    }),

  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.habit.update({
        where: { id: input.id },
        data: { deletedAt: new Date() },
      });
    }),
});
```

FEATURE 9: AI FOCUS & PRODUCTIVITY COACH

9.1 tRPC Router

Create file: `src/server/routers/focus.ts`

```
// src/server/routers/focus.ts

import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const focusRouter = router({
  /**
   * Start a focus session
```

```
  */
  startSession: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      taskId: z.string().uuid().optional(),
      title: z.string(),
      plannedMinutes: z.number().min(5).max(480),
    }))
    .mutation(async ({ input, ctx }) => {
      // Check for existing active session
      const existing = await prisma.focusSession.findFirst({
        where: { userId: ctx.user.id, status: "active" },
      });
      if (existing) throw new Error("You already have an active focus session. End it first.");

      return prisma.focusSession.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          taskId: input.taskId,
          title: input.title,
          plannedMinutes: input.plannedMinutes,
          startedAt: new Date(),
          status: "active",
          notificationsPaused: true,
        },
      });
    }),

  /**
   * End a focus session
   */
  endSession: protectedProcedure
    .input(z.object({ sessionId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const session = await prisma.focusSession.findUnique({ where: { id: input.sessionId } });
      if (!session) throw new Error("Session not found");

      const endedAt = new Date();
      const actualMinutes = Math.round((endedAt.getTime() - session.startedAt.getTime()) /
60000) - session.pausedMinutes;

      // Update session
      const updated = await prisma.focusSession.update({
```

```
      where: { id: input.sessionId },
      data: {
        endedAt,
        actualMinutes: Math.max(actualMinutes, 0),
        status: "completed",
        notificationsPaused: false,
      },
    });

    // Create time entry (Feature 21)
    if (session.taskId) {
      await prisma.timeEntry.create({
        data: {
          userId: ctx.user.id,
          workspaceId: session.workspaceId,
          taskId: session.taskId,
          focusSessionId: session.id,
          durationMinutes: Math.max(actualMinutes, 0),
          startedAt: session.startedAt,
          endedAt,
          source: "focus_session",
        },
      });
    }

    // Update daily productivity score
    const today = new Date().toISOString().split("T")[0];
    await prisma.productivityScore.upsert({
      where: { userId_date: { userId: ctx.user.id, date: new Date(today) } },
      create: {
        userId: ctx.user.id,
        workspaceId: session.workspaceId,
        date: new Date(today),
        focusMinutes: Math.max(actualMinutes, 0),
      },
      update: {
        focusMinutes: { increment: Math.max(actualMinutes, 0) },
      },
    });

    return updated;
  }),

/**
```

```
 * Pause/resume a focus session
 */
togglePause: protectedProcedure
  .input(z.object({ sessionId: z.string().uuid() }))
  .mutation(async ({ input }) => {
    const session = await prisma.focusSession.findUnique({ where: { id: input.sessionId } });
    if (!session) throw new Error("Session not found");

    const newStatus = session.status === "active" ? "paused" : "active";
    return prisma.focusSession.update({
      where: { id: input.sessionId },
      data: { status: newStatus },
    });
  }),

/**
 * Get active focus session (if any)
 */
getActiveSession: protectedProcedure
  .query(async ({ ctx }) => {
    return prisma.focusSession.findFirst({
      where: { userId: ctx.user.id, status: { in: ["active", "paused"] } },
    });
  }),

/**
 * Get productivity scores for a date range (analytics)
 */
getProductivityScores: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    startDate: z.string(),
    endDate: z.string(),
  }))
  .query(async ({ input, ctx }) => {
    return prisma.productivityScore.findMany({
      where: {
        userId: ctx.user.id,
        date: {
          gte: new Date(input.startDate),
          lte: new Date(input.endDate),
        },
      },
      orderBy: { date: "asc" },
```

```
    });
  }),

  /**
   * Get AI coaching insight
   */
  getCoachingInsight: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const thirtyDaysAgo = new Date();
      thirtyDaysAgo.setDate(thirtyDaysAgo.getDate() - 30);

      const scores = await prisma.productivityScore.findMany({
        where: { userId: ctx.user.id, date: { gte: thirtyDaysAgo } },
        orderBy: { date: "asc" },
      });

      const sessions = await prisma.focusSession.findMany({
        where: { userId: ctx.user.id, status: "completed", startedAt: { gte: thirtyDaysAgo } },
      });

      const result = await aiService.generateText(
        `Analyze this user's productivity data and provide 2-3 actionable coaching tips.

DAILY SCORES (last 30 days): ${JSON.stringify(scores.map(s => ({
          date: s.date, focusMin: s.focusMinutes, meetingMin: s.meetingMinutes,
          tasksCompleted: s.tasksCompleted, habitsCompleted: s.habitsCompleted,
        })))}

FOCUS SESSIONS: ${sessions.length} total, avg ${sessions.length > 0 ?
Math.round(sessions.reduce((sum, s) => sum + (s.actualMinutes || 0), 0) / sessions.length) : 0}
min each

Provide specific, actionable insights. Be encouraging but honest.`,
        {
          systemPrompt: "You are a productivity coach. Analyze the data and give specific,
personalized tips. Be warm and encouraging. Keep it to 2-3 short tips.",
          maxTokens: 500,
          temperature: 0.6,
        }
      );

      return { insight: result.text };
    }),
```

```
});
```

EDGE CASES FOR SPRINT 3

FEATURE 3 (Daily Planner):
1. No tasks and no meetings → Generate minimal plan with habits + communication blocks + breaks
2. Too many tasks for one day → AI selects highest priority, puts rest in "overflow" note
3. All-day meeting → Schedule tasks before/after meeting only
4. Calendar sync fails → Show warning, let user manually add meetings
5. User drags a fixed meeting → Show "This meeting can't be moved. It's from your calendar."
6. Regenerate plan with completed blocks → Preserve completed blocks, only reschedule remaining
7. Two meetings overlap → Show conflict warning with both events

FEATURE 14 (Habits):
1. Habit scheduled but day is full → AI moves to earliest available slot, or suggests skipping
2. Streak broken → Show "Streak reset to 0. Previous best: 12 days. Start again!"
3. Complete future date → Block with "Can only complete today or past dates"
4. Custom days has zero days → Validation error "Select at least one day"

FEATURE 9 (Focus):
1. Start focus with existing active session → Error: "End your current session first"
2. Browser/tab closed during focus → On next visit, show "You have an unfinished focus session" with [Resume] [End]
3. Focus session > 4 hours → Auto-pause with "You've been focusing for 4 hours. Take a break!"
4. No productivity data yet → Show "Track a few focus sessions to see your insights"

SPRINT 3 VERIFICATION CHECKLIST

□ Navigate to /my-day → page loads
□ Click "Generate Plan" → AI creates daily plan with time blocks
□ Blocks show in chronological order with correct styling per type
□ Click ✅ on a block → block marked complete, opacity reduced
□ Click ⏭ on a block → block marked skipped
□ Click 🔄 Regenerate → new plan generated, completed blocks preserved
□ Navigate to /habits → create a new habit "Deep Work 2h daily"
□ Complete the habit → streak shows "🔥 1 day"
□ Habit appears in daily plan timeline
□ Navigate to any task → click "Start Focus" → timer starts

□ Timer shows countdown, notifications indicator shows "paused"
□ End focus session → summary shown, time logged
□ View productivity analytics → charts show focus time trends
□ Click "Get Coaching" → AI provides personalized tips
□ Scheduling preferences saved and affect plan generation

Once all checks pass, proceed to Sprint 4.

SPRINT 4 — AI AGENTS + WORKFLOW RECORDER

Features 4 and 19

PREREQUISITE: Sprints 1-3 complete

PASTE THIS INTO REPLIT AI AGENT

FEATURE 4: AI AUTONOMOUS AGENTS

tRPC Router: `src/server/routers/agents.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { agentQueue } from "@/jobs/queue-config";

export const agentsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
```

```
  .query(async ({ input, ctx }) => {
    return prisma.aiAgent.findMany({
      where: {
        workspaceId: input.workspaceId,
        deletedAt: null,
        OR: [
          { visibility: "workspace" },
          { createdBy: ctx.user.id },
          { visibleToUserIds: { has: ctx.user.id } },
        ],
      },
      orderBy: { createdAt: "desc" },
    });
  }),

getById: protectedProcedure
  .input(z.object({ id: z.string().uuid() }))
  .query(async ({ input }) => {
    return prisma.aiAgent.findUnique({
      where: { id: input.id },
      include: { runs: { take: 10, orderBy: { createdAt: "desc" } } },
    });
  }),

create: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    name: z.string().min(1).max(100),
    description: z.string().optional(),
    icon: z.string().optional(),
    systemPrompt: z.string().min(10),
    model: z.string().default("gpt-4o"),
    temperature: z.number().min(0).max(2).default(0.7),
    capabilities: z.array(z.string()),
    dataAccess: z.object({
      allPages: z.boolean().default(false),
      specificPageIds: z.array(z.string()).default([]),
      allDatabases: z.boolean().default(false),
      specificDatabaseIds: z.array(z.string()).default([]),
      allChannels: z.boolean().default(false),
      specificChannelIds: z.array(z.string()).default([]),
      allMeetings: z.boolean().default(false),
    }),
```

```
      triggerType: z.enum(["manual", "scheduled", "event", "message_in_channel",
"webhook"]).default("manual"),
      triggerConfig: z.any().optional(),
      visibility: z.enum(["private", "workspace", "specific_members"]).default("workspace"),
      visibleToUserIds: z.array(z.string().uuid()).optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.aiAgent.create({
        data: { ...input, createdBy: ctx.user.id, type: "custom" },
      });
    }),

  update: protectedProcedure
    .input(z.object({
      id: z.string().uuid(),
      name: z.string().optional(),
      description: z.string().optional(),
      icon: z.string().optional(),
      systemPrompt: z.string().optional(),
      model: z.string().optional(),
      temperature: z.number().optional(),
      capabilities: z.array(z.string()).optional(),
      dataAccess: z.any().optional(),
      triggerType: z.string().optional(),
      triggerConfig: z.any().optional(),
      visibility: z.string().optional(),
      visibleToUserIds: z.array(z.string()).optional(),
      isActive: z.boolean().optional(),
    }))
    .mutation(async ({ input }) => {
      const { id, ...data } = input;
      return prisma.aiAgent.update({ where: { id }, data });
    }),

  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.aiAgent.update({ where: { id: input.id }, data: { deletedAt: new Date() } });
    }),

  // Chat with an agent
  chat: protectedProcedure
    .input(z.object({
      agentId: z.string().uuid(),
```

```
      message: z.string().min(1).max(5000),
      conversationId: z.string().uuid().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      const agent = await prisma.aiAgent.findUnique({ where: { id: input.agentId } });
      if (!agent) throw new Error("Agent not found");

      // Get or create conversation
      let conversation;
      if (input.conversationId) {
        conversation = await prisma.agentConversation.findUnique({ where: { id:
input.conversationId } });
      }
      if (!conversation) {
        conversation = await prisma.agentConversation.create({
          data: {
            agentId: input.agentId,
            userId: ctx.user.id,
            messages: [{ role: "user", content: input.message, timestamp: new Date().toISOString()
}],
          },
        });
      } else {
        const messages = conversation.messages as any[];
        messages.push({ role: "user", content: input.message, timestamp: new
Date().toISOString() });
        await prisma.agentConversation.update({
          where: { id: conversation.id },
          data: { messages },
        });
      }

      // Execute agent (enqueue for background processing)
      const run = await prisma.agentRun.create({
        data: {
          agentId: agent.id,
          workspaceId: agent.workspaceId,
          triggeredBy: "manual",
          triggerData: { message: input.message, conversationId: conversation.id },
          invokedByUserId: ctx.user.id,
          status: "queued",
          steps: [],
        },
      });
```

```
// For manual chat, execute inline (not background) for instant response
// Build messages for AI
const conversationMessages = (conversation.messages as any[]).map((m: any) => ({
  role: m.role === "user" ? "user" as const : "assistant" as const,
  content: m.content,
}));

// Build tools based on agent capabilities
const tools = buildAgentTools(agent.capabilities);

const aiMessages = [
  { role: "system" as const, content: agent.systemPrompt },
  ...conversationMessages,
];

const response = await aiService.generateWithTools(aiMessages, tools, {
  model: agent.model as any,
  temperature: agent.temperature,
});

// Process tool calls if any
let finalResponse = response.content || "";
const steps: any[] = [];

if (response.tool_calls && response.tool_calls.length > 0) {
  for (const toolCall of response.tool_calls) {
    const stepResult = await executeAgentTool(
      toolCall.function.name,
      JSON.parse(toolCall.function.arguments),
      agent,
      ctx.user.id,
    );
    steps.push({
      stepNumber: steps.length + 1,
      action: toolCall.function.name,
      input: JSON.parse(toolCall.function.arguments),
      output: stepResult,
      status: "success",
    });
  }

  // Get final response after tool execution
  const followUp = await aiService.generateText(
```

```
        `Based on these tool results, provide your final response:\\n${JSON.stringify(steps.map(s
=> ({ action: s.action, result: s.output })))}`,
        { systemPrompt: agent.systemPrompt, model: agent.model as any },
      );
      finalResponse = followUp.text;
    }

    // Save agent response to conversation
    const updatedMessages = [...(conversation.messages as any[])];
    updatedMessages.push({ role: "agent", content: finalResponse, timestamp: new
Date().toISOString(), runId: run.id });
    await prisma.agentConversation.update({
      where: { id: conversation.id },
      data: { messages: updatedMessages },
    });

    // Update run
    await prisma.agentRun.update({
      where: { id: run.id },
      data: { status: "completed", finalOutput: finalResponse, steps, completedAt: new Date() },
    });

    return {
      conversationId: conversation.id,
      response: finalResponse,
      runId: run.id,
      steps,
    };
  }),

  // Get conversation history
  getConversation: protectedProcedure
    .input(z.object({ agentId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.agentConversation.findFirst({
        where: { agentId: input.agentId, userId: ctx.user.id, status: "active" },
        orderBy: { updatedAt: "desc" },
      });
    }),

  // Get run history
  getRuns: protectedProcedure
    .input(z.object({ agentId: z.string().uuid(), limit: z.number().default(20) }))
    .query(async ({ input }) => {
```

```
    return prisma.agentRun.findMany({
      where: { agentId: input.agentId },
      orderBy: { createdAt: "desc" },
      take: input.limit,
    });
  }),

  // Generate agent instructions from description
  generateInstructions: protectedProcedure
    .input(z.object({ description: z.string() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
        `Generate detailed instructions for an AI agent with this description: "${input.description}"

Write clear, specific instructions telling the agent:
1. What its role and purpose is
2. What data it should read and how to process it
3. What actions it should take
4. How it should format its responses
5. Any rules or boundaries

Write in second person ("You are...", "You should...").`,
        {
          systemPrompt: "You are an expert at writing AI agent instructions. Be specific and clear.",
          maxTokens: 1000,
          temperature: 0.5,
        }
      );
      return { instructions: result.text };
    }),

  // Create from template
  createFromTemplate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      templateId: z.enum(["sprint_status", "qa_knowledge", "meeting_prep", "onboarding",
"task_triage", "weekly_digest"]),
    }))
    .mutation(async ({ input, ctx }) => {
      const TEMPLATES: Record<string, any> = {
        sprint_status: {
          name: "Sprint Status Bot",
          description: "Generates daily sprint status reports and posts them in a channel",
```

```
      systemPrompt: "You are a Sprint Status Bot. Read all task changes from the last 24 hours. Group by team member: completed, in-progress, blocked. Generate a clear, formatted status report. Include sprint progress percentage and any risks.",
      capabilities: ["read_tasks", "send_messages", "search_workspace"],
      triggerType: "scheduled",
      triggerConfig: { cron: "0 9 * * 1-5" },
    },
    qa_knowledge: {
      name: "Q&A Knowledge Bot",
      description: "Answers questions using workspace knowledge base",
      systemPrompt: "You are a Knowledge Bot. When someone asks a question, search the workspace for relevant information and provide a clear answer with citations to the source documents.",
      capabilities: ["search_workspace", "read_pages", "read_messages"],
      triggerType: "message_in_channel",
      triggerConfig: { keywords: ["?", "how", "what", "where", "when", "who", "why"] },
    },
    meeting_prep: {
      name: "Meeting Prep Agent",
      description: "Gathers context before meetings",
      systemPrompt: "You are a Meeting Prep Agent. Before each meeting, gather: recent task updates related to meeting topic, open action items from previous meetings, relevant channel discussions. Create a concise prep brief.",
      capabilities: ["read_tasks", "read_messages", "read_meetings", "read_calendar", "send_messages"],
      triggerType: "scheduled",
      triggerConfig: { minutesBefore: 30 },
    },
    onboarding: {
      name: "Onboarding Agent",
      description: "Welcomes and guides new team members",
      systemPrompt: "You are an Onboarding Agent. When a new member joins, send a welcome DM with: workspace overview, key channels to join, important documents to read, team introductions, and first tasks.",
      capabilities: ["send_messages", "read_pages", "search_workspace", "send_notifications"],
      triggerType: "event",
      triggerConfig: { eventType: "member_joined" },
    },
    task_triage: {
      name: "Task Triage Agent",
      description: "Auto-prioritizes and categorizes new tasks",
      systemPrompt: "You are a Task Triage Agent. When a new task is created, read its title and description. Assign: priority (Low/Medium/High/Critical), category
```

```
(Bug/Feature/Enhancement/Documentation), and suggest the best assignee based on who
usually handles similar tasks.",
        capabilities: ["read_tasks", "write_tasks", "search_workspace"],
        triggerType: "event",
        triggerConfig: { eventType: "task_created" },
      },
      weekly_digest: {
        name: "Weekly Digest Agent",
        description: "Friday afternoon summary of the week",
        systemPrompt: "You are a Weekly Digest Agent. Every Friday at 4 PM, compile: tasks
completed this week, key decisions made, meetings held, upcoming deadlines next week, and
any open blockers. Post as a formatted summary in the team channel.",
        capabilities: ["read_tasks", "read_messages", "read_meetings", "send_messages",
"search_workspace"],
        triggerType: "scheduled",
        triggerConfig: { cron: "0 16 * * 5" },
      },
    };

    const template = TEMPLATES[input.templateId];
    if (!template) throw new Error("Template not found");

    return prisma.aiAgent.create({
      data: {
        ...template,
        workspaceId: input.workspaceId,
        createdBy: ctx.user.id,
        type: "template",
        model: "gpt-4o",
        temperature: 0.5,
        visibility: "workspace",
        visibleToUserIds: [],
        dataAccess: { allPages: true, allDatabases: true, allChannels: true, allMeetings: true,
specificPageIds: [], specificDatabaseIds: [], specificChannelIds: [] },
        isActive: true,
        maxRunsPerDay: 100,
        maxActionsPerRun: 25,
        timeoutMinutes: 5,
      },
    });
  }),
});

// ============================================================
```

```
// AGENT TOOL DEFINITIONS & EXECUTION
// ==============================================================

function buildAgentTools(capabilities: string[]): any[] {
  const allTools: Record<string, any> = {
    read_pages: { type: "function", function: { name: "read_page", description: "Read the content of a page", parameters: { type: "object", properties: { pageId: { type: "string" } }, required: ["pageId"] } } },
    search_workspace: { type: "function", function: { name: "search_workspace", description: "Search across all workspace data", parameters: { type: "object", properties: { query: { type: "string" } }, required: ["query"] } } },
    read_tasks: { type: "function", function: { name: "read_tasks", description: "Read tasks from a database", parameters: { type: "object", properties: { filter: { type: "string", description: "Optional filter like 'status:in_progress' or 'assignee:alice'" } }, required: [] } } },
    write_tasks: { type: "function", function: { name: "create_task", description: "Create a new task", parameters: { type: "object", properties: { title: { type: "string" }, description: { type: "string" }, priority: { type: "string" }, dueDate: { type: "string" } }, required: ["title"] } } },
    send_messages: { type: "function", function: { name: "send_message", description: "Send a message to a channel", parameters: { type: "object", properties: { channelId: { type: "string" }, content: { type: "string" } }, required: ["channelId", "content"] } } },
    read_messages: { type: "function", function: { name: "read_messages", description: "Read recent messages from a channel", parameters: { type: "object", properties: { channelId: { type: "string" }, limit: { type: "number" } }, required: ["channelId"] } } },
  };

  return capabilities.filter(c => allTools[c]).map(c => allTools[c]);
}

async function executeAgentTool(toolName: string, args: any, agent: any, userId: string): Promise<any> {
  switch (toolName) {
    case "search_workspace":
      const searchResults = await prisma.$queryRaw`
        SELECT "sourceType", title, LEFT(content, 300) as content
        FROM "KnowledgeIndex"
        WHERE "workspaceId" = ${agent.workspaceId}::uuid
        AND to_tsvector('english', content) @@ plainto_tsquery('english', ${args.query})
        LIMIT 5
      `;
      return searchResults;

    case "read_tasks":
      return prisma.task.findMany({
        where: { workspaceId: agent.workspaceId, deletedAt: null },
```

```
      take: 20,
      orderBy: { updatedAt: "desc" },
      select: { id: true, title: true, status: true, priority: true, assigneeId: true, dueDate: true },
    });

  case "send_message":
    // Verify agent has access to this channel
    return { sent: true, channelId: args.channelId, content: args.content };

  case "read_page":
    return prisma.page.findUnique({
      where: { id: args.pageId },
      select: { id: true, title: true, content: true },
    });

  default:
    return { error: `Unknown tool: ${toolName}` };
 }
}
```

Agent Chat UI: `src/components/agents/agent-chat.tsx`

Build a standard chat interface:

- Left panel: list of agents with avatars, names, last message preview
- Right panel: conversation view with user messages (right-aligned) and agent messages (left-aligned)
- Bottom: message input with send button
- Each agent message shows: text response + collapsible "Steps" showing what tools the agent used
- "New Agent" button opens the agent builder form

Agent Builder UI: `src/components/agents/agent-builder.tsx`

Build a multi-section form:

- Name, Icon, Description fields
- Instructions textarea with "Generate from description" button
- Model selector dropdown
- Capabilities: checkboxes for each capability
- Data Access: radio buttons (All / Specific) with multi-select for specific items
- Trigger: radio buttons (Manual / Scheduled / Event / Message / Webhook) with config per type

- Visibility: radio buttons (Workspace / Specific members / Private)
- Safety: max runs/day, max actions/run, timeout inputs
- "Create Agent" button

FEATURE 19: AI WORKFLOW RECORDER

tRPC Router: `src/server/routers/workflows.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const workflowsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      return prisma.workflow.findMany({
        where: {
          workspaceId: input.workspaceId,
          deletedAt: null,
          OR: [{ isPublic: true }, { createdBy: ctx.user.id }],
        },
        orderBy: { createdAt: "desc" },
      });
    }),

  // Save recorded workflow steps
  saveRecording: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      name: z.string().min(1),
      description: z.string().optional(),
      recordedSteps: z.array(z.object({
        stepNumber: z.number(),
        actionType: z.string(),
        rawData: z.any(),
      })),
    }))
    .mutation(async ({ input, ctx }) => {
      // AI cleans up the raw steps and detects variables
      const aiResult = await aiService.generateJSON<{
```

```
      cleanedSteps: Array<{ stepNumber: number; actionType: string; cleanedData: any;
variables: Array<{ name: string; type: string; defaultValue?: string }> }>;
      variables: Array<{ name: string; type: string; description: string }>;
    }>(
      `Clean up these recorded workflow steps and detect reusable variables.

RECORDED STEPS:
${JSON.stringify(input.recordedSteps, null, 2)}

For each step:
1. Clean the data (remove unnecessary fields)
2. Detect variables (names, dates, project names, user references that would change each
time)
3. Replace hardcoded values with variable references like {{variable_name}}

Return: { cleanedSteps: [...], variables: [...] }`,
      { systemPrompt: "You are an automation expert. Clean workflow recordings and extract
variables.", temperature: 0.2 },
    );

    return prisma.workflow.create({
      data: {
        workspaceId: input.workspaceId,
        name: input.name,
        description: input.description,
        recordedSteps: aiResult.cleanedSteps,
        variables: aiResult.variables,
        createdBy: ctx.user.id,
      },
    });
  }),

  // Execute a workflow with variable values
  execute: protectedProcedure
    .input(z.object({
      workflowId: z.string().uuid(),
      variableValues: z.record(z.string()),
    }))
    .mutation(async ({ input, ctx }) => {
      const workflow = await prisma.workflow.findUnique({ where: { id: input.workflowId } });
      if (!workflow) throw new Error("Workflow not found");

      const run = await prisma.workflowRun.create({
        data: {
```

```
        workflowId: workflow.id,
        userId: ctx.user.id,
        variableValues: input.variableValues,
        steps: [],
        status: "running",
      },
    });

    // Execute each step
    const steps = workflow.recordedSteps as any[];
    const executedSteps: any[] = [];

    for (const step of steps) {
      try {
        // Replace variables in step data
        let stepData = JSON.stringify(step.cleanedData || step.rawData);
        for (const [key, value] of Object.entries(input.variableValues)) {
          stepData = stepData.replace(new RegExp(`{{${key}}}`, "g"), value);
        }
        const parsedData = JSON.parse(stepData);

        // Execute the action (adapt to your actual action types)
        const result = await executeWorkflowStep(step.actionType, parsedData,
workflow.workspaceId, ctx.user.id);
        executedSteps.push({ stepNumber: step.stepNumber, status: "success", output: result });
      } catch (error: any) {
        executedSteps.push({ stepNumber: step.stepNumber, status: "failed", error:
error.message });
      }
    }

    await prisma.workflowRun.update({
      where: { id: run.id },
      data: { steps: executedSteps, status: "completed", completedAt: new Date() },
    });

    await prisma.workflow.update({
      where: { id: workflow.id },
      data: { runCount: { increment: 1 } },
    });

    return { runId: run.id, steps: executedSteps };
  }),
```

```
  delete: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.workflow.update({ where: { id: input.id }, data: { deletedAt: new Date() } });
    }),
});

async function executeWorkflowStep(actionType: string, data: any, workspaceId: string, userId:
string): Promise<any> {
  switch (actionType) {
    case "create_task":
      return prisma.task.create({ data: { ...data, workspaceId, createdBy: userId } });
    case "create_page":
      return prisma.page.create({ data: { ...data, workspaceId, createdBy: userId } });
    case "send_message":
      // Use your messaging service
      return { sent: true };
    case "update_task":
      return prisma.task.update({ where: { id: data.taskId }, data: data.updates });
    default:
      throw new Error(`Unknown action type: ${actionType}`);
  }
}
```

EDGE CASES — SPRINT 4

Agents:
- Agent loops (calls same tool repeatedly) → Max 10 tool-call rounds, then force complete
- Agent tries to access data outside its config → Permission check rejects, log attempt
- Agent timeout → Kill after 5 min, status = 'timeout', notify user
- Scheduled agent fires but workspace has no relevant data → Agent responds "No new data to report"
- Multiple users chat with same shared agent → Separate conversations per user

Workflows:
- Recorded workflow has deleted task/page → Skip step, show warning "Step 3 skipped: original task was deleted"
- Variable value empty → Show validation error before execution
- Workflow has 0 steps → Prevent saving, show "Record at least one action"

VERIFICATION — SPRINT 4

□ Create agent from "Sprint Status Bot" template → agent appears in list
□ Chat with agent: "What tasks are due this week?" → agent searches and responds
□ Agent shows tool usage in collapsible steps panel
□ Create custom agent with scheduled trigger → save successfully
□ Agent builder form validates all required fields
□ Record a workflow (create task + send message) → save with detected variables
□ Execute workflow with variables filled in → steps execute, results shown
□ View workflow run history → see past executions with success/fail per step

---

SPRINT 5 — AI PROJECT MANAGER + TASK INTELLIGENCE + DATABASE AI

Features 5, 6, and 8

PREREQUISITE: Sprints 1-4 complete

FEATURE 5: AI PROJECT MANAGER

tRPC Router: `src/server/routers/project-manager.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { logAiUsage } from "@/server/services/ai-usage-logger";

export const projectManagerRouter = router({
  // Generate standup report
  generateStandup: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databaseId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const yesterday = new Date();
      yesterday.setDate(yesterday.getDate() - 1);
```

```
    // Get task changes in last 24 hours
    const recentTasks = await prisma.task.findMany({
      where: { workspaceId: input.workspaceId, databaseId: input.databaseId, updatedAt: { gte:
yesterday } },
      include: { assignee: { select: { id: true, name: true } } },
    });

    const completedTasks = recentTasks.filter(t => t.status === "done" || t.completedAt);
    const inProgressTasks = recentTasks.filter(t => t.status === "in_progress");
    const blockedTasks = recentTasks.filter(t => t.status === "blocked");
    const overdueTasks = await prisma.task.findMany({
      where: { databaseId: input.databaseId, dueDate: { lt: new Date() }, completedAt: null,
deletedAt: null },
      include: { assignee: { select: { name: true } } },
    });

    const totalTasks = await prisma.task.count({ where: { databaseId: input.databaseId,
deletedAt: null } });
    const doneTasks = await prisma.task.count({ where: { databaseId: input.databaseId,
completedAt: { not: null }, deletedAt: null } });
    const progressPercent = totalTasks > 0 ? Math.round((doneTasks / totalTasks) * 100) : 0;

    const result = await aiService.generateText(
      `Generate a daily standup report from this data:

COMPLETED (last 24h): ${JSON.stringify(completedTasks.map(t => ({ title: t.title, assignee:
t.assignee?.name })))}
IN PROGRESS: ${JSON.stringify(inProgressTasks.map(t => ({ title: t.title, assignee:
t.assignee?.name })))}
BLOCKED: ${JSON.stringify(blockedTasks.map(t => ({ title: t.title, assignee: t.assignee?.name
})))}
OVERDUE: ${JSON.stringify(overdueTasks.map(t => ({ title: t.title, assignee: t.assignee?.name,
daysOverdue: Math.ceil((Date.now() - (t.dueDate?.getTime() || 0)) / 86400000) })))}
PROGRESS: ${progressPercent}% complete (${doneTasks}/${totalTasks} tasks)

Format as a clear standup grouped by team member. Include sprint progress and any risks.`,
      { systemPrompt: "Generate a concise, well-formatted standup report.", maxTokens: 1500,
temperature: 0.3 },
    );

    const report = await prisma.projectStatusReport.create({
      data: {
        workspaceId: input.workspaceId,
        databaseId: input.databaseId,
```

```
        generatedBy: "ai_on_demand",
        reportType: "daily_standup",
        periodStart: yesterday,
        periodEnd: new Date(),
        content: {
          summary: result.text,
          completedTasks: completedTasks.map(t => ({ taskId: t.id, title: t.title, completedBy:
t.assignee?.name })),
          inProgressTasks: inProgressTasks.map(t => ({ taskId: t.id, title: t.title, assignee:
t.assignee?.name })),
          blockedTasks: blockedTasks.map(t => ({ taskId: t.id, title: t.title, assignee:
t.assignee?.name })),
          overdueTasks: overdueTasks.map(t => ({ taskId: t.id, title: t.title, assignee:
t.assignee?.name })),
          progressPercent,
        },
      },
    });

    return { report, summary: result.text };
  }),

  // Detect project risks
  detectRisks: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databaseId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, completedAt: null, deletedAt: null },
        include: { assignee: { select: { id: true, name: true } } },
      });

      const risks: any[] = [];

      // 1. Deadline at risk
      for (const task of tasks) {
        if (task.dueDate) {
          const daysLeft = Math.ceil((task.dueDate.getTime() - Date.now()) / 86400000);
          if (daysLeft <= 1 && task.status !== "in_progress") {
            risks.push({
              riskType: "deadline_at_risk",
              severity: daysLeft <= 0 ? "critical" : "high",
              title: `"${task.title}" deadline ${daysLeft <= 0 ? "overdue" : "tomorrow"}`,
              description: `Task "${task.title}" is due ${daysLeft <= 0 ? `${Math.abs(daysLeft)} days
overdue` : "tomorrow"} but status is "${task.status}".`,
```

```
            affectedTaskIds: [task.id],
            affectedUserIds: task.assigneeId ? [task.assigneeId] : [],
            suggestedAction: daysLeft <= 0 ? "Reassign or escalate immediately." : "Move to
in-progress and prioritize.",
          });
        }
      }
    }

    // 2. Team member overloaded
    const assigneeCounts: Record<string, { count: number; name: string; userId: string }> = {};
    for (const task of tasks) {
      if (task.assigneeId && task.assignee) {
        if (!assigneeCounts[task.assigneeId]) assigneeCounts[task.assigneeId] = { count: 0,
name: task.assignee.name, userId: task.assigneeId };
        assigneeCounts[task.assigneeId].count++;
      }
    }
    for (const [userId, data] of Object.entries(assigneeCounts)) {
      if (data.count > 8) {
        risks.push({
          riskType: "team_member_overloaded",
          severity: data.count > 12 ? "high" : "medium",
          title: `${data.name} has ${data.count} active tasks`,
          description: `${data.name} is assigned ${data.count} active tasks, which exceeds the
recommended limit of 8.`,
          affectedTaskIds: tasks.filter(t => t.assigneeId === userId).map(t => t.id),
          affectedUserIds: [userId],
          suggestedAction: `Consider redistributing ${data.count - 8} tasks to team members with
lighter workloads.`,
        });
      }
    }

    // 3. Unassigned urgent
    const unassignedUrgent = tasks.filter(t => !t.assigneeId && (t.priority === "high" || t.priority
=== "urgent"));
    if (unassignedUrgent.length > 0) {
      risks.push({
        riskType: "unassigned_urgent",
        severity: "medium",
        title: `${unassignedUrgent.length} urgent tasks have no assignee`,
        description: `${unassignedUrgent.map(t => `"${t.title}"`).join(", ")} need owners.`,
        affectedTaskIds: unassignedUrgent.map(t => t.id),
```

```
        affectedUserIds: [],
        suggestedAction: "Assign these tasks immediately based on team expertise and
availability.",
      });
    }

    // Save risks
    for (const risk of risks) {
      await prisma.aiRiskAlert.upsert({
        where: { id: risk.id || "00000000-0000-0000-0000-000000000000" },
        create: { ...risk, workspaceId: input.workspaceId, databaseId: input.databaseId },
        update: risk,
      });
    }

    return { risks, count: risks.length };
  }),

  // Get active risks
  getRisks: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databaseId: z.string().uuid().optional() }))
    .query(async ({ input }) => {
      return prisma.aiRiskAlert.findMany({
        where: {
          workspaceId: input.workspaceId,
          ...(input.databaseId ? { databaseId: input.databaseId } : {}),
          status: "active",
        },
        orderBy: [{ severity: "asc" }, { createdAt: "desc" }],
      });
    }),

  // Generate status report
  generateStatusReport: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), databaseId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, deletedAt: null },
        include: { assignee: { select: { name: true } } },
      });

      const total = tasks.length;
      const done = tasks.filter(t => t.completedAt).length;
      const inProgress = tasks.filter(t => t.status === "in_progress").length;
```

```
      const blocked = tasks.filter(t => t.status === "blocked").length;
      const overdue = tasks.filter(t => t.dueDate && t.dueDate < new Date() &&
!t.completedAt).length;

      const result = await aiService.generateText(
        `Generate an executive project status report:
Total tasks: ${total}, Done: ${done} (${Math.round(done/total*100)}%), In Progress:
${inProgress}, Blocked: ${blocked}, Overdue: ${overdue}

Task list: ${JSON.stringify(tasks.map(t => ({ title: t.title, status: t.status, assignee:
t.assignee?.name, dueDate: t.dueDate, priority: t.priority })))}

Write a concise executive summary (3-4 sentences), then list key highlights and concerns.`,
        { systemPrompt: "Write a professional project status report for stakeholders.", maxTokens:
1000, temperature: 0.3 },
      );

      return { report: result.text };
    }),

  // Create automation from natural language
  createAutomation: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      description: z.string(),
    }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
        name: string; trigger: any; conditions: any[]; actions: any[];
      }>(
        `Parse this automation description into a structured automation config:
"${input.description}"

Return JSON with: name, trigger ({type, config}), conditions ([{field, operator, value}]), actions
([{type, config}])
Trigger types: task_created, task_status_changed, task_overdue, scheduled
Action types: update_task, send_message, create_task, assign, notify`,
        { systemPrompt: "Parse natural language into automation configs.", temperature: 0.2 },
      );

      return prisma.aiWorkflowAutomation.create({
        data: {
          workspaceId: input.workspaceId,
          name: result.name,
```

```
        createdFromPrompt: input.description,
        trigger: result.trigger,
        conditions: result.conditions,
        actions: result.actions,
        createdBy: ctx.user.id,
      },
    });
  },
});
```

FEATURE 6: AI TASK INTELLIGENCE

tRPC Router: `src/server/routers/task-intelligence.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const taskIntelligenceRouter = router({
  // AI Task Breakdown (Goblin.tools-style)
  breakdown: protectedProcedure
    .input(z.object({
      taskTitle: z.string(),
      taskDescription: z.string().optional(),
      complexity: z.enum(["simple", "medium", "detailed", "very_detailed"]).default("medium"),
    }))
    .mutation(async ({ input }) => {
      const depthMap = { simple: "3-5", medium: "5-8", detailed: "8-12", very_detailed: "12-20" };
      const result = await aiService.generateJSON<{ subtasks: Array<{ title: string; subtasks?:
Array<{ title: string }> }> }>(
        `Break down this task into ${depthMap[input.complexity]} actionable subtasks:
Task: "${input.taskTitle}"
${input.taskDescription ? `Description: ${input.taskDescription}` : ""}

Return JSON: { subtasks: [{ title: string, subtasks?: [{ title: string }] }] }
Each subtask should be a clear, actionable item one person can complete.`,
        { systemPrompt: "Break tasks into clear, actionable subtasks.", temperature: 0.4 },
      );
      return result;
    }),
```

```
  // AI Prioritize all tasks in a database
  prioritize: protectedProcedure
    .input(z.object({ databaseId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, completedAt: null, deletedAt: null },
        select: { id: true, title: true, description: true, dueDate: true, priority: true, status: true },
      });

      const result = await aiService.generateJSON<{ ranked: Array<{ taskId: string;
suggestedPriority: string; reason: string }> }>(
        `Rank these tasks by priority (critical > high > medium > low):
${JSON.stringify(tasks)}

Consider: due date urgency, task complexity, blocking potential, business impact.
Return: { ranked: [{ taskId, suggestedPriority: 'critical'|'high'|'medium'|'low', reason: string }] }`,
        { systemPrompt: "Prioritize tasks based on urgency and impact.", temperature: 0.2 },
      );
      return result;
    }),

  // Natural language task creation
  parseNaturalLanguage: protectedProcedure
    .input(z.object({ text: z.string(), workspaceId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateJSON<{
        title: string; description?: string; dueDate?: string; priority?: string;
        project?: string; assignee?: string; estimatedMinutes?: number;
      }>(
        `Parse this natural language into a structured task:
"${input.text}"

Extract: title, description, dueDate (ISO format), priority (low/medium/high/urgent), project name
(if mentioned), assignee name (if mentioned), estimatedMinutes.
Return JSON. Only include fields that are explicitly mentioned or clearly implied.`,
        { systemPrompt: "Parse natural language into structured task data.", temperature: 0.2 },
      );
      return result;
    }),

  // AI Duration Estimation
  estimateDuration: protectedProcedure
    .input(z.object({ taskId: z.string().uuid() }))
    .mutation(async ({ input }) => {
```

```
      const task = await prisma.task.findUnique({ where: { id: input.taskId } });
      if (!task) throw new Error("Task not found");

      const result = await aiService.generateJSON<{ estimatedMinutes: number; confidence:
number; reasoning: string }>(
        `Estimate how long this task will take:
Title: "${task.title}"
Description: "${task.description || "No description"}"

Return: { estimatedMinutes: number (round to nearest 15), confidence: 0-1, reasoning: string }`,
        { systemPrompt: "Estimate task durations based on complexity.", temperature: 0.3 },
      );

      await prisma.taskDurationEstimate.upsert({
        where: { taskId: task.id },
        create: { taskId: task.id, workspaceId: task.workspaceId, estimatedMinutes:
result.estimatedMinutes, confidence: result.confidence },
        update: { estimatedMinutes: result.estimatedMinutes, confidence: result.confidence },
      });

      return result;
    }),

  // AI Assignee Suggestion
  suggestAssignee: protectedProcedure
    .input(z.object({ taskId: z.string().uuid(), workspaceId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const task = await prisma.task.findUnique({ where: { id: input.taskId } });
      if (!task) throw new Error("Task not found");

      const members = await prisma.workspaceMember.findMany({
        where: { workspaceId: input.workspaceId },
        include: { user: { select: { id: true, name: true } } },
      });

      // Get task counts per member
      const memberWorkloads = await Promise.all(
        members.map(async (m) => {
          const activeCount = await prisma.task.count({
            where: { assigneeId: m.userId, completedAt: null, deletedAt: null },
          });
          return { userId: m.userId, name: m.user.name, activeTasks: activeCount };
        })
      );
```

```
      const result = await aiService.generateJSON<{ suggestions: Array<{ userId: string; name:
string; score: number; reason: string }> }>(
        `Suggest the best assignee for this task:
Task: "${task.title}" - ${task.description || "No description"}

Team members and their current workload:
${JSON.stringify(memberWorkloads)}

Return top 3 suggestions with score (0-100) and reasoning.`,
        { systemPrompt: "Suggest task assignees based on skills and workload.", temperature: 0.3
},
      );
      return result;
    }),
});
```

FEATURE 8: AI DATABASE INTELLIGENCE

tRPC Router: `src/server/routers/database-ai.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const databaseAiRouter = router({
  // AI Auto-fill for a column
  autoFill: protectedProcedure
    .input(z.object({
      databaseId: z.string().uuid(),
      taskId: z.string().uuid(),
      columnId: z.string().uuid(),
    }))
    .mutation(async ({ input }) => {
      const column = await prisma.aiAutoFillColumn.findUnique({ where: { id: input.columnId } });
      if (!column) throw new Error("Auto-fill column not found");

      const task = await prisma.task.findUnique({ where: { id: input.taskId } });
      if (!task) throw new Error("Task not found");

      // Build input from specified fields
```

```
      const inputData: Record<string, string> = {};
      for (const field of column.inputFields) {
        inputData[field] = (task as any)[field] || "";
      }

      const result = await aiService.generateText(
        `${column.aiPrompt}\\n\\nInput data:\\n${JSON.stringify(inputData)}`,
        { systemPrompt: "Generate the requested value based on the input data. Return ONLY the
value, nothing else.", maxTokens: 200, temperature: 0.2 },
      );

      return { value: result.text.trim() };
    }),

  // Natural language database query
  queryDatabase: protectedProcedure
    .input(z.object({ databaseId: z.string().uuid(), query: z.string() }))
    .mutation(async ({ input }) => {
      const tasks = await prisma.task.findMany({
        where: { databaseId: input.databaseId, deletedAt: null },
        include: { assignee: { select: { name: true } } },
        take: 100,
      });

      const result = await aiService.generateText(
        `Answer this question about the database:
"${input.query}"

DATABASE CONTENTS (${tasks.length} rows):
${JSON.stringify(tasks.map(t => ({ title: t.title, status: t.status, priority: t.priority, assignee:
t.assignee?.name, dueDate: t.dueDate, createdAt: t.createdAt })))}

Provide a direct answer. If it's a count, give the number. If it's a list, list the items.`,
        { systemPrompt: "Answer database queries accurately and concisely.", maxTokens: 500,
temperature: 0.2 },
      );
      return { answer: result.text };
    }),

  // AI Database Creation from description
  createFromDescription: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), description: z.string() }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
```

```
        name: string;
        properties: Array<{ name: string; type: string; options?: string[] }>;
        views: Array<{ name: string; type: string; groupBy?: string }>;
        sampleRows: Array<Record<string, any>>;
      }>(
        `Create a database structure for: "${input.description}"

Return JSON with:
- name: database name
- properties: [{name, type: 'text'|'select'|'date'|'number'|'person'|'checkbox'|'url'|'email', options?: []
for select type}]
- views: [{name, type: 'table'|'board'|'calendar', groupBy?: property name for board view}]
- sampleRows: 3 example rows with data

Make it practical and complete.`,
        { systemPrompt: "Design practical database structures.", temperature: 0.4 },
      );
      return result;
    }),

  // AI Formula Generation
  generateFormula: protectedProcedure
    .input(z.object({ description: z.string(), properties: z.array(z.object({ name: z.string(), type:
z.string() })) }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
        `Generate a formula for: "${input.description}"
Available properties: ${JSON.stringify(input.properties)}
Return ONLY the formula expression, nothing else.`,
        { systemPrompt: "Generate database formulas.", maxTokens: 200, temperature: 0.1 },
      );
      return { formula: result.text.trim() };
    }),

  // CRUD for auto-fill columns
  createAutoFillColumn: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(), databaseId: z.string().uuid(),
      columnName: z.string(), aiPrompt: z.string(), inputFields: z.array(z.string()),
      outputType: z.enum(["text", "select", "number"]).default("text"),
      selectOptions: z.array(z.string()).optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.aiAutoFillColumn.create({
```

```
      data: { ...input, createdBy: ctx.user.id },
    });
  }),
});
```

VERIFICATION — SPRINT 5

□ Click "Generate Standup" on project board → formatted standup report appears
□ Click "Detect Risks" → risk cards shown with severity levels and suggestions
□ Click "Generate Status Report" → executive summary generated from real task data
□ Type natural language automation → AI parses → preview shown → confirm creates automation
□ Click "Break Down" on a task → subtask tree generated with adjustable complexity
□ Click "AI Prioritize" on database → all tasks get priority suggestions with reasoning
□ Type "Review Alice's PR tomorrow high priority" → AI parses into structured task fields
□ Click "Estimate Duration" → shows "~45 min" with confidence score
□ Click "Suggest Assignee" → top 3 members with scores and reasons
□ Add AI Auto-Fill column → new rows auto-populate with AI-generated values
□ Type natural language query on database → get accurate answer
□ Describe a database → AI creates with properties, views, sample data

---

SPRINT 6 — MEETING INTELLIGENCE + DOCUMENT GENERATOR

Features 7 and 10

PREREQUISITE: Sprints 1-5 complete

FEATURE 7: AI MEETING INTELLIGENCE

tRPC Router: `src/server/routers/meetings.ts`

```
import { z } from "zod";
```

```
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import { meetingQueue } from "@/jobs/queue-config";
import { indexTask } from "@/server/services/knowledge-indexer";

export const meetingsRouter = router({
  // Process meeting (transcription + AI summary) — triggered after recording ends
  process: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid(), audioUrl: z.string() }))
    .mutation(async ({ input }) => {
      // Create intelligence record
      const intel = await prisma.meetingIntelligence.create({
        data: {
          meetingId: input.meetingId,
          workspaceId: (await prisma.meeting.findUnique({ where: { id: input.meetingId }
}))!.workspaceId,
          audioUrl: input.audioUrl,
          processingStatus: "transcribing",
        },
      });

      // Enqueue for background processing
      await meetingQueue.add("process-meeting", {
        intelligenceId: intel.id,
        meetingId: input.meetingId,
        audioUrl: input.audioUrl,
      });

      return { intelligenceId: intel.id, status: "processing" };
    }),

  // Get meeting intelligence (summary, action items, etc.)
  getIntelligence: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.meetingIntelligence.findUnique({
        where: { meetingId: input.meetingId },
      });
    }),

  // Create tasks from action items
  createTasksFromActionItems: protectedProcedure
    .input(z.object({
```

```
    meetingId: z.string().uuid(),
    databaseId: z.string().uuid(),
    actionItemIndices: z.array(z.number()), // which action items to create tasks for
  }))
  .mutation(async ({ input, ctx }) => {
    const intel = await prisma.meetingIntelligence.findUnique({ where: { meetingId:
input.meetingId } });
    if (!intel || !intel.actionItems) throw new Error("No action items found");

    const actionItems = intel.actionItems as any[];
    const createdTasks: any[] = [];

    for (const index of input.actionItemIndices) {
      const item = actionItems[index];
      if (!item) continue;

      const task = await prisma.task.create({
        data: {
          title: item.title,
          description: item.description || `Action item from meeting`,
          workspaceId: intel.workspaceId,
          databaseId: input.databaseId,
          createdBy: ctx.user.id,
          dueDate: item.suggestedDueDate ? new Date(item.suggestedDueDate) : undefined,
        },
      });

      // Update action item to show task was created
      actionItems[index].taskCreated = true;
      actionItems[index].taskId = task.id;
      createdTasks.push(task);

      // Index the new task for knowledge search
      await indexTask(task.id, intel.workspaceId);
    }

    // Update meeting intelligence with created task IDs
    await prisma.meetingIntelligence.update({
      where: { meetingId: input.meetingId },
      data: { actionItems: actionItems },
    });

    return { created: createdTasks.length, tasks: createdTasks };
  }),
```

```
  // Generate meeting prep brief
  generatePrepBrief: protectedProcedure
    .input(z.object({ meetingId: z.string().uuid(), workspaceId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const meeting = await prisma.meeting.findUnique({ where: { id: input.meetingId } });
      if (!meeting) throw new Error("Meeting not found");

      // Gather context: recent tasks, last meeting's action items, relevant channel discussions
      const recentTasks = await prisma.task.findMany({
        where: { workspaceId: input.workspaceId, updatedAt: { gte: new Date(Date.now() - 7 *
86400000) } },
        take: 15,
        orderBy: { updatedAt: "desc" },
      });

      const result = await aiService.generateText(
        `Generate a meeting prep brief for: "${meeting.title}"
Meeting time: ${meeting.startTime}
Recent task updates: ${JSON.stringify(recentTasks.map(t => ({ title: t.title, status: t.status })))}

Include: agenda suggestions, relevant context, open questions to discuss.`,
        { systemPrompt: "Generate concise meeting prep briefs.", maxTokens: 800, temperature:
0.4 },
      );

      await prisma.meetingIntelligence.upsert({
        where: { meetingId: input.meetingId },
        create: { meetingId: input.meetingId, workspaceId: input.workspaceId, prepBrief: result.text
},
        update: { prepBrief: result.text },
      });

      return { prepBrief: result.text };
    }),
});
```

Background Job: `src/jobs/process-meeting.ts`

```
// src/jobs/process-meeting.ts
import { Worker } from "bullmq";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import OpenAI from "openai";
```

```
import { redis } from "@/lib/redis";

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

const worker = new Worker("meeting", async (job) => {
  const { intelligenceId, meetingId, audioUrl } = job.data;

  try {
    // Step 1: Transcribe with Whisper
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "transcribing" },
    });

    // Download audio and send to Whisper
    const audioResponse = await fetch(audioUrl);
    const audioBuffer = await audioResponse.arrayBuffer();
    const audioFile = new File([audioBuffer], "meeting.webm", { type: "audio/webm" });

    const transcription = await openai.audio.transcriptions.create({
      model: "whisper-1",
      file: audioFile,
      response_format: "verbose_json",
      timestamp_granularities: ["segment"],
    });

    const transcript = transcription.text;
    const segments = (transcription as any).segments || [];

    // Step 2: Generate AI summary
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "summarizing", transcriptRaw: transcript, transcriptSegments:
segments },
    });

    const summaryResult = await aiService.generateJSON<{
      summary: string;
      keyPoints: string[];
      decisions: Array<{ decision: string; context: string }>;
      actionItems: Array<{ title: string; description?: string; suggestedOwner?: string;
suggestedDueDate?: string }>;
      openQuestions: string[];
    }>(
```

```
      `Analyze this meeting transcript and extract structured information:

TRANSCRIPT:
${transcript.slice(0, 15000)}

Return JSON with: summary (3-5 sentences), keyPoints (array of strings), decisions (array of
{decision, context}), actionItems (array of {title, description, suggestedOwner,
suggestedDueDate}), openQuestions (array of strings).`,
      { systemPrompt: "Extract structured meeting information from transcripts.", temperature: 0.2
},
    );

    // Step 3: Save everything
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: {
        processingStatus: "completed",
        aiSummary: summaryResult.summary,
        keyPoints: summaryResult.keyPoints,
        decisions: summaryResult.decisions,
        actionItems: summaryResult.actionItems.map(a => ({ ...a, taskCreated: false })),
        openQuestions: summaryResult.openQuestions,
        audioDurationSecs: Math.round(segments.length > 0 ? segments[segments.length - 1].end
: 0),
      },
    });

    // Step 4: Index transcript for knowledge search
    const workspace = await prisma.meetingIntelligence.findUnique({
      where: { id: intelligenceId },
      select: { workspaceId: true },
    });
    if (workspace) {
      const { indexPage } = await import("@/server/services/knowledge-indexer");
      // Index as meeting_transcript type
    }

  } catch (error: any) {
    await prisma.meetingIntelligence.update({
      where: { id: intelligenceId },
      data: { processingStatus: "failed", processingError: error.message },
    });
    throw error;
  }
```

```
}, { connection: redis });

export default worker;
```

FEATURE 10: AI DOCUMENT GENERATOR

tRPC Router: `src/server/routers/doc-generator.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const docGeneratorRouter = router({
  generate: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      templateType: z.enum([
        "weekly_status", "meeting_agenda", "project_brief", "sprint_retro",
        "handoff_doc", "client_update", "onboarding_guide",
      ]),
      config: z.object({
        databaseId: z.string().uuid().optional(),
        channelId: z.string().uuid().optional(),
        dateRangeStart: z.string().optional(),
        dateRangeEnd: z.string().optional(),
        projectName: z.string().optional(),
      }),
    }))
    .mutation(async ({ input, ctx }) => {
      const { workspaceId, templateType, config } = input;

      // Gather real data based on template type
      let contextData = "";

      if (templateType === "weekly_status" || templateType === "sprint_retro") {
        const startDate = config.dateRangeStart ? new Date(config.dateRangeStart) : new
Date(Date.now() - 7 * 86400000);
        const endDate = config.dateRangeEnd ? new Date(config.dateRangeEnd) : new Date();

        const tasks = config.databaseId
```

```
        ? await prisma.task.findMany({ where: { databaseId: config.databaseId, updatedAt: { gte:
startDate, lte: endDate } }, include: { assignee: { select: { name: true } } }, take: 50 })
        : [];

      const completed = tasks.filter(t => t.completedAt && t.completedAt >= startDate);
      const inProgress = tasks.filter(t => t.status === "in_progress");
      const blocked = tasks.filter(t => t.status === "blocked");

      contextData = `Period: ${startDate.toLocaleDateString()} to
${endDate.toLocaleDateString()}
Tasks completed: ${completed.length} (${completed.map(t => t.title).join(", ")})
In progress: ${inProgress.length} (${inProgress.map(t => t.title).join(", ")})
Blocked: ${blocked.length} (${blocked.map(t => t.title).join(", ")})`;
    }

    if (templateType === "meeting_agenda") {
      const recentTasks = config.databaseId
        ? await prisma.task.findMany({ where: { databaseId: config.databaseId, status: { in:
["in_progress", "blocked"] } }, take: 10 })
        : [];
      contextData = `Open tasks: ${JSON.stringify(recentTasks.map(t => ({ title: t.title, status:
t.status })))}`;
    }

    const TEMPLATE_PROMPTS: Record<string, string> = {
      weekly_status: "Generate a weekly status report with: Executive Summary, Completed This
Week, In Progress, Blocked Items, Risks, Plan for Next Week.",
      meeting_agenda: "Generate a meeting agenda with: Review Action Items, Status Updates,
Discussion Topics, Decisions Needed, Next Steps.",
      project_brief: `Generate a project brief for "${config.projectName || "Project"}" with:
Overview, Objectives, Requirements, Timeline, Team, Risks.`,
      sprint_retro: "Generate a sprint retrospective with: Sprint Summary, What Went Well, What
Could Improve, Action Items for Next Sprint.",
      handoff_doc: "Generate a handoff document with: Project Overview, Current State, Key
Decisions Made, Open Items, Contact Points, Important Resources.",
      client_update: "Generate a client-facing project update with: Summary, Progress,
Milestones, Next Steps, Timeline.",
      onboarding_guide: "Generate an onboarding guide with: Welcome, Getting Started, Key
Resources, Team Overview, First Week Tasks.",
    };

    const result = await aiService.generateText(
      `${TEMPLATE_PROMPTS[templateType]}
```

```
WORKSPACE DATA:
${contextData}

Write a complete, professional document using the real data provided.`,
        { systemPrompt: "Generate professional documents from workspace data.", maxTokens:
3000, temperature: 0.4 },
      );

      return { content: result.text, templateType };
    }),
});
```

VERIFICATION — SPRINT 6

□ Record a meeting → stop → processing starts
□ After processing: summary, key points, decisions, action items displayed
□ Click "Create All Tasks" from action items → tasks created in database
□ Click "Generate Prep Brief" before meeting → brief shown with context
□ Generate weekly status report → pulls real task data, creates formatted report
□ Generate meeting agenda → includes open tasks and action items
□ Generate project brief → structured document with all sections
□ All generated documents are editable in the document editor

---

SPRINT 7 — MESSAGING INTELLIGENCE + EMAIL ASSISTANT

Features 11 and 13

PREREQUISITE: Sprints 1-6 complete

FEATURE 11: AI MESSAGING INTELLIGENCE

tRPC Router: `src/server/routers/messaging-ai.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const messagingAiRouter = router({
  // Summarize a thread
  summarizeThread: protectedProcedure
    .input(z.object({ threadParentId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: { OR: [{ id: input.threadParentId }, { threadParentId: input.threadParentId }] },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
      });

      const result = await aiService.generateText(
        `Summarize this thread in 3-5 sentences:\\n\\n${messages.map(m => `${m.author.name}:
${m.content}`).join("\\n")}`,
        { systemPrompt: "Summarize message threads concisely.", maxTokens: 300, temperature:
0.3 },
      );
      return { summary: result.text };
    }),

  // Channel catch-up
  catchUp: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), since: z.string() })) // ISO datetime
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: { channelId: input.channelId, createdAt: { gte: new Date(input.since) } },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
        take: 100,
      });

      if (messages.length === 0) return { summary: "Nothing new since you were last here." };

      const result = await aiService.generateText(
        `Summarize what happened in this channel (${messages.length}
messages):\\n\\n${messages.map(m => `${m.author.name}: ${m.content}`).join("\\n")}`,
        { systemPrompt: "Provide a catch-up summary. Highlight: key decisions, important
updates, questions that need answers, action items.", maxTokens: 500, temperature: 0.3 },
```

```
      );
      return { summary: result.text, messageCount: messages.length };
    }),

  // Smart reply suggestions
  suggestReplies: protectedProcedure
    .input(z.object({ messageId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const message = await prisma.message.findUnique({
        where: { id: input.messageId },
        include: { author: { select: { name: true } } },
      });
      if (!message) throw new Error("Message not found");

      const result = await aiService.generateJSON<{ replies: string[] }>(
        `Suggest 3 short reply options for this message:
"${message.author.name}: ${message.content}"

Return JSON: { replies: ["reply1", "reply2", "reply3"] }
Make replies natural, varied (one agreeing, one asking follow-up, one providing info).`,
        { systemPrompt: "Generate natural reply suggestions.", temperature: 0.7 },
      );
      return result;
    }),

  // AI compose in messaging
  compose: protectedProcedure
    .input(z.object({
      action: z.enum(["draft_reply", "make_professional", "translate"]),
      inputText: z.string().optional(),
      threadContext: z.string().optional(),
      language: z.string().optional(),
    }))
    .mutation(async ({ input }) => {
      const prompts: Record<string, string> = {
        draft_reply: `Draft a reply to this conversation:\\n${input.threadContext}\\n\\nDraft a helpful,
professional response.`,
        make_professional: `Rewrite this message professionally:\\n"${input.inputText}"`,
        translate: `Translate to ${input.language || "English"}:\\n"${input.inputText}"`,
      };

      const result = await aiService.generateText(prompts[input.action], {
        systemPrompt: "Write clear, professional messages.", maxTokens: 500, temperature: 0.5,
      });
```

```
      return { text: result.text };
    }),

  // Extract decisions from channel
  extractDecisions: protectedProcedure
    .input(z.object({ channelId: z.string().uuid(), dateRange: z.object({ start: z.string(), end:
z.string() }) }))
    .mutation(async ({ input }) => {
      const messages = await prisma.message.findMany({
        where: { channelId: input.channelId, createdAt: { gte: new Date(input.dateRange.start), lte:
new Date(input.dateRange.end) } },
        include: { author: { select: { name: true } } },
        orderBy: { createdAt: "asc" },
        take: 200,
      });

      const result = await aiService.generateJSON<{ decisions: Array<{ decision: string;
decidedBy: string; context: string; date: string }> }>(
        `Extract all decisions made in these messages:\\n${messages.map(m =>
`[${m.createdAt.toISOString()}] ${m.author.name}: ${m.content}`).join("\\n")}

Return: { decisions: [{decision, decidedBy, context, date}] }`,
        { systemPrompt: "Extract team decisions from conversation history.", temperature: 0.2 },
      );
      return result;
    }),
});
```

FEATURE 13: AI EMAIL ASSISTANT

tRPC Router: `src/server/routers/email.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const emailRouter = router({
  // AI compose email
  compose: protectedProcedure
    .input(z.object({
      action: z.enum(["compose", "reply", "follow_up"]),
```

```
      subject: z.string().optional(),
      to: z.string().optional(),
      threadContext: z.string().optional(),
      instructions: z.string().optional(),
      tone: z.enum(["professional", "casual", "friendly", "formal"]).default("professional"),
    }))
    .mutation(async ({ input }) => {
      const prompts: Record<string, string> = {
        compose: `Write an email:\\nTo: ${input.to}\\nSubject: ${input.subject}\\nInstructions:
${input.instructions}\\nTone: ${input.tone}`,
        reply: `Write a reply to this email thread:\\n${input.threadContext}\\n\\nInstructions:
${input.instructions || "Write a professional reply."}\\nTone: ${input.tone}`,
        follow_up: `Write a follow-up email for this
thread:\\n${input.threadContext}\\n\\nInstructions: ${input.instructions || "Politely follow
up."}\\nTone: ${input.tone}`,
      };

      const result = await aiService.generateText(prompts[input.action], {
        systemPrompt: "Write professional emails. Include subject line if composing new email.",
        maxTokens: 1000, temperature: 0.5,
      });
      return { text: result.text };
    }),

  // Triage emails by priority
  triageInbox: protectedProcedure
    .input(z.object({ emailAccountId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const threads = await prisma.emailThread.findMany({
        where: { emailAccountId: input.emailAccountId, aiPriority: null },
        take: 50,
        orderBy: { lastMessageAt: "desc" },
      });

      for (const thread of threads) {
        const result = await aiService.generateJSON<{ priority: string; labels: string[] }>(
          `Classify this email:
Subject: ${thread.subject}
Preview: ${thread.latestSnippet}
From: ${JSON.stringify(thread.participants)}

Return: { priority: 'urgent'|'important'|'fyi'|'low', labels:
['action_required'|'waiting'|'informational'|'scheduled'] }`,
          { systemPrompt: "Triage emails by priority.", temperature: 0.2 },
```

```
      );

      await prisma.emailThread.update({
        where: { id: thread.id },
        data: { aiPriority: result.priority, aiLabels: result.labels },
      });
    }

    return { triaged: threads.length };
  }),

  // Summarize email thread
  summarize: protectedProcedure
    .input(z.object({ threadId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const thread = await prisma.emailThread.findUnique({ where: { id: input.threadId } });
      if (!thread) throw new Error("Thread not found");

      const result = await aiService.generateText(
        `Summarize this email thread:\\nSubject: ${thread.subject}\\nContent:
${thread.latestSnippet}`,
        { systemPrompt: "Summarize email threads concisely.", maxTokens: 300, temperature: 0.3
},
      );

      await prisma.emailThread.update({
        where: { id: input.threadId },
        data: { aiSummary: result.text },
      });

      return { summary: result.text };
    }),
});
```

VERIFICATION — SPRINT 7

□ Click "Summarize" on a long thread → 3-5 sentence summary appears
□ Click "Catch me up" in channel → summary since last visit shown
□ Reply suggestions appear as quick-reply pills below messages
□ AI compose in message composer → draft generated
□ Extract decisions from channel → list of decisions with who/when
□ AI compose email → professional email generated
□ Inbox triage → emails categorized as urgent/important/fyi/low

□ Email thread summarization works

Continue to Sprints 8-

SPRINTS 8-11 — REMAINING FEATURES + POLISH

PREREQUISITE: Sprints 1-7 complete

PASTE ONE SPRINT AT A TIME INTO REPLIT AI AGENT

SPRINT 8 — GOALS + CALENDAR OPTIMIZER + TIME TRACKING

Features 15, 16, and 21

FEATURE 15: AI GOAL & OKR SYSTEM

tRPC Router: `src/server/routers/goals.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const goalsRouter = router({
  list: protectedProcedure
```

```
    .input(z.object({ workspaceId: z.string().uuid(), level: z.string().optional() }))
    .query(async ({ input }) => {
      return prisma.goal.findMany({
        where: { workspaceId: input.workspaceId, deletedAt: null, ...(input.level ? { level: input.level
} : {}) },
        include: { keyResults: true },
        orderBy: { createdAt: "desc" },
      });
    }),

  create: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      title: z.string().min(1),
      description: z.string().optional(),
      level: z.enum(["company", "team", "individual"]),
      parentGoalId: z.string().uuid().optional(),
      startDate: z.string(),
      targetDate: z.string(),
      linkedDatabaseIds: z.array(z.string().uuid()).optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.goal.create({
        data: { ...input, startDate: new Date(input.startDate), targetDate: new
Date(input.targetDate), ownerId: ctx.user.id },
      });
    }),

  addKeyResult: protectedProcedure
    .input(z.object({
      goalId: z.string().uuid(),
      title: z.string(),
      metricType: z.enum(["number", "percentage", "currency", "boolean"]),
      targetValue: z.number(),
      startValue: z.number().default(0),
      unit: z.string().optional(),
      autoTrackSource: z.string().optional(),
      autoTrackConfig: z.any().optional(),
    }))
    .mutation(async ({ input }) => {
      return prisma.keyResult.create({ data: input });
    }),

  updateKeyResultProgress: protectedProcedure
```

```
    .input(z.object({ keyResultId: z.string().uuid(), currentValue: z.number() }))
    .mutation(async ({ input }) => {
      const kr = await prisma.keyResult.findUnique({ where: { id: input.keyResultId } });
      if (!kr) throw new Error("Key result not found");

      const progress = kr.targetValue !== kr.startValue
        ? Math.min(100, Math.max(0, ((input.currentValue - kr.startValue) / (kr.targetValue -
kr.startValue)) * 100))
        : 0;

      await prisma.keyResult.update({
        where: { id: input.keyResultId },
        data: { currentValue: input.currentValue, progressPercent: progress },
      });

      // Recalculate goal progress (average of all key results)
      const allKRs = await prisma.keyResult.findMany({ where: { goalId: kr.goalId } });
      const avgProgress = allKRs.reduce((sum, k) => sum + k.progressPercent, 0) / allKRs.length;

      const goalStatus = avgProgress >= 100 ? "completed" : avgProgress < 30 ? "behind" :
avgProgress < 60 ? "at_risk" : "on_track";

      await prisma.goal.update({
        where: { id: kr.goalId },
        data: { progressPercent: avgProgress, status: goalStatus },
      });

      return { progress, goalStatus };
    }),

  // AI suggest key results for a goal
  suggestKeyResults: protectedProcedure
    .input(z.object({ goalTitle: z.string(), goalDescription: z.string().optional() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateJSON<{ keyResults: Array<{ title: string; metricType:
string; targetValue: number; unit: string }> }>(
        `Suggest 3-5 measurable key results for this goal:
"${input.goalTitle}" ${input.goalDescription ? `- ${input.goalDescription}` : ""}

Return: { keyResults: [{title, metricType: 'number'|'percentage'|'currency'|'boolean', targetValue,
unit}] }
Each key result must be specific and measurable.`,
        { systemPrompt: "Suggest measurable OKR key results.", temperature: 0.5 },
      );
```

```
      return result;
    }),

  // AI risk assessment for a goal
  assessRisk: protectedProcedure
    .input(z.object({ goalId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const goal = await prisma.goal.findUnique({
        where: { id: input.goalId },
        include: { keyResults: true },
      });
      if (!goal) throw new Error("Goal not found");

      const daysLeft = Math.ceil((goal.targetDate.getTime() - Date.now()) / 86400000);

      const result = await aiService.generateText(
        `Assess risk for this goal:
"${goal.title}" — ${goal.progressPercent}% complete, ${daysLeft} days remaining
Key Results: ${JSON.stringify(goal.keyResults.map(kr => ({ title: kr.title, progress:
kr.progressPercent })))}

Provide: risk level (low/medium/high), reasoning, and 2-3 specific recommendations.`,
        { systemPrompt: "Assess goal risks and provide actionable recommendations.",
maxTokens: 500, temperature: 0.3 },
      );

      await prisma.goal.update({ where: { id: input.goalId }, data: { aiRiskAssessment: result.text }
});
      return { assessment: result.text };
    }),
});
```

Goal UI: `src/app/(dashboard)/[workspaceId]/goals/page.tsx`

Build a page with:

- Goal hierarchy tree view (Company > Team > Individual)
- Each goal card: title, progress ring (0-100%), status badge, key results list
- Click goal → expand key results with progress bars
- "Add Goal" button → modal with form (title, level, parent, dates)
- "Add Key Result" → inline form within goal card
- "Suggest Key Results" button → AI generates suggestions
- "Assess Risk" button → AI risk assessment card

FEATURE 16: AI CALENDAR OPTIMIZER

tRPC Router: `src/server/routers/calendar-optimizer.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const calendarOptimizerRouter = router({
  // Find best meeting time for multiple participants
  findMeetingTime: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      participantIds: z.array(z.string().uuid()),
      durationMinutes: z.number().min(15).max(480),
      dateRange: z.object({ start: z.string(), end: z.string() }),
      preferMorning: z.boolean().default(false),
    }))
    .mutation(async ({ input }) => {
      // Get all participants' busy times from calendar
      const busySlots = await Promise.all(
        input.participantIds.map(async (userId) => {
          const events = await prisma.meeting.findMany({
            where: {
              participants: { some: { userId } },
              startTime: { gte: new Date(input.dateRange.start), lte: new Date(input.dateRange.end)
},
            },
            select: { startTime: true, endTime: true },
          });
          // Also get focus blocks
          const focusBlocks = await prisma.focusTimeBlock.findMany({
            where: {
              userId,
              date: { gte: new Date(input.dateRange.start), lte: new Date(input.dateRange.end) },
            },
          });
          return { userId, events, focusBlocks };
        })
      );
```

```
      // Get preferences
      const prefs = await Promise.all(
        input.participantIds.map(async (userId) => {
          return prisma.schedulingPreference.findUnique({ where: { userId } });
        })
      );

      // Use AI to find optimal slots
      const result = await aiService.generateJSON<{ suggestions: Array<{ date: string; startTime:
string; endTime: string; score: number; reason: string }> }>(
        `Find the best ${input.durationMinutes}-minute meeting slots for
${input.participantIds.length} people.

BUSY TIMES: ${JSON.stringify(busySlots.map(s => ({ userId: s.userId, events: s.events.map(e
=> ({ start: e.startTime, end: e.endTime })) })))}
PREFERENCES: ${JSON.stringify(prefs.map(p => ({ workingHours: p?.workingHours,
preferMeetingsIn: p?.preferMeetingsIn })))}
DATE RANGE: ${input.dateRange.start} to ${input.dateRange.end}
PREFER MORNING: ${input.preferMorning}

Return top 5 suggestions: { suggestions: [{date, startTime, endTime, score: 0-100, reason}] }
Score based on: respects working hours, avoids focus blocks, minimal disruption, consistent
timing.`,
        { systemPrompt: "Find optimal meeting times.", temperature: 0.2 },
      );
      return result;
    }),

  // Get/set optimization rules
  getRules: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.calendarOptimizationRule.findMany({
        where: { workspaceId: input.workspaceId, isActive: true },
      });
    }),

  createRule: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      ruleType: z.enum(["no_meeting_window", "buffer_time", "meeting_defrag",
"focus_protection"]),
      config: z.any(),
      scope: z.enum(["workspace", "team", "individual"]).default("workspace"),
```

```
    scopeId: z.string().uuid().optional(),
  }))
  .mutation(async ({ input, ctx }) => {
    return prisma.calendarOptimizationRule.create({
      data: { ...input, createdBy: ctx.user.id },
    });
  }),
});
```

FEATURE 21: AI TIME TRACKING

tRPC Router: `src/server/routers/time-tracking.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const timeTrackingRouter = router({
  // Log a manual time entry
  logTime: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      taskId: z.string().uuid().optional(),
      description: z.string().optional(),
      durationMinutes: z.number().min(1),
      startedAt: z.string(),
      endedAt: z.string().optional(),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.timeEntry.create({
        data: {
          ...input,
          userId: ctx.user.id,
          startedAt: new Date(input.startedAt),
          endedAt: input.endedAt ? new Date(input.endedAt) : undefined,
          source: "manual",
        },
      });
    }),

  // Get time entries for a date range
  getEntries: protectedProcedure
```

```
  .input(z.object({
    workspaceId: z.string().uuid(),
    startDate: z.string(),
    endDate: z.string(),
    taskId: z.string().uuid().optional(),
  }))
  .query(async ({ input, ctx }) => {
    return prisma.timeEntry.findMany({
      where: {
        userId: ctx.user.id,
        startedAt: { gte: new Date(input.startDate), lte: new Date(input.endDate) },
        ...(input.taskId ? { taskId: input.taskId } : {}),
      },
      orderBy: { startedAt: "desc" },
    });
  }),

// Get time analytics (grouped by day, project, etc.)
getAnalytics: protectedProcedure
  .input(z.object({
    workspaceId: z.string().uuid(),
    startDate: z.string(),
    endDate: z.string(),
  }))
  .query(async ({ input, ctx }) => {
    const entries = await prisma.timeEntry.findMany({
      where: {
        userId: ctx.user.id,
        startedAt: { gte: new Date(input.startDate), lte: new Date(input.endDate) },
      },
      include: { task: { select: { title: true, databaseId: true } } },
    });

    // Group by day
    const byDay: Record<string, number> = {};
    for (const entry of entries) {
      const day = entry.startedAt.toISOString().split("T")[0];
      byDay[day] = (byDay[day] || 0) + entry.durationMinutes;
    }

    // Group by task
    const byTask: Record<string, { title: string; minutes: number }> = {};
    for (const entry of entries) {
      const key = entry.taskId || "no-task";
```

```
      if (!byTask[key]) byTask[key] = { title: entry.task?.title || entry.description || "Untracked",
minutes: 0 };
      byTask[key].minutes += entry.durationMinutes;
    }

    const totalMinutes = entries.reduce((sum, e) => sum + e.durationMinutes, 0);

    return { totalMinutes, byDay, byTask: Object.values(byTask).sort((a, b) => b.minutes -
a.minutes), entryCount: entries.length };
  }),

 // Get estimation accuracy (predicted vs actual)
 getEstimationAccuracy: protectedProcedure
  .input(z.object({ workspaceId: z.string().uuid() }))
  .query(async ({ input, ctx }) => {
    const estimates = await prisma.taskDurationEstimate.findMany({
      where: { workspaceId: input.workspaceId, actualMinutes: { not: null } },
      take: 50,
      orderBy: { createdAt: "desc" },
    });

    const accuracy = estimates.map(e => ({
      estimated: e.estimatedMinutes,
      actual: e.actualMinutes!,
      error: Math.abs(e.estimatedMinutes - e.actualMinutes!) / e.actualMinutes! * 100,
    }));

    const avgError = accuracy.length > 0 ? accuracy.reduce((sum, a) => sum + a.error, 0) /
accuracy.length : 0;

    return { estimates: accuracy, averageErrorPercent: Math.round(avgError), totalEstimates:
accuracy.length };
  }),
});
```

VERIFICATION — SPRINT 8

□ Create company goal → appears in goal hierarchy
□ Add 3 key results → progress bars shown
□ Update key result value → goal progress auto-recalculates
□ "Suggest Key Results" → AI generates relevant suggestions
□ "Assess Risk" → AI risk analysis with recommendations
□ Find meeting time → top 5 suggestions with scores

□ Create "No Meeting Wednesday 10-12" rule → saved
□ Log time manually → entry appears in time tracking
□ View time analytics → daily breakdown chart, by-task breakdown
□ Estimation accuracy view shows predicted vs actual comparison

---

SPRINT 9 — NOTIFICATIONS + TEMPLATES + VOICE

Features 20, 17, and 18

FEATURE 20: AI SMART NOTIFICATIONS

tRPC Router: `src/server/routers/notifications.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const notificationsRouter = router({
  // Get notifications (filtered by priority)
  getNotifications: protectedProcedure
    .input(z.object({
      priority: z.enum(["urgent", "important", "normal", "low", "all"]).default("all"),
      unreadOnly: z.boolean().default(false),
      limit: z.number().default(50),
    }))
    .query(async ({ input, ctx }) => {
      return prisma.notificationItem.findMany({
        where: {
          userId: ctx.user.id,
          ...(input.priority !== "all" ? { aiPriority: input.priority } : {}),
          ...(input.unreadOnly ? { isRead: false } : {}),
        },
        orderBy: [{ aiPriority: "asc" }, { createdAt: "desc" }],
        take: input.limit,
```

```
    });
  }),

// Mark as read
markRead: protectedProcedure
  .input(z.object({ ids: z.array(z.string().uuid()) }))
  .mutation(async ({ input }) => {
    await prisma.notificationItem.updateMany({
      where: { id: { in: input.ids } },
      data: { isRead: true, readAt: new Date() },
    });
  }),

// Get/update notification preferences
getPreferences: protectedProcedure
  .query(async ({ ctx }) => {
    return prisma.notificationPreference.upsert({
      where: { userId: ctx.user.id },
      create: { userId: ctx.user.id, workspaceId: "" }, // will be set on first real save
      update: {},
    });
  }),

updatePreferences: protectedProcedure
  .input(z.object({
    alwaysNotify: z.array(z.string()).optional(),
    batchNotify: z.array(z.string()).optional(),
    muteNotify: z.array(z.string()).optional(),
    batchFrequency: z.string().optional(),
    batchTimes: z.array(z.string()).optional(),
    focusModeAction: z.string().optional(),
    aiTriageEnabled: z.boolean().optional(),
  }))
  .mutation(async ({ input, ctx }) => {
    return prisma.notificationPreference.update({
      where: { userId: ctx.user.id },
      data: input,
    });
  }),

// Get unread count by priority
getUnreadCounts: protectedProcedure
  .query(async ({ ctx }) => {
    const counts = await prisma.notificationItem.groupBy({
```

```
      by: ["aiPriority"],
      where: { userId: ctx.user.id, isRead: false },
      _count: true,
    });
    return Object.fromEntries(counts.map(c => [c.aiPriority, c._count]));
  }),
});
```

FEATURE 17: AI TEMPLATE ENGINE

tRPC Router: `src/server/routers/templates.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const templatesRouter = router({
  list: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid().optional(),
      category: z.string().optional(),
      isPublic: z.boolean().optional(),
    }))
    .query(async ({ input }) => {
      return prisma.template.findMany({
        where: {
          deletedAt: null,
          ...(input.category ? { category: input.category } : {}),
          OR: [
            { isPublic: true },
            ...(input.workspaceId ? [{ workspaceId: input.workspaceId }] : []),
          ],
        },
        orderBy: { usageCount: "desc" },
      });
    }),

  // AI generate template from description
  generateFromDescription: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
```

```
      description: z.string(),
      type: z.enum(["page", "database", "workspace"]).default("database"),
    }))
    .mutation(async ({ input, ctx }) => {
      const result = await aiService.generateJSON<{
        name: string; description: string; category: string;
        content: any; // structure depends on type
      }>(
        `Create a ${input.type} template for: "${input.description}"

${input.type === "database" ? `Return: { name, description, category, content: { properties:
[{name, type, options?}], views: [{name, type, groupBy?}], sampleRows: [{}] } }` : ""}
${input.type === "page" ? `Return: { name, description, category, content: { blocks: [{type:
'heading'|'paragraph'|'list'|'callout', content: string}] } }` : ""}

Categories: project_management, personal, marketing, engineering, hr, sales, design, custom`,
        { systemPrompt: "Create practical, complete templates.", temperature: 0.5 },
      );

      return prisma.template.create({
        data: {
          workspaceId: input.workspaceId,
          name: result.name,
          description: result.description,
          category: result.category,
          type: input.type,
          content: result.content,
          createdBy: ctx.user.id,
        },
      });
    }),

  // Use a template (increment usage count)
  useTemplate: protectedProcedure
    .input(z.object({ templateId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.template.update({
        where: { id: input.templateId },
        data: { usageCount: { increment: 1 } },
      });
    }),

  // AI recommend templates based on user role/workspace
  getRecommendations: protectedProcedure
```

```
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.template.findMany({
        where: { isPublic: true },
        orderBy: { usageCount: "desc" },
        take: 10,
      });
    }),
});
```

FEATURE 18: AI VOICE COMMANDS & NOTES

tRPC Router: `src/server/routers/voice.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";
import OpenAI from "openai";

const openai = new OpenAI({ apiKey: process.env.OPENAI_API_KEY });

export const voiceRouter = router({
  // Transcribe audio
  transcribe: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      audioUrl: z.string(),
    }))
    .mutation(async ({ input, ctx }) => {
      const note = await prisma.voiceNote.create({
        data: {
          userId: ctx.user.id,
          workspaceId: input.workspaceId,
          audioUrl: input.audioUrl,
          durationSecs: 0,
          status: "transcribing",
        },
      });

      try {
        const audioResponse = await fetch(input.audioUrl);
```

```
      const audioBuffer = await audioResponse.arrayBuffer();
      const audioFile = new File([audioBuffer], "voice.webm", { type: "audio/webm" });

      const transcription = await openai.audio.transcriptions.create({
        model: "whisper-1",
        file: audioFile,
      });

      const summary = await aiService.generateText(
        `Summarize this voice note in 1-2 sentences:\\n"${transcription.text}"`,
        { systemPrompt: "Summarize voice notes concisely.", maxTokens: 100, temperature: 0.3
},
      );

      await prisma.voiceNote.update({
        where: { id: note.id },
        data: { transcript: transcription.text, aiSummary: summary.text, status: "completed" },
      });

      return { id: note.id, transcript: transcription.text, summary: summary.text };
    } catch (error: any) {
      await prisma.voiceNote.update({
        where: { id: note.id },
        data: { status: "failed" },
      });
      throw error;
    }
  }),

  // Convert voice note to task
  convertToTask: protectedProcedure
    .input(z.object({ voiceNoteId: z.string().uuid(), workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const note = await prisma.voiceNote.findUnique({ where: { id: input.voiceNoteId } });
      if (!note || !note.transcript) throw new Error("Voice note not found or not transcribed");

      const parsed = await aiService.generateJSON<{ title: string; description?: string; dueDate?:
string; priority?: string }>(
        `Parse this voice note into a task:\\n"${note.transcript}"\\nReturn: { title, description?,
dueDate?, priority? }`,
        { systemPrompt: "Parse voice into structured task.", temperature: 0.2 },
      );

      const task = await prisma.task.create({
```

```
        data: { ...parsed, workspaceId: input.workspaceId, createdBy: ctx.user.id, dueDate:
parsed.dueDate ? new Date(parsed.dueDate) : undefined },
      });

      await prisma.voiceNote.update({
        where: { id: input.voiceNoteId },
        data: { convertedTo: "task", convertedId: task.id },
      });

      return task;
    }),

  // List voice notes
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid(), limit: z.number().default(20) }))
    .query(async ({ input, ctx }) => {
      return prisma.voiceNote.findMany({
        where: { userId: ctx.user.id, workspaceId: input.workspaceId },
        orderBy: { createdAt: "desc" },
        take: input.limit,
      });
    }),
});
```

VERIFICATION — SPRINT 9

□ Notification inbox shows items grouped by priority (urgent/important/normal/low)
□ Mark notifications as read → disappear from unread view
□ Notification preferences saved and respected
□ Template gallery displays templates sorted by usage
□ "Generate from description" → AI creates full template → preview shown
□ Apply template → creates page/database with template content
□ Record voice note → transcription appears → summary shown
□ Convert voice note to task → task created with parsed fields
□ Voice notes list shows all recordings with transcripts

SPRINT 10 — DASHBOARD + TEAM INSIGHTS + INTEGRATIONS + ONBOARDING

Features 23, 22, 24, and 25

FEATURE 23: AI PERSONAL COMMAND CENTER

tRPC Router: `src/server/routers/command-center.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const commandCenterRouter = router({
  // Get all dashboard data in one call (minimize API calls)
  getDashboard: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input, ctx }) => {
      const today = new Date();
      const todayStr = today.toISOString().split("T")[0];

      const [dailyPlan, habits, goals, notifications, meetings, recentActivity, focusSession,
productivityScore] = await Promise.all([
        // Today's plan
        prisma.dailyPlan.findUnique({ where: { userId_date: { userId: ctx.user.id, date: new
Date(todayStr) } } }),
        // Active habits with today's completion status
        prisma.habit.findMany({
          where: { userId: ctx.user.id, isActive: true, deletedAt: null },
          include: { completions: { where: { date: new Date(todayStr) } } },
        }),
        // Active goals
        prisma.goal.findMany({
          where: { workspaceId: input.workspaceId, ownerId: ctx.user.id, deletedAt: null, status: {
not: "completed" } },
```

```
        include: { keyResults: true },
        take: 5,
      }),
      // Urgent + important notifications
      prisma.notificationItem.findMany({
        where: { userId: ctx.user.id, isRead: false, aiPriority: { in: ["urgent", "important"] } },
        take: 10,
        orderBy: { createdAt: "desc" },
      }),
      // Today's meetings
      prisma.meeting.findMany({
        where: {
          participants: { some: { userId: ctx.user.id } },
          startTime: { gte: new Date(`${todayStr}T00:00:00`), lt: new Date(`${todayStr}T23:59:59`)
},
        },
        orderBy: { startTime: "asc" },
      }),
      // Recent activity in workspace
      prisma.task.findMany({
        where: { workspaceId: input.workspaceId, updatedAt: { gte: new Date(Date.now() -
86400000) } },
        include: { assignee: { select: { name: true } } },
        take: 10,
        orderBy: { updatedAt: "desc" },
      }),
      // Active focus session
      prisma.focusSession.findFirst({ where: { userId: ctx.user.id, status: { in: ["active", "paused"]
} } }),
      // Today's productivity score
      prisma.productivityScore.findUnique({ where: { userId_date: { userId: ctx.user.id, date: new
Date(todayStr) } } }),
    ]);

    const planBlocks = (dailyPlan?.timeBlocks as any[]) || [];
    const completedBlocks = planBlocks.filter((b: any) => b.isCompleted).length;
    const totalBlocks = planBlocks.filter((b: any) => !["break", "buffer"].includes(b.type)).length;

    return {
      dailyPlan: dailyPlan ? {
        blocksCount: totalBlocks,
        completedCount: completedBlocks,
        nextBlock: planBlocks.find((b: any) => !b.isCompleted && !b.isSkipped),
        summary: dailyPlan.aiSummary,
```

```
        } : null,
        habits: habits.map(h => ({
          id: h.id, name: h.name, icon: h.icon, streak: h.streakCurrent,
          completedToday: h.completions.length > 0,
        })),
        goals: goals.map(g => ({
          id: g.id, title: g.title, progress: g.progressPercent, status: g.status,
        })),
        notifications: { urgentCount: notifications.filter(n => n.aiPriority === "urgent").length,
importantCount: notifications.filter(n => n.aiPriority === "important").length, items: notifications },
        meetings: meetings.map(m => ({ id: m.id, title: m.title, startTime: m.startTime })),
        recentActivity: recentActivity.map(t => ({ id: t.id, title: t.title, assignee: t.assignee?.name,
updatedAt: t.updatedAt })),
        focusSession,
        productivityScore,
      };
    }),

  // Get AI suggestion for "what to do next"
  getAiSuggestion: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      const todayStr = new Date().toISOString().split("T")[0];
      const plan = await prisma.dailyPlan.findUnique({ where: { userId_date: { userId: ctx.user.id,
date: new Date(todayStr) } } });
      const nextBlock = plan ? (plan.timeBlocks as any[]).find((b: any) => !b.isCompleted &&
!b.isSkipped) : null;

      const overdueTasks = await prisma.task.findMany({
        where: { assigneeId: ctx.user.id, dueDate: { lt: new Date() }, completedAt: null, deletedAt:
null },
        take: 3,
      });

      const result = await aiService.generateText(
        `What should this user do next?
Next planned block: ${nextBlock ? `"${nextBlock.title}" at ${nextBlock.startTime}` : "No plan"}
Overdue tasks: ${overdueTasks.map(t => t.title).join(", ") || "None"}
Current time: ${new Date().toLocaleTimeString()}

Give ONE specific, actionable suggestion in 1-2 sentences.`,
        { systemPrompt: "Give specific next-action suggestions.", maxTokens: 150, temperature:
0.5 },
      );
```

```
    return { suggestion: result.text };
  }),
});
```

Command Center UI: `src/app/(dashboard)/[workspaceId]/command-center/page.tsx`

Build the dashboard exactly as specified in the overview:

- Top: Greeting with date
- 2x2 grid: Today's Plan card, Priority Inbox card, Streaks & Habits card, Goals card
- AI Suggestion card (full width)
- Bottom row: Upcoming meetings, Recent activity

FEATURE 22: AI TEAM COLLABORATION INSIGHTS

tRPC Router: `src/server/routers/team-insights.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const teamInsightsRouter = router({
  // Get team workload dashboard
  getTeamWorkload: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      const members = await prisma.workspaceMember.findMany({
        where: { workspaceId: input.workspaceId },
        include: { user: { select: { id: true, name: true, email: true } } },
      });

      const memberStats = await Promise.all(members.map(async (m) => {
        const activeTasks = await prisma.task.count({ where: { assigneeId: m.userId, completedAt:
null, deletedAt: null } });
        const overdueTasks = await prisma.task.count({ where: { assigneeId: m.userId, dueDate: {
lt: new Date() }, completedAt: null, deletedAt: null } });
        const todayScore = await prisma.productivityScore.findUnique({ where: { userId_date: {
userId: m.userId, date: new Date(new Date().toISOString().split("T")[0]) } } });
```

```
      const workloadScore = activeTasks > 12 ? "red" : activeTasks > 8 ? "yellow" : "green";

      return {
        userId: m.userId, name: m.user.name,
        activeTasks, overdueTasks,
        focusMinutes: todayScore?.focusMinutes || 0,
        meetingMinutes: todayScore?.meetingMinutes || 0,
        workloadScore,
      };
    }));

    return { members: memberStats };
  }),

  // Get AI team insights
  getInsights: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input }) => {
      const snapshot = await prisma.teamInsightSnapshot.findFirst({
        where: { workspaceId: input.workspaceId },
        orderBy: { date: "desc" },
      });

      if (!snapshot) return { insights: "Not enough data yet. Team insights will appear after a few
days of usage." };

      const result = await aiService.generateText(
        `Analyze this team data and provide 3 actionable insights:
${JSON.stringify(snapshot.memberStats)}
${JSON.stringify(snapshot.teamMetrics)}`,
        { systemPrompt: "Provide actionable team productivity insights. Be specific.", maxTokens:
500, temperature: 0.4 },
      );
      return { insights: result.text };
    }),
});
```

FEATURE 24: AI INTEGRATION HUB

tRPC Router: `src/server/routers/integrations.ts`

```
import { z } from "zod";
```

```
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";

export const integrationsRouter = router({
  list: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .query(async ({ input }) => {
      return prisma.externalIntegration.findMany({ where: { workspaceId: input.workspaceId } });
    }),

  connect: protectedProcedure
    .input(z.object({
      workspaceId: z.string().uuid(),
      type: z.enum(["google_drive", "github", "gitlab", "jira", "linear", "slack", "figma",
"confluence"]),
      displayName: z.string(),
      accessToken: z.string(),
      refreshToken: z.string().optional(),
      scopes: z.array(z.string()),
    }))
    .mutation(async ({ input, ctx }) => {
      return prisma.externalIntegration.create({
        data: {
          ...input,
          accessTokenEncrypted: input.accessToken, // TODO: encrypt in production
          refreshTokenEncrypted: input.refreshToken,
          createdBy: ctx.user.id,
        },
      });
    }),

  disconnect: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
      return prisma.externalIntegration.update({
        where: { id: input.id },
        data: { syncEnabled: false, syncStatus: "disconnected" },
      });
    }),

  // Trigger manual sync
  syncNow: protectedProcedure
    .input(z.object({ id: z.string().uuid() }))
    .mutation(async ({ input }) => {
```

```
      await prisma.externalIntegration.update({
        where: { id: input.id },
        data: { syncStatus: "syncing" },
      });
      // Enqueue sync job
      return { status: "syncing" };
    }),
});
```

FEATURE 25: AI ONBOARDING ASSISTANT

tRPC Router: `src/server/routers/onboarding.ts`

```
import { z } from "zod";
import { router, protectedProcedure } from "@/lib/trpc";
import { prisma } from "@/lib/prisma";
import { aiService } from "@/lib/ai/ai-service";

export const onboardingRouter = router({
  getProgress: protectedProcedure
    .query(async ({ ctx }) => {
      return prisma.onboardingProgress.upsert({
        where: { userId: ctx.user.id },
        create: { userId: ctx.user.id, workspaceId: "" },
        update: {},
      });
    }),

  completeStep: protectedProcedure
    .input(z.object({ step: z.string() }))
    .mutation(async ({ input, ctx }) => {
      const progress = await prisma.onboardingProgress.findUnique({ where: { userId: ctx.user.id
} });
      if (!progress) throw new Error("Progress not found");

      const steps = [...progress.completedSteps, input.step];
      const TOTAL_STEPS = ["welcome", "profile", "first_task", "first_page", "join_channel",
"first_habit", "ai_search", "daily_plan"];
      const isComplete = TOTAL_STEPS.every(s => steps.includes(s));

      return prisma.onboardingProgress.update({
        where: { userId: ctx.user.id },
```

```
      data: { completedSteps: steps, isComplete },
    });
  }),

  dismiss: protectedProcedure
    .mutation(async ({ ctx }) => {
      return prisma.onboardingProgress.update({
        where: { userId: ctx.user.id },
        data: { dismissedAt: new Date() },
      });
    }),

  // AI answers "how do I..." questions
  askHelp: protectedProcedure
    .input(z.object({ question: z.string() }))
    .mutation(async ({ input }) => {
      const result = await aiService.generateText(
        `Answer this user question about the productivity platform:
"${input.question}"

Provide a clear, step-by-step answer. If it involves a specific feature, explain how to access it
and use it.`,
        { systemPrompt: "You are a helpful onboarding assistant for a productivity platform. Give
clear, step-by-step instructions.", maxTokens: 500, temperature: 0.4 },
      );
      return { answer: result.text };
    }),

  // AI suggests unused features
  getFeatureSuggestions: protectedProcedure
    .input(z.object({ workspaceId: z.string().uuid() }))
    .mutation(async ({ input, ctx }) => {
      // Check which features the user has used
      const hasUsedWriting = await prisma.aiWritingHistory.count({ where: { userId: ctx.user.id } })
> 0;
      const hasUsedSearch = await prisma.knowledgeQuery.count({ where: { userId: ctx.user.id }
}) > 0;
      const hasUsedPlan = await prisma.dailyPlan.count({ where: { userId: ctx.user.id } }) > 0;
      const hasUsedFocus = await prisma.focusSession.count({ where: { userId: ctx.user.id } }) >
0;
      const hasUsedHabits = await prisma.habit.count({ where: { userId: ctx.user.id } }) > 0;

      const suggestions: Array<{ feature: string; reason: string }> = [];
```

```
    if (!hasUsedWriting) suggestions.push({ feature: "AI Writing", reason: "Press Space on empty lines to generate content with AI" });
    if (!hasUsedSearch) suggestions.push({ feature: "AI Search", reason: "Press Cmd+K to search everything or ask AI questions" });
    if (!hasUsedPlan) suggestions.push({ feature: "Daily Planner", reason: "AI can auto-schedule your day based on tasks and meetings" });
    if (!hasUsedFocus) suggestions.push({ feature: "Focus Sessions", reason: "Start a focus timer to track time and block notifications" });
    if (!hasUsedHabits) suggestions.push({ feature: "Habits", reason: "Create recurring habits and track streaks" });

    return { suggestions };
  }),
});
```

VERIFICATION — SPRINT 10

□ Command Center dashboard loads with all widgets populated
□ "What should I do next?" AI suggestion is specific and actionable
□ Team workload shows all members with workload indicators (green/yellow/red)
□ Team insights provides AI analysis
□ Integration hub lists available connections
□ Connect a test integration → shows in list with sync status
□ Onboarding progress tracks completed steps
□ "How do I create a habit?" → AI provides step-by-step answer
□ Feature suggestions show unused features

SPRINT 11 — INTEGRATION TESTING + POLISH

□ ROUTER MERGE: Ensure all 25 tRPC routers are merged in _app.ts
□ CROSS-FEATURE: Daily plan uses habits (Feature 14) + calendar (Feature 16) + tasks (Feature 6)
□ CROSS-FEATURE: Agents (Feature 4) can search knowledge (Feature 2) and create tasks (Feature 6)
□ CROSS-FEATURE: Meeting action items (Feature 7) create tasks in databases (Feature 6)
□ CROSS-FEATURE: Command Center (Feature 23) aggregates all feature data
□ CROSS-FEATURE: Focus sessions (Feature 9) auto-log time (Feature 21)
□ CROSS-FEATURE: Habit completions (Feature 14) update goal progress (Feature 15)

□ PERFORMANCE: All pages load < 2 seconds
□ PERFORMANCE: AI responses start streaming < 3 seconds
□ PERFORMANCE: Knowledge search returns results < 500ms
□ MOBILE: All pages responsive on 375px width
□ ERRORS: Every API has proper error handling (try/catch + user-friendly messages)
□ LOADING: Every async operation shows loading state
□ EMPTY: Every list/view shows appropriate empty state
□ KEYBOARD: Cmd+K (search), Cmd+J (AI), Cmd+Shift+D (daily plan), Cmd+N (new task)
□ TypeScript: npx tsc --noEmit → zero errors
□ README: Create/update README-AI-FEATURES.md documenting all 25 features

PROJECT COMPLETE.